

Chương 1. CÁC KHÁI NIỆM CƠ BẢN

❖ Đề Cương

- Các dạng biểu diễn trị số.
- Hệ thống kỹ thuật số và hệ thống kỹ thuật tương tự.
- Các hệ thống số trong kỹ thuật số.
- Biểu diễn các đại lượng nhị phân trong kỹ thuật.
- Mạch số.
- Truyền song song và truyền nối tiếp.
- Thuộc tính nhớ, mạch tổ hợp và mạch tuần tự.
- Máy tính kỹ thuật số.

❖ Mục Đích

Sau khi hoàn thành chương này, bạn phải nắm được kiến thức:

- Cách phân biệt dạng biểu thị tương tự và dạng biểu thị số.
- Cách so sánh ưu và khuyết giữa kỹ thuật số và kỹ thuật tương tự.
- Tầm quan trọng của bộ chuyển đổi tương tự – số (ADC) và bộ chuyển đổi số – tương tự (DAC).
- Khái niệm về một số hệ thống số.
- Giảm độ thời gian của tín hiệu số
- Cách phân biệt cách truyền song song và truyền nối tiếp.
- Cách phân biệt mạch tổ hợp và mạch tuần tự, thuộc tính nhớ của mạch tuần tự.
- Các thành phần chính của một máy tính.

❖ Các Thuật Ngữ Tiếng Anh:

- | | |
|---|----------------------------------|
| • Analog representation: | biểu diễn dạng tương tự. |
| • Digital representation: | biểu diễn dạng số. |
| • <u>A</u> nalog-to- <u>d</u> igital <u>c</u> onverter (ADC): | bộ biến đổi từ tương tự sang số. |
| • <u>D</u> igital-to- <u>a</u> nalog <u>c</u> onverter (DAC): | bộ biến đổi từ số sang tương tự. |
| • Decimal number system: | hệ thống số thập phân. |
| • Binary number system: | hệ thống số nhị phân. |
| • Octal number system: | hệ thống số bát phân. |
| • Hexadecimal number system: | hệ thống số thập lục phân. |
| • Bit (<u>b</u> inary <u>d</u> igit): | chữ số trong hệ nhị phân. |
| • Timing diagram: | giản đồ thời gian. |
| • Parallel: | song song. |
| • Serial: | nối tiếp. |
| • Microprocessor/ Microcontroller: | vi xử lý/ vi điều khiển. |
| • <u>M</u> ost <u>S</u> ignificant <u>B</u> it (MSB): | bit có trọng số lớn nhất |
| • <u>L</u> east <u>S</u> ignificant <u>B</u> it (LSB): | bit có trọng số nhỏ nhất |

1.1- Các Dạng Biểu Diễn Trị Số

Khi nghiên cứu một đối tượng bất kỳ, chúng ta có nhu cầu đo lường, tính toán số lượng, thể tích, khối lượng, cường độ, mật độ ... của đối tượng đó. Trong khoa học kỹ thuật, những thuộc tính đó được gọi chung là **đại lượng**. Trong thực tế, có hai cách biểu diễn đại lượng: biểu diễn dạng **tương tự** và biểu diễn dạng **số**.

1.1.1- Biểu diễn dạng tương tự

- Trong cách biểu diễn dạng tương tự, một đại lượng có thể được biểu diễn thành điện thế hay cường độ dòng điện, hay một số đo tương quan với giá trị của đại lượng đó.
- Ví dụ: đồng hồ đo tốc độ của xe hơi, đồng hồ đo VOM dùng kim, nhiệt kế, micro, biến trở ...
- Đặc điểm chính của cách biểu diễn dạng tương tự là dạng biểu diễn thay đổi theo một khoảng giá trị liên tục.
- Trong toán học, biểu diễn dạng tương tự là hàm số liên tục.
- Trong kỹ thuật điện tử, biểu diễn dạng tương tự là công nghệ đèn điện tử, vi mạch tương tự ...

1.1.2- Biểu diễn dạng số

- Trong cách biểu diễn dạng số, một đại lượng có thể được biểu diễn thành một dãy các ký số.
Ví dụ: đồng hồ đo VOM dùng số, nhiệt kế dùng số, số hạt cát trong một bình chứa ...
- Đặc điểm chính của cách biểu diễn dạng số là dạng biểu diễn thay đổi theo từng bước, gián đoạn, không liên tục.
- Trong toán học, biểu diễn dạng số là hàm số gián đoạn. Và toán học dùng trong dạng biểu diễn này được gọi là **toán rời rạc**.
- Trong kỹ thuật điện tử, biểu diễn dạng số là công nghệ vi mạch số ...

1.2- Hệ Thống Kỹ Thuật Số Và Hệ Thống Kỹ Thuật Tương Tự

1.2.1- Hệ thống kỹ thuật số

- Là một hệ thống gồm nhiều thiết bị kỹ thuật để lưu trữ, xử lý các đại lượng được biểu diễn dưới dạng số.
- Các thiết bị kỹ thuật dạng số thường là thiết bị điện tử, cơ khí, từ...

- Ví dụ: Máy tính, máy ảnh số, máy quay phim số, truyền hình số, đĩa nhạc Compact ...

1.2.2- Hệ thống kỹ thuật tương tự

- Là một hệ thống gồm nhiều thiết bị kỹ thuật để lưu trữ, xử lý các đại lượng được biểu diễn dưới dạng tương tự.
- Các thiết bị kỹ thuật dạng tương tự thường là thiết bị điện tử, cơ khí, từ ... đa số là thiết bị về điện tử.
- Ví dụ: máy ảnh dùng phim, máy quay phim dùng băng từ, truyền hình thông thường, đĩa than, loa, radio ...

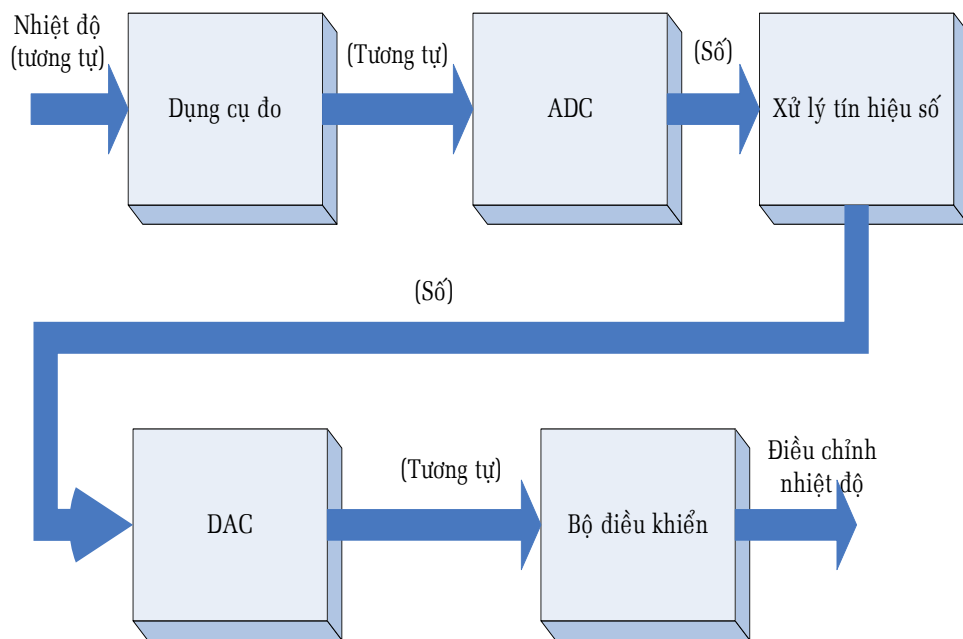
1.2.3- Ưu điểm của hệ thống kỹ thuật số so với kỹ thuật tương tự

Càng ngày, càng nhiều lãnh vực dùng kỹ thuật tương tự chuyển sang dùng kỹ thuật số như công nghệ ghi âm, công nghệ máy ảnh, máy quay phim, công nghệ truyền hình... do những ưu điểm sau:

- Dễ thiết kế hơn.
- Thông tin được lưu trữ dễ dàng.
- Độ chính xác và độ tin cậy cao hơn.
- Dễ tự động hoá hơn.
- Ảnh hưởng do nhiễu thấp hơn.
- Khả năng tích hợp càng ngày càng cao, và kích thước càng ngày càng nhỏ gọn hơn.

1.2.4- Nhược điểm của hệ thống kỹ thuật số so với kỹ thuật tương tự

Tuy nhiên, do **thế giới thực là thế giới tương tự** nên để áp dụng được hệ thống kỹ thuật số, ta phải chuyển đổi đại lượng ngõ nhập thành dạng số, sau khi xử lý xong lại chuyển thành dạng tương tự như sơ đồ khối ở hình 1:



Hình 1. Sơ đồ khối của một hệ thống dùng ADC và DAC

- Như vậy, vấn đề là bộ chuyển đổi tương tự – số (ADC) và bộ chuyển đổi số – tương tự (DAC), hai bộ chuyển đổi này phải đạt yêu cầu là việc chuyển đổi càng trung thực càng tốt.
- Tùy thuộc vào từng yêu cầu cụ thể của hệ thống, chúng ta có thể sử dụng cả hai loại kỹ thuật này, thành phần nào dùng kỹ thuật tương tự, thành phần nào dùng kỹ thuật số.

1.3- Các Hệ Thống Số Trong Kỹ Thuật Số

Có nhiều hệ thống số được dùng trong kỹ thuật số, trong đó có hệ thống số thập phân, hệ thống số nhị phân, hệ thống số bát phân và hệ thống số thập lục phân.

1.3.1- Cách biểu diễn một hệ thống số đếm

- Những hệ thống số đếm nói trên đều dựa trên vị trí của từng chữ số, vì giá trị của một chữ số phụ thuộc vào vị trí của nó, theo nguyên tắc ở bảng sau:

e^4	e^3	e^2	e^1	e^0	Dấu •	e^{-1}	e^{-2}	e^{-3}	e^{-4}	e^{-5}
		a_2	a_1	a_0						
MSD				LSD						
a_4	a_3	a_2	a_1	a_0	•	a_{-1}	a_{-2}	a_{-3}	a_{-4}	a_{-5}
MSD										LSD

Bảng 1. Quy tắc biểu diễn một hệ thống đếm cơ số e

- Ở đây có hai số $a_2a_1a_0$ và $a_4a_3a_2a_1a_0.a_{-1}a_{-2}a_{-3}a_{-4}a_{-5}$ của một hệ thống đếm cơ số e , với $e \in \mathbb{N}$, $a_i \in \{0,1,2,\dots,e-1\}$.
- Số x_1 được viết là $a_2a_1a_0$ có nghĩa là $x_1 = a_2.e^2 + a_1.e^1 + a_0.e^0$
- Số x_2 được viết là $a_4a_3a_2a_1a_0.a_{-1}a_{-2}a_{-3}a_{-4}a_{-5}$ có nghĩa là $x_2 = a_4.e^4 + a_3.e^3 + a_2.e^2 + a_1.e^1 + a_0.e^0 + a_{-1}.e^{-1} + a_{-2}.e^{-2} + a_{-3}.e^{-3} + a_{-4}.e^{-4} + a_{-5}.e^{-5}$
- Một hệ thống đếm cơ số e có N chữ số sẽ có e^N số bắt đầu từ số 0 đến số e^N-1 .
- Chữ số nằm tận cùng bên phải là chữ số ở vị trí nhỏ nhất, được gọi tắt theo tiếng Anh là LSD, chữ số nằm tận cùng bên tay trái là chữ số ở vị trí lớn nhất, được gọi tắt theo tiếng Anh là MSD.
Số x_1 có a_2 là MSD và a_0 là LSD.
Số x_2 có a_4 là MSD và a_{-5} là LSD.
- Trọng số: là giá trị của e^{N-1} tại vị trí N của phần nguyên, và bằng e^{-N} tại vị trí $-N$ của phần định trị.

1.3.2- Hệ thống số thập phân (hệ thống đếm cơ số 10)

- Hệ thống số thập phân có 10 chữ số là 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- Hệ thống số thập phân là hệ thống số đếm theo vị trí với cơ số $e=10$.
- Để phân biệt với các hệ thống số khác, ta thêm chữ D nằm sau LSD để hiểu đây là hệ thống số thập phân.
- Cho 2 số 276_D và 9563.18_D , ta có kết quả cụ thể theo bảng sau:

...	10 ³	10 ²	10 ¹	10 ⁰	Dấu ●	10 ⁻¹	10 ⁻²	10 ⁻³	10 ⁻⁴	...
2		7	6 _D							
MSD				LSD						
9	5	6	3	●	1	8 _D				
MSD				LSD						
Trọng số	1000	100	10	1		0.1	0.01	0.001	0.0001	...

Bảng 2. Quy tắc biểu diễn một hệ thống thập phân

Vậy $276_D = 2 \times 10^2 + 7 \times 10^1 + 6 \times 10^0$ trong đó chữ số 2 là MSD và chữ số 6 là LSD.

Và $9563.18_D = 9 \times 10^3 + 5 \times 10^2 + 6 \times 10^1 + 3 \times 10^0 + 1 \times 10^{-1} + 8 \times 10^{-2}$ trong đó chữ số 9 là MSD và chữ số 8 là LSD.

- Trọng số lần lượt là ... 1000, 100, 10, 1 cho phần nguyên, và 0.1, 0.01, 0.001 ... cho phần định trị.
- Đặc điểm của hệ thống số thập phân là với N chữ số ta có thể đếm được 10^N số khác nhau, từ 0 đến 10^N-1 .
Ví dụ: với 2 chữ số có thể đếm được một trăm số từ 0_D đến 99_D (10^2-1), với ba chữ số có thể đếm được một ngàn số từ 0_D đến 999_D (10^3-1).

1.3.3- Hệ thống số nhị phân (hệ thống đếm cơ số 2)

- Hệ thống số nhị phân có 2 chữ số là 0, 1 và chữ số nhị phân được gọi tắt theo tiếng Anh là **bit**.
- Hệ thống số nhị phân là hệ thống số đếm theo vị trí với cơ số $e=2$.
- Riêng trong hệ thống số nhị phân, do chữ số nhị phân được gọi **bit**, nên bit nằm tận cùng bên phải là chữ số ở vị trí nhỏ nhất, được gọi tắt theo tiếng Anh là LSB, bit nằm tận cùng bên tay trái là bit ở vị trí lớn nhất, được gọi tắt theo tiếng Anh là MSB.
- Để phân biệt với các hệ thống số khác, ta thêm chữ B nằm sau LSB để hiểu đây là hệ thống số nhị phân.

- Cho 2 số 101_B và 1011.11_B ta có kết quả cụ thể theo bảng sau:

...	2^3	2^2	2^1	2^0	Dấu •	2^{-1}	2^{-2}	2^{-3}	2^{-4}	...
1		0		1 _B						
MSB				LSB						
1	0	1	1	•	1	1 _B				
MSB				LSB						
Trọng số	8	4	2	1		0.5	0.25	0.125	0.0625	...

Bảng 3. Quy tắc biểu diễn một hệ thống nhĩ phân

Vậy $101_B = 1x2^2 + 0x2^1 + 1x2^0 = 4 + 1 = 5_D$. Và $1011.11_B = 1x2^3 + 0x2^2 + 1x2^1 + 1x2^0 + 1x2^{-1} + 1x2^{-2} = 8 + 2 + 1 + 0.5 + 0.25 = 11.75_D$.

- Trọng số lần lượt là ... 8, 4, 2, 1 ... cho phần nguyên, và 0.5, 0.25, 0.125 ... cho phần thập phân.

- Đặc điểm của hệ thống số nhị phân là với N bit ta có thể đếm được 2^N số khác nhau, từ 0 đến 2^N-1 .
Ví dụ: với 2 bit có thể đếm được từ 4 số từ 00_B đến 11_B (2^2-1), với ba bit có thể đếm được 8 số từ 000_B đến 111_B (2^3-1).

1.3.4- Hệ thống số bát phân (hệ thống đếm cơ số 8)

- Hệ thống số bát phân có 8 chữ số là 0, 1, 2, 3, 4, 5, 6, 7.
- Hệ thống số bát phân là hệ thống số đếm theo vị trí với cơ số $e=8$.
- Để phân biệt với các hệ thống số khác, ta thêm chữ O nằm sau LSD để hiểu đây là hệ thống số thập phân.
- Trọng số lần lượt là ... 512, 64, 8, 1 ... cho phần nguyên, và 0.125, 0.15625 ... cho phần định trị.
- Đặc điểm của hệ thống số bát phân là với N chữ số ta có thể đếm được 8^N chữ số khác nhau, từ 0 đến 8^N-1 .
Ví dụ: với 2 chữ số có thể đếm được một 64 số từ 00_O đến 77_O (8^2-1), với ba chữ số có thể đếm được 512 số từ 000_O đến 777_O (8^3-1).
- Cho 2 số 276_O và 1563.18_O , ta có kết quả cụ thể theo bảng sau:

$$\text{Vậy } 276_O = 2 \times 8^2 + 7 \times 8^1 + 6 \times 8^0 = 128 + 56 + 6 = 190_D. \text{ Và } 1563.18_O = 1 \times 8^3 + 5 \times 8^2 + 6 \times 8^1 + 3 \times 8^0 + 1 \times 8^{-1} + 8 \times 8^{-2} = 512 + 320 + 48 + 3 + 0.125 + 0.125 = 883.25_D.$$

...	8^3	8^2	8^1	8^0	Dấu●	8^{-1}	8^{-2}	8^{-3}	8^{-4}	...
	2	7	6							
	1	5	6	3	●	1	8			
	MSD					LSD				
Trọng số	512	64	8	1		0.125	0.015625	0.001953125	0.000244140625	...

Bảng 4. Quy tắc biểu diễn một hệ thống bát phân

1.3.5- Hệ thống số thập lục phân (hệ thống đếm cơ số 16)

- Hệ thống số thập lục phân có 16 chữ số là 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F. Với A = 10, B = 11, C = 12, D = 13, E = 14, F = 15 trong hệ thống số thập phân.
- Hệ thống số thập lục phân là hệ thống số đếm theo vị trí với cơ số $e=16$.

- Để phân biệt với các hệ thống số khác, ta thêm chữ H nằm sau LSD để hiểu đây là hệ thống số thập lục phân.
- Cho 2 số 276_H và $1A6F.08_H$, ta có kết quả cụ thể theo bảng sau:

...	16^3	16^2	16^1	16^0	Dấu •	16^{-1}	16^{-2}	...
	2	7		6_H				
	MSD			LSD				
	1	A	6	F	•	0	8_H	
	MSD			LSD				
Trọng số	4096	256	16	1		0.0625	0.00390625	...

Bảng 5. Quy tắc biểu diễn một hệ thống thập lục phân

Vậy $276_H = 2 \times 16^2 + 7 \times 16^1 + 6 \times 16^0 = 512 + 112 + 6 = 630_D$. Và $1A6F.08_H = 1 \times 16^3 + 10 \times 16^2 + 6 \times 16^1 + 15 \times 16^0 + 0 \times 16^{-1} + 8 \times 16^{-2} = 4096 + 2560 + 96 + 15 + 0.03125 = 6767.03125_D$.

- Trọng số lần lượt là ... 4096, 256, 16, 1 ... cho phần nguyên, và 0.0625, 0.00390625 ... cho phần định trị.
- Đặc điểm của hệ thống số thập lục phân là với N chữ số ta có thể đếm được 16^N chữ số khác nhau, từ 0 đến $16^N - 1$. Ví dụ: với 2 chữ số có thể đếm được một 256 số từ 0_H đến FF_H ($16^2 - 1$), với ba chữ số có thể đếm được 4096 số từ 0_H đến FFF_H ($16^3 - 1$).

1.3.6- Mối quan hệ giữa các hệ thống số

Mối quan hệ giữa các hệ thống số có thể minh họa qua bảng sau:

Số thập phân	Số nhị phân	Số bát phân	Số thập lục phân
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7

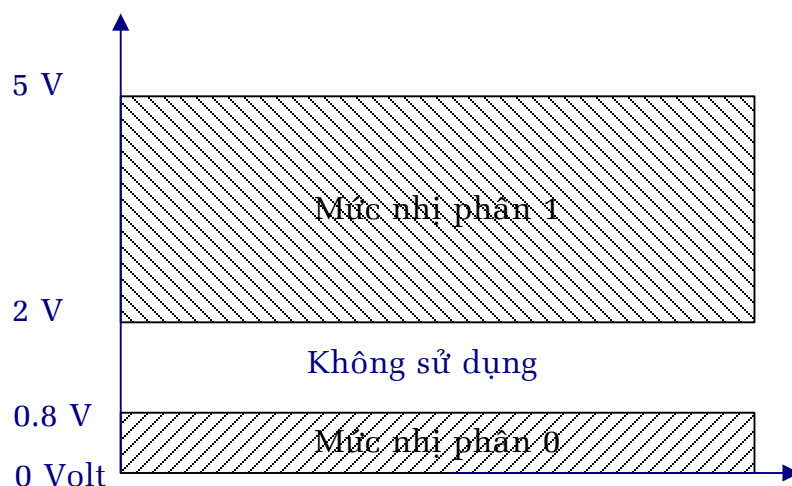
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Bảng 6. Quan hệ giữa các hệ thống số

1.4- Biểu Diễn Các Đại Lượng Nhị Phân Trong Kỹ Thuật

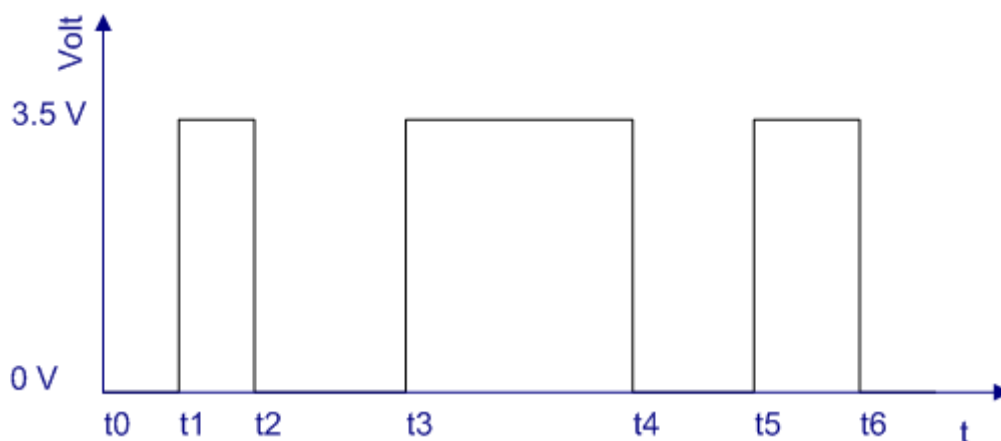
Trong kỹ thuật hiện thời, hầu hết các thiết bị hoạt động ở hai trạng thái như diod (dẫn điện/không dẫn), relay (ngắt/đóng), đĩa từ (từ hoá/khử từ)...Như vậy, thông tin về mặt vật lý được biểu diễn dưới dạng nhị phân phù hợp với bản chất thiết bị.

- Trong các thiết bị điện tử số, thông tin nhị phân được biểu diễn bằng mức điện thế. Tùy theo công nghệ chế tạo IC, như công nghệ lưỡng cực, họ TTL thì số 0 là mức điện thế từ 0 $\Rightarrow$ 0.8 Volt và được gọi là mức 0, số 1 là mức điện thế từ 2 $\Rightarrow$ 5 Volt và được gọi là mức 1, như công nghệ đơn cực, họ CMOS thì số 0 hay mức 0 là mức điện thế từ 0 Volt, số 1 hay mức 1 là mức điện thế từ 3 $\Rightarrow$ 15 Volt tùy theo điện áp cấp nguồn.
- Ở đây, ta nhắc lại sự khác biệt giữa hệ thống biểu diễn số và tương tự ở chỗ này. Nếu ở thiết bị số thì mức điện thế là 3 Volt hay 5 Volt đều là mức 1, trong khi đó trong thiết bị tương tự đây là sự khác biệt lớn. Như vậy, nếu gặp nhiều thì thiết bị số ít bị ảnh hưởng hơn thiết bị tương tự. Để thiết kế một mạch tương tự với độ chính xác cao là điều rất khó khăn so với thiết kế một mạch với khoảng điện thế cho mức 1 là khá lớn.



Hình 2. Mức điện thế họ TTL

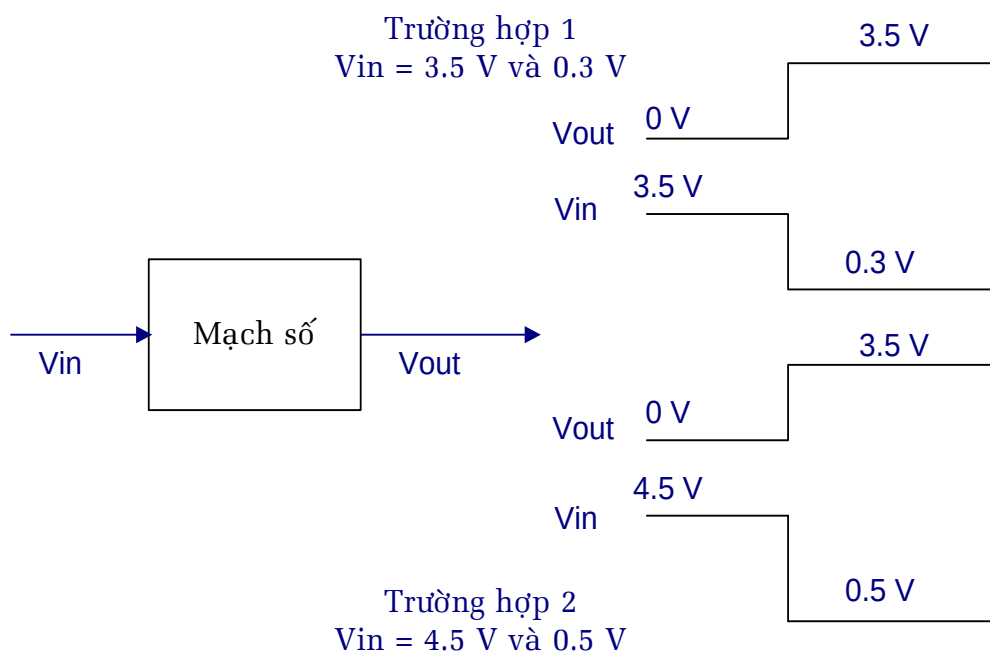
- Giản đồ thời gian của tín hiệu số là sơ đồ biểu diễn sự thay đổi trạng thái của tín hiệu theo thời gian như hình sau:



Hình 3. Giản đồ thời gian của tín hiệu số

- Về mặt lý thuyết, sự chuyển đổi trạng thái từ $0 \Rightarrow 1$ hay từ $1 \Rightarrow 0$ là tức thời, nên trên giản đồ thời gian ta nhận thấy là những đường vuông góc. Về mặt kỹ thuật, sự chuyển đổi trạng thái từ $0 \Rightarrow 1$ hay từ $1 \Rightarrow 0$ là có khoảng thời gian trễ, nên trên giản đồ thời gian là những đường không vuông góc. Nhưng nói chung thì thời gian trễ này là quá bé so với thời gian ở mức 0 hay mức 1 nên ta vẫn vẽ giống như lý thuyết, trừ những trường hợp cần xét cụ thể.
- Để quan sát giản đồ thời gian của tín hiệu số, ta dùng **dao động ký**.

1.5- Mạch Số



Hình 4. Mạch số đáp ứng theo mức 0 hay 1

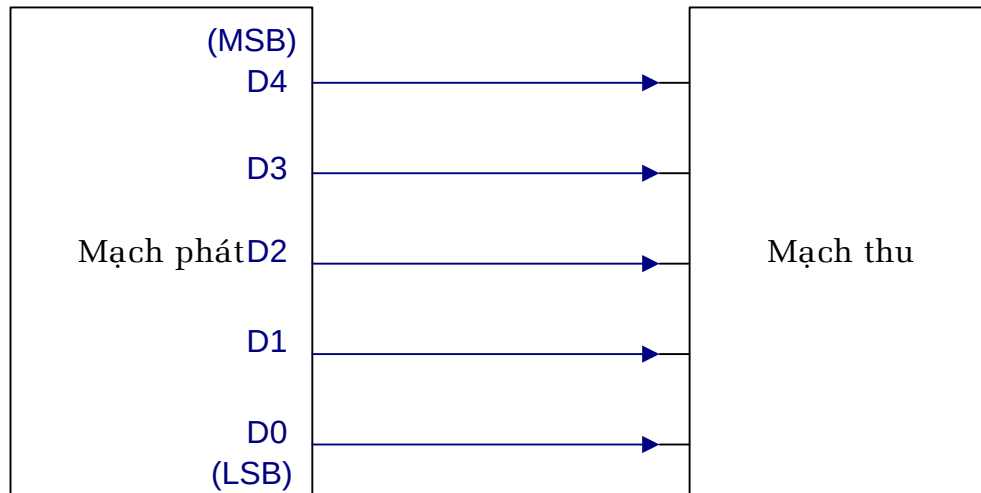
- Mạch số là mạch được thiết kế để ngõ xuất có mức điện thế là mức 0 hay mức 1 như đã nói ở phần 1.4, và ngõ nhập cũng được thiết kế sao cho mức điện thế là mức 0 hay mức 1 tương ứng như trên.
- Mạch Logic là là mạch số tuân theo quy tắc của đại số Bool.
- Vì mạch số là tích hợp nhiều mạch logic bên trong một chip. Tùy theo công nghệ chế tạo, ta có các họ IC như TTL, CMOS...

1.6- Truyền Song Song Và Truyền Nối Tiếp

Có hai phương pháp để truyền thông tin từ nơi này sang nơi khác, đó là phương pháp truyền song song và truyền nối tiếp.

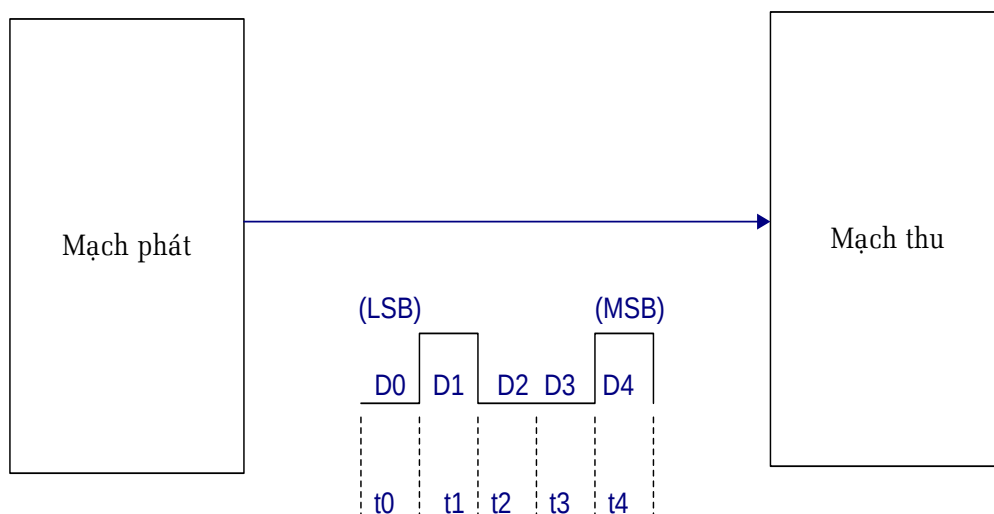
1.6.1- Truyền song song

Truyền mỗi bit mỗi đường truyền, và tất cả các bit đều được truyền đồng thời.



Hình 5. Truyền song song 5 bit

1.6.2- Truyền nối tiếp



Hình 6. Truyền nối tiếp 5 bit

- Truyền nối tiếp là chỉ sử dụng một đường truyền, và tuần tự truyền xong bit này đến bit khác.

1.6.3- So sánh hai cách truyền song song và nối tiếp

- Truyền song song thì tốc độ cao, nhưng tốn kém dây dẫn. Nên những thiết bị gần thì sử dụng cách truyền song song.
- Truyền nối tiếp tốc độ thấp nhưng chi phí thấp, thường được dùng trong kết nối có khoảng cách xa.

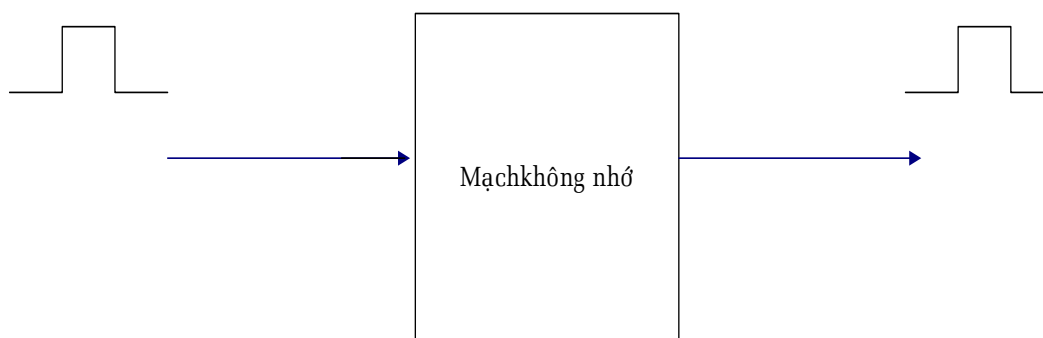
- Tuy nhiên, hiện tại một số chuẩn truyền nối tiếp mới có tốc độ rất cao không thua gì truyền song song như USB 2.0, IEEE1394, SerialATA..., nhưng chỉ dùng được trong khoảng cách gần.

1.7- Thuộc Tính Nhớ, Mạch Tổ Hợp và Mạch Tuần Tự

Trong hệ thống kỹ thuật số, có thể phân loại mạch số làm hai loại: **mạch có nhớ** (mạch tuần tự) và **mạch không nhớ** (mạch tổ hợp).

1.7.1- Mạch không nhớ (mạch tổ hợp)

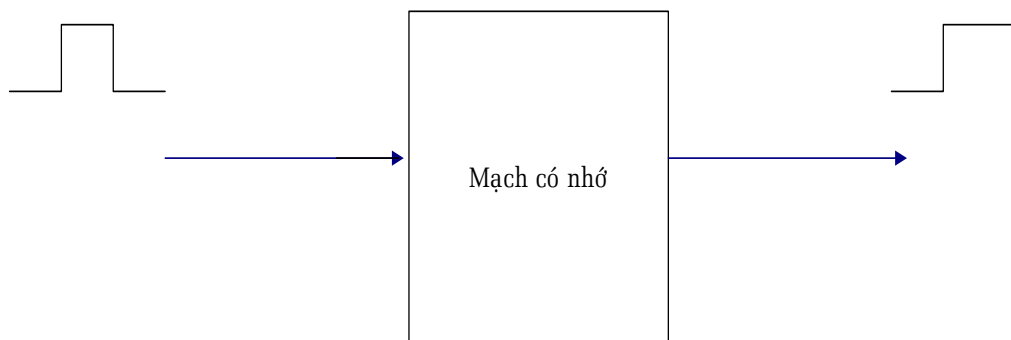
Mạch không nhớ là mạch mà ngõ xuất chỉ phụ thuộc vào trạng thái ngõ vào tại đúng thời điểm đó.



Hình 7. Mạch không nhớ

1.7.2- Mạch có nhớ (mạch tuần tự)

Mạch có nhớ là mạch có ngõ xuất không chỉ phụ thuộc vào trạng thái ngõ vào, mà còn phụ thuộc trạng thái bên trong trước đó của mạch.



Hình 8. Mạch có nhớ

1.8- Máy Tính Kỹ Thuật Số

1.8.1- Máy tính kỹ thuật số:

- Máy tính kỹ thuật số gọi tắt là máy tính hoạt động theo mô hình Von Neuman gồm ba phần chính:

- Bộ xử lý trung tâm (gọi tắt là CPU hay MPU), hay còn được gọi là bộ vi xử lý, bao gồm bộ xử lý các phép toán số học và luận lý, (gọi tắt là ALU) và bộ điều khiển.
- Bộ nhớ để lưu trữ chương trình và dữ liệu.
- Các thiết bị nhập/xuất.

1.8.2- Bộ vi điều khiển

- Bộ vi điều khiển thì tích hợp cả ba bộ phận là bộ xử lý trung tâm, bộ nhớ và thiết bị nhập/xuất.
- Bộ vi điều khiển được sử dụng rộng rãi trong việc điều khiển thiết bị chuyên dụng.

❖ Tóm Tắt Chương 1

- Tín hiệu tương tự: biên độ tín hiệu thay đổi liên tục theo thời gian.
- Tín hiệu số: biên độ tín hiệu gián đoạn theo thời gian.
- Các hệ thống số đếm, cách biểu diễn một hệ thống số đếm cơ số e. Một số
$$X_n = a_{n-1}...a_3a_2a_1a_0.a_{-1}a_{-2}a_{-3}...a_{-m-1}a_{-m} = a_{n-1}.e^{n-1} + ... + a_3.e^3 + a_2.e^2 + a_1.e^1 + a_0.e^0 + a_{-1}.e^{-1} + a_{-2}.e^{-2} + a_{-3}.e^{-3} + ... + a_{-m-1}.e^{-m-1} + a_{-m}.e^{-m}$$
- Hệ thống số thập phân: có 10 chữ số 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- Hệ thống số nhị phân: có 2 chữ số 0, 1 và thường được gọi là 2 bit 0, 1.
- Hệ thống số bát phân: có 8 chữ số 0, 1, 2, 3, 4, 5, 6, 7.
- Hệ thống số thập lục phân: có 16 chữ số 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.
- Trọng số: là giá trị của e^{N-1} tại vị trí N của phần nguyên, và bằng e^{-N} tại vị trí -N của phần định trị.
- Truyền song song N bit: truyền đồng thời N bit trên N đường truyền.
- Truyền tuần tự: truyền lần lượt bit này đến bit khác trên một đường truyền.
- Mạch số: chia làm hai loại là mạch tổ hợp và mạch tuần tự.

Chương 2. CÁC HỆ THỐNG SỐ ĐẾM VÀ MÃ

❖ Đề Cương

- Các phương pháp chuyển đổi giữa các hệ thống số đếm.
- Số nhị phân không dấu và các phép toán số học trên số nhị phân không dấu.
- Số nhị phân có dấu và các phép toán số học trên hệ thống số nhị phân
- Các bộ mã BCD và các phép toán cộng, trừ trên mã NBCD.
- Các loại mã khác: mã ASCII, mã quá 3, mã quá 6, mã Gray.
- Kiểm tra lỗi bằng phương pháp chẵn lẻ.

❖ Mục Đích

Sau khi hoàn thành chương này, bạn phải nắm được kiến thức:

- Các phương pháp chuyển đổi từ một hệ thống số đếm này sang hệ thống số đếm khác.
- Cách biểu diễn số nhị phân có dấu và không dấu trong hệ thống kỹ thuật số.
- Các phép toán cộng, trừ, nhân, chia trên số nhị phân có dấu.
- Các bộ mã BCD và phép toán cộng trên mã NBCD.
- Một số loại mã thông dụng trong kỹ thuật và ứng dụng của chúng.
- Một phương pháp kiểm tra lỗi thông dụng khi truyền thông tin.

❖ Các Thuật Ngữ Tiếng Anh:

- Binary-Coded-Decimal code (BCD): Mã thập phân được mã hóa dưới dạng nhị phân.
- Alphanumeric code: Mã ký tự
- American Standard Code for Information Interchange (ASCII): Mã trao đổi thông tin theo tiêu chuẩn Mỹ.
- Parity method: Phương pháp kiểm tra chẵn-lẻ
- Parity bit: Bit kiểm tra
- Institute Electrical and Electronics Engineers (IEEE): tiêu chuẩn kỹ thuật trong điện và điện tử

2.1- Các Phương Pháp Chuyển Đổi Giữa Các Hệ Thống Số Đếm

2.1.1- Chuyển đổi từ số nhị phân sang số thập phân

- Cách chuyển đổi này đã trình bày ở chương trước, giờ nhắc lại.
- Cho 2 số 101_B và 1011.11_B ta có kết quả cụ thể theo bảng sau:

2^4	2^3	2^2	2^1	2^0	Dấu $\bullet$	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}
1		0	1_B							
1	0	1	1	$\bullet$	1	1_B				

Bảng 7. Quy tắc biểu diễn một hệ thống nhị phân

$$101_{\text{B}} = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 4 + 1 = 5_{\text{D}}$$

$$1011.11_{\text{B}} = 1x2^3 + 0x2^2 + 1x2^1 + 1x2^0 + 1x2^{-1} + 1x2^{-2} = 8 + 2 + 1 + 0.5 + 0.25 = 11.75_{\text{D}}$$

2.1.2- Chuyển đổi từ số bát phân sang số thập phân

- Cách chuyển đổi này đã trình bày ở chương trước, giờ nhắc lại.
- Cho 2 số 276_{10} và 1563.18_{10} , ta có kết quả cụ thể theo bảng sau:

8^4	8^3	8^2	8^1	8^0	Dấu $\bullet$	8^{-1}	8^{-2}	8^{-3}	8^{-4}	8^{-5}
		2	7	6						
1		5	6	3	$\bullet$	1	8			

Bảng 8. Quy tắc biểu diễn một hệ thống bát phân

$$276_{\text{O}} = 2 \times 8^2 + 7 \times 8^1 + 6 \times 8^0 = 128 + 56 + 6 = 190_{\text{D}}$$

$$1563.18_{\text{O}} = 1x8^3 + 5x8^2 + 6x8^1 + 3x8^0 + 1x8^{-1} + 8x8^{-2} = 512 + 320 + 48 + 3 + 0.125 + 0.125 = 883.25_{\text{D}}$$

2.1.3- Chuyển đổi từ số thập lục phân sang số thập phân

- Cách chuyển đổi này đã trình bày ở chương trước, giờ nhắc lại.
- Cho 2 số 276_H và $1A6F.08_H$, ta có kết quả cụ thể theo bảng sau:

16^4	16^3	16^2	16^1	16^0	Dấu •	16^{-1}	16^{-2}	16^{-3}	16^{-4}	16^{-5}
2		7	6 _H							
1	A	6	F	•	0	8 _H				

Bảng 9. Quy tắc biểu diễn một hệ thống thập lục phân.

$$276_H = 2 \times 16^2 + 7 \times 16^1 + 6 \times 16^0 = 512 + 112 + 6 = 630_D.$$

$$1A6F.08_H = 1 \times 16^3 + 10 \times 16^2 + 6 \times 16^1 + 15 \times 16^0 + 0 \times 16^{-1} + 8 \times 16^{-2} = 4096 + 2560 + 96 + 15 + 0.03125 = 6767.03125_D.$$

2.1.4- Chuyển đổi từ số thập phân sang số nhị phân

➤ Cách chuyển đổi:

Số thập phân gồm hai phần: phần nguyên và phần định trị.

Số nhị phân tương ứng cũng sẽ gồm hai phần: phần nguyên và phần định trị.

Bước 1: lấy phần nguyên của số thập phân cần đổi chia nguyên cho 2, ta có kết quả gồm hai phần: kết quả nguyên và số dư. Ghi lại số dư.

Bước 2: nếu kết quả nguyên chưa bằng 0, lấy kết quả nguyên đó chia tiếp cho 2 và ghi lại số dư như bước 1 cho đến khi nào kết quả nguyên bằng 0.

Bước 3: viết lại chuỗi số dư đã ghi theo thứ tự ngược từ dưới lên trên, ta đã có phần nguyên số nhị phân tương ứng với phần nguyên số thập phân cần đổi.

Bước 4: lấy phần định trị của số thập phân cần đổi nhân cho 2, ghi lại kết quả nguyên.

Bước 5: nếu kết quả nhân 2 vẫn còn phần định trị, lấy phần định trị đó tiếp tục nhân 2 và ghi lại kết quả nguyên như bước 4 cho đến khi nào phần định trị bằng 0.

Bước 6: viết lại chuỗi kết quả nguyên đã ghi theo thứ tự thuận từ trên xuống dưới, ta đã có phần định trị số nhị phân tương ứng với phần định trị của số thập phân cần đổi.

➤ Ví dụ: đổi số 29.375_D thành số nhị phân

Lấy phần nguyên là 29 để đổi

$$\frac{29}{2} = 14 \text{ dư } 1$$

$$\frac{14}{2} = 7 \text{ dư } 0$$

$$\frac{7}{2} = 3 \text{ dư } 1$$

$$\frac{3}{2} = 1 \text{ dư } 1$$

$$\frac{1}{2} = 0 \text{ dư } 1$$

$$\Rightarrow 29_D = 11101_B$$

Lấy phần định trị 0.375 để đổi

$$0.375 \times 2 = 0.75 \text{ phần nguyên } 0$$

$$0.75 \times 2 = 1.5 \text{ phần nguyên } 1$$

$$0.5 \times 2 = 1.0 \text{ phần nguyên } 1$$

$$\Rightarrow 0.375_D = 011_B$$

➤ Vậy $29.375_D = 11101.011_B$

2.1.5- Chuyển đổi từ số thập phân sang số bát phân

➤ Cách chuyển đổi:

Số thập phân gồm hai phần: phần nguyên và phần định trị.

Số bát phân tương ứng cũng sẽ gồm hai phần: phần nguyên và phần định trị.

Bước 1: lấy phần nguyên của số thập phân cần đổi chia nguyên cho 8, ta có kết quả gồm hai phần: kết quả nguyên và số dư. Ghi lại số dư.

Bước 2: nếu kết quả nguyên chưa bằng 0, lấy kết quả nguyên đó chia tiếp cho 8 và ghi lại số dư như bước 1 cho đến khi nào kết quả nguyên bằng 0.

Bước 3: viết lại chuỗi số dư đã ghi theo thứ tự ngược từ dưới lên trên, ta đã có phần nguyên số bát phân tương ứng với phần nguyên số thập phân cần đổi.

Bước 4: lấy phần định trị của số thập phân cần đổi nhân cho 8, ghi lại kết quả nguyên.

Bước 5: nếu kết quả nhân 8 vẫn còn phần định trị, lấy phần định trị đó tiếp tục nhân 8 và ghi lại kết quả nguyên như bước 4 cho đến khi nào phần định trị bằng 0.

Bước 6: viết lại chuỗi kết quả nguyên đã ghi theo thứ tự thuận từ trên xuống dưới, ta đã có phần định trị số bát phân tương ứng với phần định trị của số thập phân cần đổi.

➤ Ví dụ: đổi số 129.375_D sang số bát phân

Lấy phần nguyên là 129 để đổi

$$\frac{129}{8} = 16 \text{ dư} \dots\dots\dots 1$$

$$\frac{16}{8} = 2 \text{ dư} \dots\dots\dots 0$$

$$\frac{2}{8} = 0 \text{ dư} \dots\dots\dots 2$$

$$\Rightarrow 129_D = 201_O$$

Lấy phần định trị là 0.375 để đổi

$$0.375 \times 8 = 3.0 \text{ phần nguyên} \dots\dots\dots 3$$

$$\Rightarrow 0.375_D = 0.3_O$$

➤ Vậy $129.375_D = 201.3_O$

2.1.6- Chuyển đổi từ số thập phân sang số thập lục phân

➤ Cách chuyển đổi:

Số thập phân gồm hai phần: phần nguyên và phần định trị.

Số thập lục phân tương ứng cũng sẽ gồm hai phần: phần nguyên và phần định trị.

Bước 1: lấy phần nguyên của số thập phân cần đổi chia nguyên cho 16, ta có kết quả gồm hai phần: kết quả nguyên và số dư. Ghi lại số dư, lưu ý nếu số dư từ 10 $\Rightarrow$ 15 thì ta đổi thành từ A $\Rightarrow$ F tương ứng .

Bước 2: nếu kết quả nguyên chưa bằng 0, lấy kết quả nguyên đó chia tiếp cho 16 và ghi lại số dư như bước 1 cho đến khi nào kết quả nguyên bằng 0.

Bước 3: viết lại chuỗi số dư đã ghi theo thứ tự ngược từ dưới lên trên, ta đã có phần nguyên số thập lục phân tương ứng với phần nguyên số thập phân cần đổi.

Bước 4: lấy phần định trị của số thập phân cần đổi nhân cho 16, ghi lại kết quả nguyên.

Bước 5: nếu kết quả nhân 16 vẫn còn phần định trị, lấy phần định trị đó tiếp tục nhân 16 và ghi lại kết quả nguyên như bước 4 cho đến khi nào phần định trị bằng 0.

Bước 6: viết lại chuỗi kết quả nguyên đã ghi theo thứ tự thuận từ trên xuống dưới, ta đã có phần định trị số thập lục phân tương ứng với phần định trị của số thập phân cần đổi.

➤ Ví dụ: đổi số 429.375_D sang số bát phân

Lấy phần nguyên là 429 để đổi

$$\frac{429}{16} = 26 \text{ dư } \dots\dots\dots 13 \Leftrightarrow D$$

$$\frac{26}{16} = 1 \text{ dư } \dots\dots\dots 10 \Leftrightarrow A$$

$$\frac{1}{16} = 0 \text{ dư } \dots\dots\dots 1$$

$$\Rightarrow 429_D = 1AD_H$$

Lấy phần định trị là 0.375 để đổi

$$0.375 \times 16 = 6.0 \text{ phần nguyên } \dots\dots\dots 6$$

$$\Rightarrow 0.375_D = 0.6_H$$

➤ Vậy 429.375_D = 1AD.6_H

2.1.7- Chuyển đổi từ số nhị phân sang số bát phân và ngược lại

➤ Giữa số nhị phân và số bát phân có một quan hệ theo bảng sau:

Số nhị phân	000	001	010	011	100	101	110	111
-------------	-----	-----	-----	-----	-----	-----	-----	-----

Số bát phân	0	1	2	3	4	5	6	7
-------------	---	---	---	---	---	---	---	---

Bảng 10. Quan hệ giữa số nhị phân và bát phân

2.1.7.1- Cách chuyển đổi từ số nhị phân sang số bát phân:

Bước 1: lấy phần nguyên của số nhị phân nhóm từ phải sang trái, cứ 3 bit thành một nhóm. Nhóm cuối cùng chứa MSB có thể không đủ 3 bit thì ta cứ điền các bit 0 vào bên trái MSB cho đủ 3 bit. Sau đó, đối chiếu với bảng 10 để thay các chữ số bát phân tương ứng với nhóm 3 bit đó.
 Bước 2: lấy phần định trị của số nhị phân nhóm từ trái sang phải, cứ 3 bit thành một nhóm. Nhóm cuối cùng chứa LSB có thể không đủ 3 bit thì ta cứ điền các bit 0 vào bên phải LSB cho đủ 3 bit. Sau đó, đối chiếu với bảng 10 để thay các chữ số bát phân tương ứng với nhóm 3 bit đó.

Ví dụ: đổi số 11011110.1111_B sang số bát phân

Lấy phần nguyên là 11011110 để đổi

Nhóm 3 bit lại rồi tra bảng: $11011110_B \Rightarrow '011 \ '011 \ '110_B = 336_O$

Lấy phần định trị là 0.1111 để đổi

Nhóm 3 bit lại rồi tra bảng: $1111_B \Rightarrow '111 \ '100_B = 74_O$

Vậy $11011110.1111_B = 336.74_O$

2.1.7.2- Cách chuyển đổi từ số bát phân sang số nhị phân:

Bước 1: lấy phần nguyên của số bát phân, rồi thay một chữ số bát phân là 3 bit nhị phân tương ứng trong bảng 10. Xong bỏ những bit 0 vô nghĩa ở bên trái.

Bước 2: lấy phần định trị của số bát phân, rồi thay một chữ số bát phân là 3 bit nhị phân tương ứng trong bảng 10. Xong bỏ những bit 0 vô nghĩa ở bên phải.

Ví dụ: đổi số 336.74_O sang số nhị phân

Lấy phần nguyên là 336 để đổi

$336_O = 011011110_B = 11011110_B$

Lấy phần định trị là 0.74 để đổi

$0.74_O = 0.111100_B = 0.1111_B$

2.1.8- Chuyển đổi từ số nhị phân sang số thập lục phân và ngược lại

➤ Giữa số nhị phân và số thập lục phân có một quan hệ theo bảng sau:

Số nhị phân	0000	0001	0010	0011	0100	0101	0110	0111
Số thập lục phân	0	1	2	3	4	5	6	7
Số nhị phân	1000	1001	1010	1011	1100	1101	1110	1111
Số thập lục phân	8	9	A	B	C	D	E	F

Bảng 11. Quan hệ giữa số nhị phân và thập lục phân

2.1.8.1- Cách chuyển đổi từ số nhị phân sang số thập lục phân:

Bước 1: lấy phần nguyên của số nhị phân nhóm từ phải sang trái, cứ 4 bit thành một nhóm. Nhóm cuối cùng chứa MSB có thể không đủ 4 bit thì ta cứ điền các bit 0 vào bên trái MSB cho đủ 4 bit. Sau đó, đối chiếu với bảng 11 để thay các chữ số thập lục phân tương ứng với nhóm 4 bit đó.

Bước 2: lấy phần định trị của số nhị phân nhóm từ trái sang phải, cứ 4 bit thành một nhóm. Nhóm cuối cùng chứa LSB có thể không đủ 4 bit thì ta cứ điền các bit 0 vào bên phải LSB cho đủ 4 bit. Sau đó, đối chiếu với bảng 11 để thay các chữ số thập lục phân tương ứng với nhóm 4 bit đó.

Ví dụ: đổi số 11011011110.100111_B sang số thập lục phân

Lấy phần nguyên là 11011011110 để đổi

Nhóm 4 bit lại rồi tra bảng: $11011011110_B \Rightarrow '0110 \ '1101 \ '1110_B = 6DE_H$

Lấy phần định trị là 0.100111 để đổi

Nhóm 4 bit lại rồi tra bảng: $100111_B \Rightarrow '1001 \ '1100_B = 9C_H$

Vậy $11011011110.100111_B = 6DE.9C_H$

2.1.8.2- Cách chuyển đổi từ số thập lục phân sang số nhị phân:

Bước 1: lấy phần nguyên của số thập lục phân, rồi thay một chữ số thập lục phân là 4 bit nhị phân tương ứng trong bảng 11. Xong bỏ những bit 0 vô nghĩa ở bên trái.

Bước 2: lấy phần định trị của số thập lục phân, rồi thay một chữ số thập lục phân là 4 bit nhị phân tương ứng trong bảng 11. Xong bỏ những bit 0 vô nghĩa ở bên phải.

Ví dụ: đổi số $6DE.9C_H$ sang số nhị phân

Lấy phần nguyên là $6DE$ để đổi

$6DE_H = 011011011110_B = 11011011110_B$

Lấy phần định trị là $0.9C$ để đổi

$0.9C_H = 0.10011100_B = 0.100111_B$

2.1.9- Chuyển đổi từ số bát phân sang số thập lục phân và ngược lại

Tốt nhất là ta chuyển đổi thông qua trung gian số nhị phân.

2.2- Số Nhị Phân Nguyên Không Dấu Và Các Phép Toán Số Học Trên Số Nhị Phân Nguyên Không Dấu

2.2.1- Số nhị phân nguyên không dấu

Hệ thống số nhị phân đang đề cập đến là hệ thống số nhị phân nguyên không dấu. Có nghĩa là số nhị phân có N bit thì số nhỏ nhất là số 0 và số lớn nhất là $2^N - 1$.

2.2.2- Các phép toán số học trên số nhị phân nguyên không dấu

2.2.2.1- Phép cộng

Quy tắc: $0 + 0 = 0$; $0 + 1 = 1 + 0 = 1$; $1 + 1 = 10$ (bằng 0 nhớ 1).

Ví dụ: cộng hai số nhị phân không dấu 4 bit sau $9_D + 5_D = 14_D$

$$\begin{array}{r} 1001 \\ + 0101 \\ \hline 1110 \end{array}$$

2.2.2.2- Phép trừ

Quy tắc: $0 - 0 = 0$; $1 - 1 = 0$; $1 - 0 = 1$; $0 - 1 = 1$ (bằng 1 mượn 1).

Ví dụ: trừ hai số nhị phân không dấu 4 bit sau $9_D - 5_D = 4_D$

$$\begin{array}{r} 1001 \\ - 0101 \\ \hline 0100 \end{array}$$

2.2.2.3- Phép nhân

Quy tắc: $0 \times 0 = 0$; $0 \times 1 = 1 \times 0 = 0$; $1 \times 1 = 1$.

Ví dụ: nhân hai số nhị phân không dấu 4 bit cần một số nhị phân không dấu 8 bit $11_D \times 9_D = 99_D$

$$\begin{array}{r} \\ \\ \\ \\ \\ \\ \\ \hline 1 \\ \\ \\ \\ \\ \\ \hline 1 \end{array}$$

2.2.2.4- Phép dịch trái

Quy tắc: dịch trái một bit tương đương nhân 2^1 , dịch trái n bit tương đương nhân 2^n .

Ví dụ: $00011001_B = 25_D$ dịch trái 2 bit thành $01100100_B = 100_D$

Vậy số 00011001_B dịch trái 2 bit tương đương nhân 4

2.2.2.5- Phép chia

Quy tắc: giống như phép chia của số thập phân, phép chia số nhị phân là nghịch đảo của phép nhân $9_D : 3_D = 3_D$

Ví dụ: chia hai số nhị phân không dấu 4 bit sau

$$\begin{array}{r} 1001 \overline{) 11} \\ - 11 \\ \hline 11 \end{array}$$

$$\frac{11}{00}$$

2.2.2.6- Phép dịch phải

Quy tắc: dịch phải một bit tương đương chia 2^1 , dịch phải n bit tương đương chia 2^n .

Ví dụ: $01100100_B = 100_D$ dịch phải 2 bit thành $00011001_B = 25_D$

Vậy số 01100100_B dịch phải 2 bit tương đương chia 4

2.2.2.7- Sự tràn số

Sau khi thực hiện một phép toán số học trên số nhị phân N bit, nếu kết quả lớn hơn N bit, thì ta gọi đó là sự tràn số, và những bit lớn hơn N được gọi là bit bị tràn. Thường thì những số bị tràn sẽ được loại bỏ, không tính vào kết quả phép toán.

2.3- Số Nhị Phân Nguyên Có Dấu Và Các Phép Toán Số Học Trên Số Nhị Phân Nguyên Có Dấu

- Khi thực hiện các phép toán số học trên hệ thống số nhị phân, sẽ phát sinh nhu cầu về số âm, ví dụ như $0101_B - 0111_B$ ($5_D - 7_D = -2_D$)
- Muốn biểu diễn số có dấu, theo quy ước chung lấy MSB làm bit dấu, nếu MSB = 0 là số dương, MSB = 1 là số âm. Như vậy số dương lớn nhất của một số nhị phân N bit là $2^{N-1}-1$.
- Ví dụ như một số nhị phân 4 bit không dấu, số lớn nhất là $2^4-1 = 15_D$. thì một số nhị phân 4 bit có dấu, số lớn nhất là $2^{4-1}-1 = 7_D$.

2.3.1- Các cách biểu diễn số âm

- Số âm được biểu diễn theo nhiều cách, và thực tế đã có 4 cách biểu diễn số âm

2.3.1.1- Cách 1: Số đối

- Về mặt nguyên tắc, dễ chấp nhận nhất là biểu diễn theo cách biểu diễn thông thường của số thập phân là $+5_D = 0101_B$ thì $-5_D = 1101_B$. Tuy nhiên, cách biểu diễn có hai nhược điểm lớn: nhược điểm thứ nhất là số 0 có tới 2 cách viết là 0000_B và 1000_B và như vậy số nhị phân có dấu 4 bit chỉ có 15 số từ -7_D đến $+7_D$. Nhược điểm thứ hai là khi thực hiện phép cộng một số dương với một số âm như $(+5_D) + (-3_D)$ thì ta phải làm thành phép trừ $5_D - 3_D$. Và trong những phép toán theo cách biểu diễn này, thì bit dấu phải tách riêng không thực hiện phép cộng hay trừ theo các quy tắc đã đề cập ở mục 2.2. Cách này hiện vẫn còn dùng trong lưu trữ số thực dấu chấm động.

- Về mặt kỹ thuật giải quyết những vấn đề trên rất khó khăn, nên giải pháp khác được đưa ra là biểu diễn số âm dưới dạng **số bù**.

2.3.1.2- Cách 2: Số bù 1

- Số âm nhị phân theo quan điểm này là lấy đảo lại với số dương, gọi là số bù 1. Ví dụ số dương $+5_D = 0101_B$ thì $-5_D = 1010_B$. Nhưng gặp khó khăn là khi thực hiện phép cộng hai số, thì ta phải lấy bit nhớ cộng ngược lại với kết quả, ví dụ như $0101_B + 1011_B = 10000_B \Rightarrow 0000_B + 1_B = 0001_B$, do sự không thuận tiện đó, số bù 1 không được dùng làm số âm nữa

2.3.1.3- Cách 3: Số bù 2

- Số âm biểu diễn theo cách này tương đối thuận tiện nhất, nên hiện giờ sử dụng trong các phép tính số nguyên có dấu. Nó chỉ có nhược điểm và cũng là ưu điểm là không đối xứng, luôn có một số âm mà không có số dương tương ứng.

2.3.1.4- Cách 4: Số dư 128 hay số dư 1024

- Số âm biểu diễn theo cách này dùng 8 bit trong hệ thống số dư 128 hay 11 bit trong số dư 1024. Trong hệ thống số dư 128, ta lấy giá trị của nó trừ cho 128, nếu nó lớn hơn 0 thì đó là số dương, nếu ngược lại thì lấy 128 trừ cho nó đó sẽ là số âm tương ứng. Và có điều thú vị là số âm dạng số dư 128 và số âm dạng bù 2 giống nhau hoàn toàn ở 7 bit cuối, chỉ khác nhau ở bit đầu mà thôi. Số âm dạng số dư 128 và số âm dạng số dư 1024 được dùng trong phần mũ của số thực dấu chấm động theo tiêu chuẩn IEEE 754

2.3.2- Số bù

2.3.2.1- Số bù e

Số bù e của một số nguyên N thuộc hệ thống đếm cơ số e ký hiệu là N_{BUE} được tính theo công thức:

$$N_{BUE} = e^N - N$$

Ví dụ: số 101011_B trong hệ thống đếm nhị phân có số bù 2 là $2^6_B - 101011_B = 1000000_B - 101011_B = 010101_B$

Số 51276_D trong hệ thống đếm thập phân có số bù 10 là $10^5_D - 51276_D = 100000_D - 51276_D = 48724_D$

2.3.2.2- Số bù e - 1

Số bù e của một số nguyên N thuộc hệ thống đếm cơ số e ký hiệu là $N_{\text{BÙ } e-1}$ được tính theo công thức:

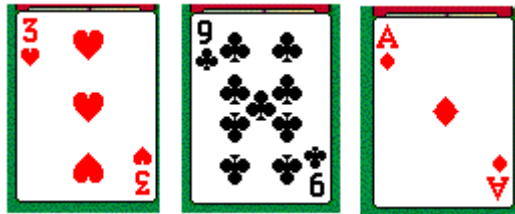
$$N_{\text{BÙ } e-1} = e^N - N - 1$$

Ví dụ: số 101011_B trong hệ thống đếm nhị phân có số bù 1 là $2^6_B - 101011_B = 1000000_B - 101011_B - 1_B = 010100_B$

Số 51276_D trong hệ thống đếm thập phân có số bù 9 là $10^5_D - 51276_D = 100000_D - 51276_D - 1_D = 48723_D$

2.3.3- Số bù trong hệ thập phân

- Trong hệ thập phân, ngoài cách biểu diễn thông dụng là một số đối của một số A là số $-A$ theo quy tắc $A + (-A) = 0$. Thì người ta còn dùng một cách khác đó là số bù 10 của A để biểu diễn số đối của A theo quy tắc $A + (A_{\text{BÙ } 10}) = 0$. Vậy số bù 10 của A, ký hiệu là $A_{\text{BÙ } 10}$ được biểu diễn theo quy tắc nào.
- Để dễ hiểu, ta xem lại quy tắc chơi của trò chơi bài ba lá



Hình 9. Trò chơi bài 3 lá

- Trong trò chơi này, số nút của người chơi là $3 + 9 + 1 = 13 = 3$ có nghĩa là kết quả bằng 10 xem như là 0, nếu quá 10 thì chỉ lấy hàng đơn vị.
- Trong trò chơi này, ta thấy số đối của số 3_D chẳng hạn là số 7_D vì $3_D + 7_D = 10_D = 0_D$, như vậy số 7 trong trường hợp này chính là số $3_{\text{BÙ } 10}$, hay số 7 chính là số -3 theo cách biểu diễn thông thường.
- Với một số thập phân 4 chữ số 4637, ta tìm một số thập phân 4 chữ số sau khi cộng thì kết quả là 0. Số đó chính là số âm hay số bù 10 của 4637.

$$\begin{array}{r}
 4 \ 6 \ 3 \ 7 \\
 + \quad ? \ ? \ ? \ ? \\
 \hline
 0 \ 0 \ 0 \ 0
 \end{array}$$

- Ta tìm ra được số 5363. Ta nói số âm của số 4637 là số 5363 vì:

$$\begin{array}{r}
 4 \ 6 \ 3 \ 7 \\
 + \quad 5 \ 3 \ 6 \ 3 \\
 \hline
 1 \ 0 \ 0 \ 0 \ 0
 \end{array}$$

- Chữ số thứ năm là chữ số bị tràn sẽ bị bỏ không tính. Số 5363 $\Leftrightarrow 4637_{\text{BÙ } 10}$ vì $4637 + 4637_{\text{BÙ } 10} = 0000$.
- Như vậy về mặt toán học, ta có thể phát triển là biểu diễn số âm bằng số bù với một số thập phân N chữ số.
- Nhưng để tìm một số âm của số cho trước được biểu diễn dưới dạng số bù 10 là công việc không được trực quan, và phải tính như trường hợp trên, con số 5363 tìm được là do lấy $10^5 - 4637$. Để tính số âm cho nhanh, ta sẽ tiến hành theo hai bước
- Bước 1: tính số bù 9 của số đã cho.

$$\begin{array}{r}
 4 \ 6 \ 3 \ 7 \\
 + \quad ? \ ? \ ? \ ? \\
 \hline
 9 \ 9 \ 9 \ 9
 \end{array}$$

Công việc này đơn giản vì cứ lấy từng chữ số đã cho cộng với chữ số tương ứng sao cho kết quả là 9 như sau:

$$\begin{array}{r}
 4 \ 6 \ 3 \ 7 \\
 + \quad 5 \ 3 \ 6 \ 2 \\
 \hline
 9 \ 9 \ 9 \ 9
 \end{array}$$

Vậy số 5362 là số bù 9 của số 4637 vì $4637 + 5362 = 9999$

Bước 2: lấy số bù 9 cộng cho 1 ta được số bù 10

$5362 + 1 = 5363$ đây chính là kết quả ta thu được ở trên.

- Với cách biểu diễn số âm bằng số bù 10, ta thấy không còn phép trừ, chúng ta đã thay phép trừ bằng phép cộng số bù 10.

2.3.4- Số bù trong hệ nhị phân nguyên

- Trong hệ nhị phân, ta áp dụng quy tắc lấy số âm theo số bù ở trên. Do ở đây là hệ nhị phân nên ta chỉ có hai số bù đó là bù 1 và bù 2. Với số bù 2 là số đối của số đã cho
- Cho một số dương không dấu 4 bit là $0101_B (+5_D)$
- Số bù 1 của $0101_B \Rightarrow 1010_B$ (số bù 1 của 1 số là đảo của số đó)
- Số bù 2 của 0101_B bằng số bù 1 cộng 1 $\Rightarrow 1010_B + 1_B = 1011_B (-5_D)$
 Nhận xét: $(+5_D) + (-5_D) = 0101_B + 1011_B = 0000$ (bit 5 bỏ, không xét)

$$\begin{array}{r} 0 \ 1 \ 0 \ 1 \\ + \quad 1 \ 0 \ 1 \ 1 \\ \hline 1 \ 0 \ 0 \ 0 \ 0 \end{array}$$

- Lấy 2 số $(+7_D) + (-6_D) = +1_D$

$$\begin{array}{r} 0 \ 1 \ 1 \ 1 \\ + \quad 1 \ 0 \ 1 \ 0 \\ \hline 1 \ 0 \ 0 \ 0 \ 1 \end{array}$$

- Với cách biểu diễn này, ta đã giải quyết được hai nhược điểm đã đề cập ở mục 2.3 là chỉ dùng một phép cộng thay vì phải dùng mạch trừ, và với N bit ta có thể biểu diễn được 2^N số nhị phân.
- Ta lập bảng số nhị phân có dấu trong trường hợp 4 bit

Số nhị phân không dấu	0000	0001	0010	0011	0100	0101	0110	0111
Số thập phân tương ứng	0	1	2	3	4	5	6	7
Số nhị phân không dấu	1000	1001	1010	1011	1100	1101	1110	1111
Số thập phân tương ứng	8	9	10	11	12	13	14	15

Bảng 12. Số nhị phân không dấu ứng với số thập phân

Số nhị phân có dấu	0000	0001	0010	0011	0100	0101	0110	0111
---------------------------	-------------	-------------	-------------	-------------	-------------	-------------	-------------	-------------

Số thập phân tương ứng	0	1	2	3	4	5	6	7
Số nhị phân có dấu	1000	1001	1010	1011	1100	1101	1110	1111
Số thập phân tương ứng	-8	-7	-6	-5	-4	-3	-2	-1

Bảng 13. Số nhị phân có dấu ứng với số thập phân

2.3.5- Các phép toán số học trên số nhị phân nguyên có dấu (dạng bù 2)

2.3.5.1- Phép cộng

Quy tắc: $0 + 0 = 0$; $0 + 1 = 1 + 0 = 1$; $1 + 1 = 10$ (bằng 0 nhớ 1).

Ví dụ: cộng hai số nhị phân có dấu 4 bit sau $(-7_D) + (5_D) = (-2_D)$

$$\begin{array}{r} 1001 \\ + 0101 \\ \hline 1110 \end{array}$$

2.3.5.2- Phép trừ

Thay thế phép trừ bằng phép cộng cho số bù 2.

Theo quy tắc: $A - C = A + C_{B\bar{U}2}$

Ví dụ: trừ hai số nhị phân có dấu 4 bit sau $(+7_D) - (5_D) = (+2_D)$

$\Rightarrow 0111_B - 0101_B = 0111_B + 1011_B = 10010_B = 0010_B$ bỏ bit 5 không xét.

$$\begin{array}{r} 0111 \\ + 1011 \\ \hline 10010 \end{array}$$

2.3.5.3- Phép nhân

Thay thế phép nhân bằng phép cộng dồn.

Theo quy tắc $A * C = C + C + \dots + C$ (tất cả A lần)

Ví dụ: $3_D * 2_D = 2_D + 2_D + 2_D = 0010_B + 0010_B + 0010_B = 0110_B = 6_D$ (tất cả 3 lần)

Hay thay thế phép nhân bằng phép cộng kết hợp với phép dịch trái

2.3.5.4- Phép chia

Thay thế phép chia bằng phép cộng số bù 2 dồn. Theo quy tắc lấy số bị chia cộng cho số chia bù 2, nếu kết quả phép cộng lớn hơn số chia bù 2, thì ta lấy kết quả đó cộng tiếp cho số chia bù 2 cho kết quả nhỏ hơn số chia bù 2 thì dừng. Số lần

làm phép cộng là kết quả nguyên. Số dư là kết quả cuối cùng của phép cộng.

$A : C \Leftrightarrow$

(lần 1) $A + C_{B\bar{U}2} = A_1$ nếu $A_1 < C$ thì dừng ngược lại thì tiếp tục

(lần 2) $A_1 + C_{B\bar{U}2} = A_2$ nếu $A_1 < C$ thì dừng ngược lại thì tiếp tục

.

.

.

(lần N) $A_{N-1} + C_{B\bar{U}2} = A_N$ với $A_N < C$.

Ta có $A : C = N$ dư A_N .

Ví dụ: $7_D : 2_D \Leftrightarrow 0111_B : 0010_B$

(lần 1) $0111_B + 1110_B = 10101_B \Rightarrow 0101_B$ (bỏ bit thứ 5)

(lần 2) $0101_B + 1110_B = 10011_B \Rightarrow 0011_B$ (bỏ bit thứ 5)

(lần 3) $0011_B + 1110_B = 10001_B \Rightarrow 0001_B$ (bỏ bit thứ 5)

Vậy: $7_D : 2_D = 3_D$ dư 1_D

2.4- Các Bộ Mã BCD Và Phép Toán Cộng Trên Mã NBCD

2.4.1- Các bộ mã BCD

- Chúng ta quen sử dụng hệ thống số thập phân, trong khi hệ thống kỹ thuật số về mặt bản chất là hệ thống nhị phân. Nên thường xuyên phải chuyển đổi giữa hai hệ thống là không tiện lợi trong một số nhu cầu. Nên người ta đã tìm ra những bộ mã để biểu diễn hệ thống số thập phân dưới dạng các bit nhị phân. Đó là các bộ mã BCD. Mã BCD có nghĩa là mỗi chữ số thập phân được mã hoá bằng 4 bit nhị phân. Có nhiều bộ mã BCD như 8421 (được gọi là BCD-Normal hay NBCD), 7421, 2421, 5121 ... Mỗi bộ mã BCD được dùng tùy theo yêu cầu cụ thể, tương ứng với các chữ số thập phân theo bảng sau:

Số thập phân	0	1	2	3	4	5	6	7	8	9
Mã 8421	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001
Mã 7421	0000	0001	0010	0011	0100	0101	0110	1000	1001	1010
Mã 2421	0000	0001	0010	0011	0100	1011	1100	1101	1110	1111
Mã 5121	0000	0001	0010	0011	0111	1000	1001	1010	1011	1111

Bảng 14. Bảng mã BCD

- Bộ mã 8421 dùng để tính toán như là số thập phân thông thường, với trọng số của 4 vị trí từ MSB sang LSB lần lượt là 8, 4, 2, 1.
- Bộ mã 7421 thì các trọng số 4 vị trí từ MSB sang LSB lần lượt là 7, 4, 2, 1.
- Bộ mã 2421 và 5121 thì nhằm mục đích là số bù 9 của bộ mã này sẽ bằng với số bù 1 của nó để thiết kế mạch cộng trừ số thập phân. Và trọng số của nó đúng như tên bộ mã là 2, 4, 2, 1 cho bộ mã 2421 và 5, 1, 2, 1 cho bộ mã 5121.

2.4.2- Phép toán cộng trên mã NBCD

2.4.2.1- Trường hợp 1: tổng hai số NBCD ≤ 9

Trường hợp này cách cộng như cộng số nhị phân bình thường.

Ví dụ: $53_D + 26_D = 79_D \Rightarrow 01010011_{NBCD} + 00100110_{NBCD}$

$$\begin{array}{r} 0 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1 \\ + \ 0 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \\ \hline 0 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 1 \end{array}$$

2.4.2.2- Trường hợp 2: tổng hai số NBCD > 9 và ≤ 15

Ta chia làm hai bước:

Bước 1: cộng nhị phân bình thường.

Bước 2: cộng thêm chữ số 6_{NBCD} vào tổng nào bị vượt quá 9 và ≤ 15 , bit bị tràn sẽ được cộng bổ sung cho chữ số NBCD nằm kề bên trái.

Ví dụ: $38_D + 46_D = 84_D \Rightarrow 00111000_{NBCD} + 01000110_{NBCD}$

Bước 1: cộng nhị phân bình thường.

$$\begin{array}{r} 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \\ + \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \\ \hline 0 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 0 \end{array}$$

Bước 2: do 4 bit cuối là 1110 không phải mã NBCD, nên ta cộng bổ sung với 6_{NBCD} (0110)

$$\begin{array}{r} 0 \ 1 \ 1 \ 1 \ 1 \ 1 \ 0 \\ + \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \\ \hline 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \end{array}$$

2.4.2.3- Trường hợp 3: tổng hai số NBCD > 15

Ta chia làm hai bước:

Bước 1: cộng nhị phân bình thường, tổng hai chữ số NBCD nào vượt quá 15, thì xảy ra trường hợp tràn số, bit bị tràn kia vẫn được cộng cho chữ số NBCD nằm kề bên trái.

Bước 2: cộng thêm 6_{NBCD} vào tổng nào bị tràn số,

Ví dụ: $38_D + 49_D = 87_D \Rightarrow 00111000_{\text{NBCD}} + 10001001_{\text{NBCD}}$

Bước 1: cộng nhị phân bình thường.

$$\begin{array}{r} 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \\ + \ 0 \ 1 \ 0 \ 0 \ 1 \ 0 \ 0 \ 1 \\ \hline 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \end{array}$$

Bước 2: do tổng hai chữ số NBCD bị tràn số, nên ta cộng bổ sung với 6_{NBCD} (0110)

$$\begin{array}{r} 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \\ + \ 0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \\ \hline 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \end{array}$$

2.5- Các Loại Mã Khác: Mã ASCII, Mã Quá 3, Mã Quá 6, Mã Gray

2.5.1- Các loại mã 4 bit thường dùng

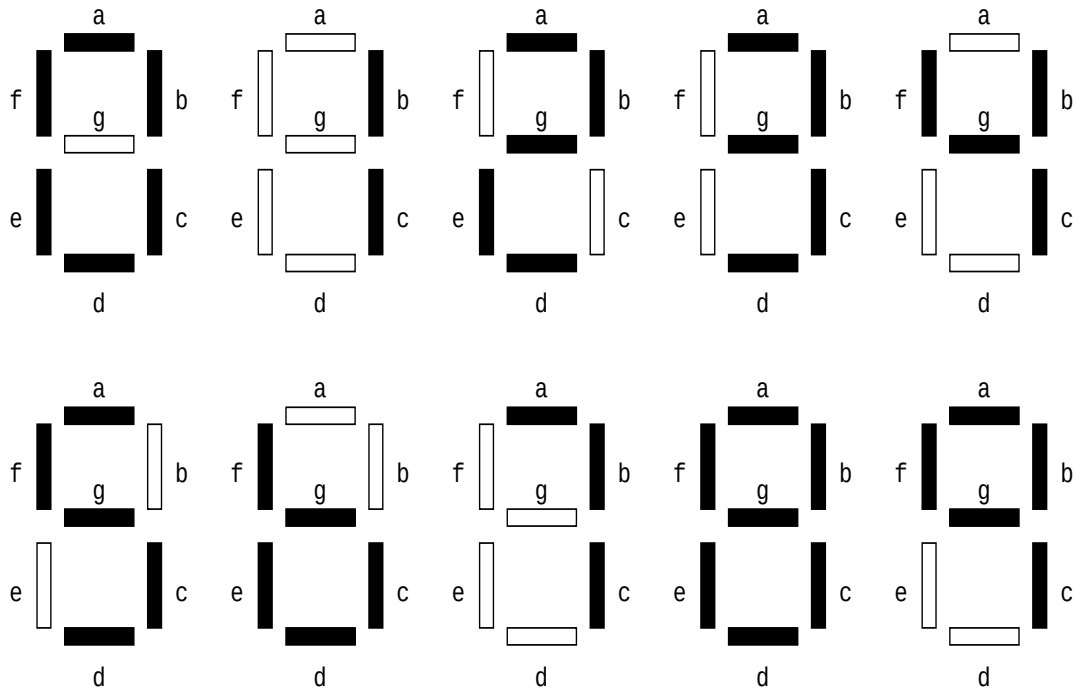
- Mã quá 3: lấy số nhị phân 4 bit cộng với 3. Được dùng khá nhiều trong các thiết bị tính toán số học số thập phân.
- Mã quá 6: lấy số nhị phân 4 bit cộng với 6. Được dùng khá nhiều trong các thiết bị tính toán số học số thập phân.
- Mã Gray: các số mã cạnh nhau chỉ khác nhau 1 bit. Do không sắp xếp theo vị trí, nên mã này không dùng để thực hiện các phép tính số học. Được dùng trong kỹ thuật đo lường để giảm sai số như cân điện tử, máy đo vị trí, hoặc dùng để thiết kế biểu đồ Karnaugh.
- Mã Gray quá 3: tương tự như mã Gray, nhưng lệch đi 3 hàng.

Số nhị phân	Mã quá 3	Mã quá 6	Mã Gray	Mã Gray quá 3	Mã 7 đoạn - mức 0						
					a	b	c	d	e	f	g
0000	0011	0110	0000	0010	0	0	0	0	0	0	1

0001	0100	0111	0001	0110	1	0	0	1	1	1	1
0010	0101	1000	0011	0111	0	0	1	0	0	1	0
0011	0110	1001	0010	0101	0	0	0	0	1	1	0
0100	0111	1010	0110	0100	1	0	0	1	1	0	0
0101	1000	1011	0111	1100	0	1	0	0	1	0	0
0110	1001	1100	0101	1101	1	1	0	0	0	0	0
0111	1010	1101	0100	1111	0	0	0	1	1	1	1
1000	1011	1110	1100	1110	0	0	0	0	0	0	0
1001	1100	1111	1101	1010	0	0	0	1	1	0	0
1010			1111	1011	1	1	1	0	0	1	0
1011			1110	1001	1	1	0	0	1	1	0
1100			1010	1000	1	0	1	1	1	0	0
1101			1011	0000	0	1	1	0	1	0	0
1110			1001	0001	1	1	1	0	0	0	0
1111			1000	0011	1	1	1	1	1	1	1

Bảng 15. Bảng mã quá 3, quá 6, Gray, Gray quá 3, 7 đoạn

- Mã 7 đoạn – mức 0: ứng với một số nhị phân là mã của một con số từ 0 – 9, còn từ 10 – 15 thì ít khi sử dụng. Ứng với mỗi đoạn sáng thì đoạn đó có giá trị 0, nếu không sáng thì có giá trị 1. Còn mã 7 đoạn mức 1 thì ngược lại.



Bảng 16. Bảng mã 7 đoạn từ 0 – 9

2.5.2- Các loại mã 5 bit thường dùng

- Mã 2 trên 5: bộ mã này dùng 5 bit, trong đó có 2 bit 1 và 3 bit 0 để biểu diễn hệ thập phân.
- Mã Johnson: bộ mã này cũng dùng 5 bit để biểu diễn hệ thống thập phân. Theo quy tắc là số ban đầu là 00000, bit 1 thay dần bit 0 từ phải sang trái. Cho đến khi 5 bit là 11111, thì bit 0 thay dần bit 1.
- Mã vòng: dùng 5 bit trong đó có duy nhất một bit có giá trị 1 và dịch dần từ phải sang trái.
- Đặc điểm của bộ mã 5 bit này là giảm bớt ảnh hưởng của nhiễu, và có khả năng phát hiện lỗi khi truyền dữ liệu.
- Còn nhiều loại mã khác tùy thuộc ứng dụng, và có độ dài lớn hơn 5 bit như mã Hamming, mã CRC, ... trong môn Lý Thuyết Thông Tin và Kỹ Thuật Truyền Số Liệu.

Số thập phân	Mã 2 trên 5	Mã Johnson	Mã vòng
--------------	-------------	------------	---------

0	00011	00000	10000
1	00101	00001	01000
2	00110	00011	00100
3	01001	00111	00010
4	01010	01111	00001
5	01100	11111	10000
6	10001	11110	01000
7	10010	11100	00100
8	10100	11000	00010
9	11000	10000	00001

Bảng 17. Bảng mã 2 trên 5 và mã Johnson

2.5.3- Các loại mã ký tự thường dùng

2.5.3.1- Mã ASCII: bộ mã này sử dụng 7 bit để mã hoá cho 1 ký tự. Mã từ 0000000_B – 0011111_B là các mã điều khiển như CR (về đầu dòng) có mã là 0001101_B , hay LF (xuống dòng) có mã là 0001010_B , hay mã ESC là 0011011_B .

Ký tự	Số nhị phân	Ký tự	Số nhị phân	Ký tự	Số nhị phân
A	1000001	a	1100001	0	0110000
B	1000010	b	1100010	1	0110001
C	1000011	c	1100011	2	0110010
D	1000100	d	1100100	3	0110011
E	1000101	e	1100101	4	0110100
F	1000110	f	1100110	5	0110101
G	1000111	g	1100111	6	0110110
H	1001000	h	1101000	7	0110111
I	1001001	i	1101001	8	0111000
J	1001010	j	1101010	9	0111001

Thiết Kế Hệ Thống Số

K	1001011	k	1101011	space	0100000
L	1001100	l	1101100	.	0101110
M	1001101	m	1101101	,	0101100
N	1001110	n	1101110	(	0101000
O	1001111	o	1101111	)	0101001
P	1010000	p	1110000	+	0101011
Q	1010001	q	1110001	-	0101101
R	1010010	r	1110010	*	0101010
S	1010011	s	1110011	/	0101111
T	1010100	t	1110100	=	0111101
U	1010101	u	1110101	\$	0100100
V	1010110	v	1110110	%	0100101
W	1010111	w	1110111	!	0100001
X	1011000	x	1111000	#	0100010
Y	1011001	y	1111001	CR	0001101
Z	1011010	z	1111010	LF	0001010

Bảng 18. Một phần của bảng mã ASCII

2.5.3.2- Mã BAUDOT: bộ mã này dùng 5 bit để mã hoá 1 ký tự. Được dùng trong bưu điện và telex.

Ký tự	blank	T	CR	O	space	H	N	M
Hình	blank	5	CR	9	space		,	.
Số nhị phân	00000	00001	00010	00011	00100	00101	00110	00111
Ký tự	LF	L	R	G	I	P	C	V
Hình	LF	)	4	&	8	0	:	;
Số nhị phân	01000	01001	01010	01011	01100	01101	01110	01111
Ký tự	E	Z	D	B	S	Y	F	X
Hình	3	“	\$	?	chuông	6	!	/

Số nhị phân	10000	10001	10010	10011	10100	10101	10110	10111
Ký tự	A	W	J	Chọn hình	U	Q	K	Chọn ký tự
Hình	-	2	,		7	1	(	
Số nhị phân	11000	11001	11010	11011	11100	11101	11110	11111

Bảng 19. Bảng mã Baudot

2.6- Kiểm Tra Lỗi Bằng Phương Pháp Chẵn - Lẻ.

Trong khi truyền thông tin từ nơi này đi nơi khác, thông tin bị ảnh hưởng do nhiễu, nên có thể bị sai lệch. Do đó, khi gửi thông tin đi, thường ta phải kèm thêm một số bit để báo lỗi và sửa sai. Một trong những phương pháp đơn giản nhất cho việc báo lỗi là phương pháp kiểm tra chẵn - lẻ.

- Bit chẵn – lẻ, là bit thêm vào nhóm bit dữ liệu sắp được gửi đi. Trong phương pháp kiểm tra chẵn, thì giá trị bit chẵn – lẻ được chọn sao cho số bit có giá trị 1 (sau này gọi tắt là bit 1) là số chẵn (tính luôn bit chẵn – lẻ được thêm vào). Ví dụ: cần gửi đi 8 bit 01100001, trong số 8 bit này, có 3 bit là bit 1. Vậy bit thêm vào phải là bit 1, và ta gửi đi 9 bit 101100001 trong đó có số bit 1 là 4. Bên nhận dữ liệu 9 bit, sẽ kiểm tra xem số bit 1 là số chẵn hay không, nếu là số chẵn thì nhận, còn số bit 1 là số lẻ thì không nhận. Giả sử trong trường hợp trên, bên nhận nhận được 9 bit như sau 100100001. Thì khi kiểm tra, sẽ thấy số bit 1 là 3, như vậy là đã bị lỗi, và sẽ không nhận dữ liệu này. Tuy nhiên, nếu sai hai bit trở lên, thì bên nhận xem như là đúng và nhận dữ liệu này. Như vậy, phương pháp này sẽ không phát hiện được lỗi nếu số bit bị sai là số chẵn 2, 4, 6. Nhưng xác suất để sai hai bit trở lên là rất thấp so với sai một bit. Nên phương pháp này rất thông dụng trong việc truyền dữ liệu.
- Phương pháp kiểm tra lẻ thì quy ước là số bit 1 gửi đi phải là số lẻ.
- Nếu muốn kiểm tra nếu sai từ hai bit trở lên, và bên nhận có thể phát hiện vị trí sai để sửa, ta phải dùng các phương pháp khác trong môn Lý Thuyết Thông Tin và Kỹ Thuật Truyền Số Liệu.

❖ Tóm Tắt Chương 2:

- Chuyển đổi giữa các hệ thống số:
 - Chuyển đổi một số của hệ thống đếm cơ số e sang hệ thập phân: biểu diễn theo lũy thừa của e.

- Chuyển đổi một số của hệ thống thập phân sang hệ thống đếm cơ số e:
Phần nguyên: lấy phần nguyên của số thập phân chia cho e, lấy số dư. Kết quả đọc ngược từ dưới lên.
Phần thập phân: lấy phần định trị của số thập phân nhân cho e, lấy phần nguyên đọc từ trên xuống.
- Chuyển đổi một số của hệ thống nhị phân sang hệ bát phân:
Phần nguyên: nhóm 3 bit từ phải sang trái, tra bảng tìm chữ số bát phân tương ứng.
Phần định trị: nhóm 3 bit từ trái sang phải, tra bảng tìm chữ số bát phân tương ứng.
- Chuyển đổi một số của hệ thống nhị phân sang hệ thập lục phân:
Phần nguyên: nhóm 4 bit từ phải sang trái, tra bảng tìm chữ số bát phân tương ứng.
Phần định trị: nhóm 4 bit từ trái sang phải, tra bảng tìm chữ số bát phân tương ứng.
- Chuyển đổi một số của hệ thống bát phân sang hệ nhị phân: tra bảng thay chữ số bát phân với 3 bit nhị phân tương ứng, bỏ các bit thừa bên tay trái phần nguyên và bỏ các bit thừa bên tay phải phần định trị.
- Chuyển đổi một số của hệ thống thập lục phân sang hệ nhị phân: tra bảng thay chữ số thập lục phân với 4 bit nhị phân tương ứng, bỏ các bit thừa bên tay trái phần nguyên và bỏ các bit thừa bên tay phải phần định trị.
- **Phép toán trên số nhị phân có dấu:**
 - Cộng hai số nhị phân: $0 + 0 = 0$; $0 + 1 = 1 + 0 = 1$; $1 + 1 = 10$
 - Số nhị phân có dấu:
Số dương: bit đầu tiên là bit 0. Các bit còn lại tính theo quy tắc thông thường.
Số âm: bit đầu tiên là bit 1. Được lưu trữ dưới dạng bù 2.
 - Các phép tính trên số nhị phân có dấu: chỉ thực hiện phép cộng, các phép toán trừ, nhân, chia đều quy đổi về phép cộng.
- **Mã BCD và phép toán cộng trên mã NBCD:**

- Tùy thuộc vào cách chọn trọng số cho từng vị trí, ta có các bộ mã BCD như 8421, 7421, 2421 và 5121. Loại mã BCD thông dụng nhất là mã 8421 hay còn gọi là NBCD. Trong tài liệu này, nếu không có gì cần chú thích thì ta gọi mã NBCD là mã BCD.
- Phép cộng trên mã NBCD thì cộng mỗi nhóm là 4 bit, theo cách cộng nhị phân thông thường, nếu kết quả phép cộng lớn hơn 9, thì ta cộng bổ sung thêm nhóm đó với số 6_{BCD} , bit bị tràn sẽ được nhớ qua nhóm bit kế tiếp.
- **Các loại mã khác:**
 - Mã quá 3 và mã quá 6 cũng là những loại mã hiển thị một chữ số thập phân.
 - Mã 7 vạch hiển thị một số thập phân bằng hình dạng những con số.
 - Mã Gray và mã Gray quá 3 là những bộ mã mà hai mã kế nhau chỉ khác nhau một bit.
 - Mã 2 trên 5 và mã Johnson là những loại mã 5 bit để hiển thị số thập phân.
 - Mã ký tự: hiển thị các ký tự và chữ số bằng một nhóm bit theo một tiêu chuẩn thông dụng nào đó. Như mã ASCII, mã Baudot,...
- **Phương pháp kiểm tra chẵn-lẻ:** thêm vào 1 bit kèm với N bit dữ liệu sao cho trong (N + 1) bit gởi đi này thì số bit 1 là số chẵn (nếu là phương pháp kiểm tra chẵn), hoặc số bit 1 là số lẻ (nếu là phương pháp kiểm tra lẻ).

Chương 3. ĐẠI SỐ BOOL VÀ CỔNG LOGIC

❖ Đề Cương:

- Biến logic, hằng logic, hàm logic và mạch logic.
- Các hàm logic cơ bản và các cổng logic.
- Mô tả mạch logic bằng hàm logic.
- Xây dựng mạch logic từ biểu thức logic hay hàm logic.
- Xác định ngõ xuất của mạch logic ứng với một bộ giá trị ngõ nhập.
- Xác định giá trị của hàm logic ứng với một bộ giá trị biến logic.
- Các phép toán logic và cổng logic được xây dựng thêm.
- Các định lý của đại số logic.
- Các vi mạch chứa các cổng logic.
- Các phương pháp biểu diễn hàm logic theo các dạng chuẩn
- Rút gọn bằng phương pháp đại số
- Rút gọn bằng phương pháp biểu đồ Karnaugh- dạng chuẩn tuyến
- Rút gọn bằng phương pháp biểu đồ Karnaugh- dạng chuẩn hội
- Rút

❖ Mục Đích:

Sau khi hoàn thành chương này, bạn phải nắm được kiến thức:

- Hàm logic, cổng logic và mạch logic.
- Các định lý của đại số logic.
- Các vi mạch chứa cổng logic.
- Xác định ngõ xuất của mạch logic ứng với một bộ giá trị ngõ nhập.
- Các phương pháp biểu diễn hàm logic.
- Rút gọn hàm logic bằng phương pháp đại số và bằng biểu đồ Karnaugh.

❖ Các Thuật Ngữ Tiếng Anh:

- | | |
|---------------------|----------------------------------|
| • Truth table: | bảng thực trị |
| • Logic variable: | biến logic |
| • Logic constant: | hằng logic |
| • Logic expression: | biểu thức logic |
| • Gate: | cổng logic |
| • Input: | ngõ nhập |
| • Output: | ngõ xuất |
| • Active Low: | tích cực mức thấp |
| • Active High: | tích cực mức cao |
| • Minterm: | dạng chuẩn tuyến (tổng các tích) |
| • Maxterm: | dạng chuẩn hội (tích các tổng) |

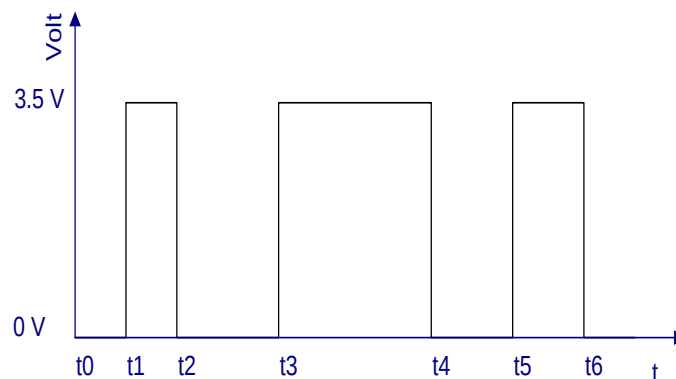
3.1- Biến Logic, Hằng Logic, Hàm Logic Và Mạch Logic

Cuối thế kỷ 19, nhà toán học Bool đã sáng lập ra một ngành toán học mang tên ông là **Đại Số Bool** hay cũng được gọi là **Đại Số Logic**. Đại số Bool nghiên cứu về hệ thống nhị phân. Đây là cơ sở và cũng là công cụ toán học cho mọi lĩnh vực có liên quan đến hệ thống kỹ thuật số.

- Cho 1 tập hợp $X = \{0, 1\}$, A được gọi là biến logic nếu $A \in X$, có nghĩa là A chỉ nhận 1 trong 2 giá trị: $A = 0$ hoặc $A = 1$.
- Trong logic mệnh đề, đây là hai giá trị **đúng** hay **sai**. Trong kỹ thuật thì nó tương ứng với mức điện thế 0 hay 1.
- Trong logic mệnh đề, nếu lấy giá trị 0 là sai và giá trị 1 là đúng, thì ta gọi là **logic dương**. Nếu chọn giá trị 1 là sai và giá trị 0 là đúng thì ta gọi là **logic âm**.
- Logic dương trong kỹ thuật (họ TTL), nếu chọn mức điện thế 0V-0.8V là giá trị 0, và 2V-5V là giá trị 1. Logic âm trong kỹ thuật, nếu chọn mức điện thế 0V-0.8V là giá trị 1, và 2V-5V là giá trị 0.
- Trong kỹ thuật số hiện nay, hầu hết mạch được thiết kế và chế tạo theo **logic dương**, nên người ta cũng hay dùng từ **mức cao** chỉ giá trị 1 và từ **mức thấp** chỉ giá trị 0.
- Và **giá trị 0** và **1** này là **ký hiệu logic**, không phải là **con số 0** và **số 1** trong **hệ thống số nhị phân** đã đề cập ở những chương trước. Từ giờ trở đi, ta sẽ dùng các chữ cái A, B, C... là ký hiệu đại số cho biến logic.

3.1.1- Biến logic

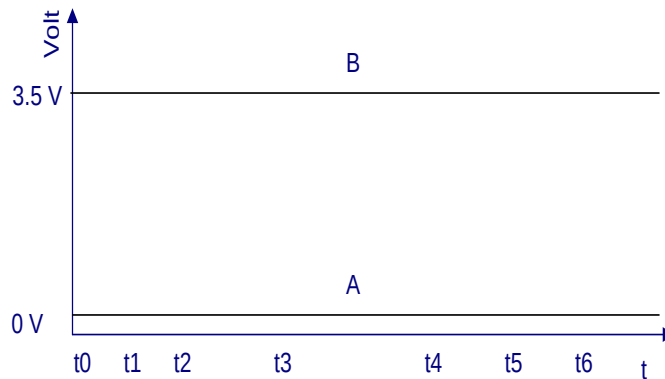
A được gọi là **biến logic**, có nghĩa là nó có thể thay đổi giá trị là 0 hay 1 theo thời gian



Hình 10. Giản đồ thời gian của biến A

3.1.2- Hằng logic

Biến logic A được gọi là ***hằng logic*** khi nó bằng 0 (được gọi là ***hằng 0***) hay bằng 1 (được gọi là ***hằng 1***) trong toàn bộ thời gian. Như vậy, hằng logic là trường hợp đặc biệt của biến logic.



A là hằng 0, B là hằng 1

Hình 11. Giản đồ thời gian của hằng A và hằng B.

3.1.3- Hàm Logic Và Mạch Logic

- Một hàm $x=f(A_1, A_2, \dots, A_n)$ được gọi là ***hàm logic n biến***, nếu $A_1, A_2, \dots, A_n$ là biến logic và $x \in X$ với $X = \{0, 1\}$.
- Vế phải của hàm là một ***biểu thức logic***. Vế trái là một biến logic.
- Biểu thức logic (biểu thức Bool) là một biểu diễn đại số với các toán hạng là biến logic và các toán tử là ***phép toán logic*** căn bản.
- Từ một biểu thức logic, ta xây dựng được một ***mạch logic*** tương ứng và ngược lại, từ một mạch logic tương ứng ta có thể biểu diễn bằng một biểu thức logic. Các ngõ nhập của mạch logic tương ứng với các biến logic, ngõ xuất tương ứng với giá trị của hàm số.

- Một mạch logic tương ứng với một phép toán logic căn bản của đại số Bool thì được gọi là **cổng logic**.
- Để thể hiện hoạt động của một hàm logic hay một mạch logic, người ta thường liệt kê giá trị của hàm (ngõ xuất của mạch logic) tương ứng với một bộ giá trị của biến (hay ngõ nhập của mạch logic). Bảng liệt kê toàn bộ giá trị của hàm theo biến (hay ngõ xuất theo ngõ nhập của mạch logic) được gọi là bảng thực trị.

3.1.4- Bảng thực trị

- Bảng thực trị cho một hàm $x=f(A_1, A_2, \dots, A_n)$ là một bảng gồm $N+1$ cột và 2^N dòng. Thể hiện toàn bộ giá trị của hàm tương ứng với các tổ hợp biến.

Ví dụ: cho 2 hàm $x = f(A, B)$; $y = g(A, B, C)$ ta lập bảng thực trị như sau:

A	B	x
0	0	1
0	1	0
1	0	1
1	1	0

C	B	A	y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Bảng 20. Bảng thực trị của hàm $x = f(A, B)$ và hàm $y = g(A, B, C)$

- Nhận xét: bảng thực trị cho hàm hai biến $x = f(A, B)$ có $2^2 = 4$ dòng và $2 + 1 = 3$ cột. và hàm ba biến $y = g(A, B, C)$ có $2^3 = 8$ dòng và $3 + 1 = 4$ cột.
- Bảng thực trị thể hiện mối quan hệ giữa hàm và biến bằng cách lập bảng liệt kê tất cả bộ giá trị của biến số và hàm số.

Ưu điểm là trực quan, nếu cho trước một bộ giá trị của biến logic, ta biết ngay giá trị hàm số tương ứng.

3.2- Các Phép Toán Logic Cơ Bản Và Các Cổng Logic

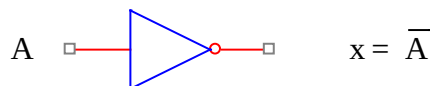
3.2.1- Phép toán NOT và cổng NOT

- Phép toán NOT là hàm một biến $x = f(A)$.
- Ký hiệu đại số: $x = \overline{A}$ (đọc là x bằng NOT A, hay x bằng A đảo, hay x bằng A bù).
- Phép toán NOT là một phép toán cơ bản của đại số Bool, nó hoạt động theo bảng thực trị sau:

A	$x = \overline{A}$
0	1
1	0

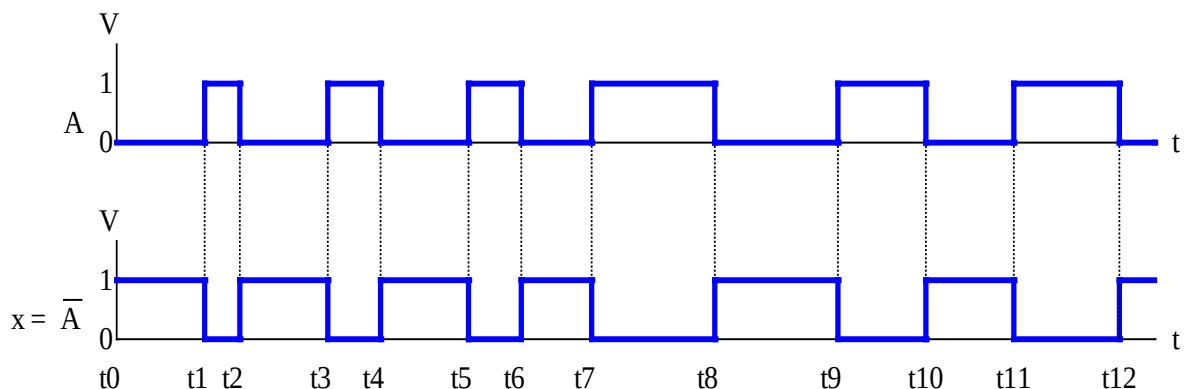
Bảng 21. Bảng thực trị của phép toán NOT

- Cổng NOT (hay còn gọi là **cổng đảo**) là mạch số thực hiện phép toán NOT, có 1 ngõ nhập và 1 ngõ xuất và có ký hiệu cổng như sau:



Hình 12. Ký hiệu cổng NOT

- Biểu đồ thời gian của biến A và hàm $x = \overline{A}$ được biểu diễn như hình sau:



Hình 13. Giản đồ thời gian của biến A và hàm $x = \bar{A}$

- Để vẽ giản đồ của $x = \bar{A}$, ta nhận thấy nó, luôn luôn ngược lại với A.

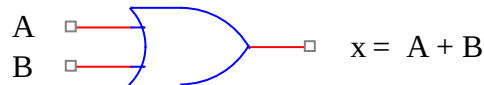
3.2.2- Phép toán OR và cổng OR

- Phép toán OR là hàm hai biến $x = f(A, B)$.
- Ký hiệu đại số: $x = A+B$ (đọc là x bằng A or B).
- Đây là phép toán cơ bản của đại số Bool, nó hoạt động theo bảng thực trị sau:

B	A	$x = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

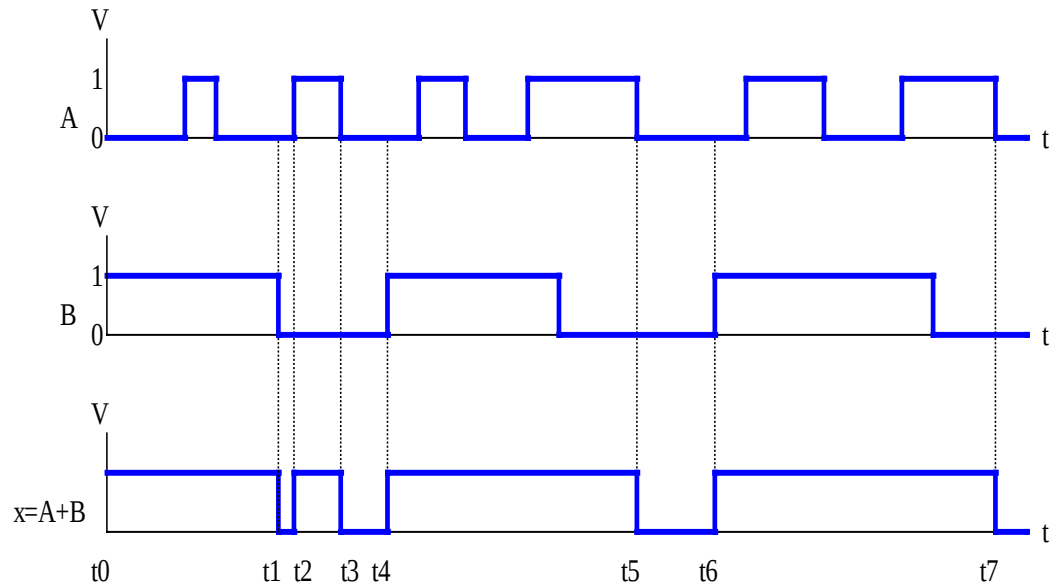
Bảng 22. Bảng thực trị của phép toán OR

- Cổng OR là mạch số thực hiện phép toán OR, có 2 ngõ nhập và 1 ngõ xuất và có ký hiệu cổng như sau:



Hình 14. Ký hiệu cổng OR

- Lưu ý: mặc dù là dùng ký hiệu (+), nhưng đây không phải là phép cộng nhị phân thông thường, mà là phép OR logic, nên có một số tác giả đã sử dụng ký hiệu khác thay cho ký hiệu (+) là ký hiệu (V). Nhưng hầu hết các tài liệu dùng trong kỹ thuật số đều dùng ký hiệu +. Nên trong tài liệu này vẫn dùng ký hiệu + biểu diễn cho phép OR.
- Giản đồ thời gian của hai biến A, B và hàm $x=A+B$ được biểu diễn như hình sau:



Hình 15. Giản đồ thời gian của biến A, B và hàm $x=A+B$

- Để vẽ được giản đồ thời gian của $x = A + B$, ta chỉ cần quan tâm đến những thời điểm A và B đồng thời bằng 0 $\Rightarrow x = 0$, trong trường hợp này là các thời điểm $t1 \rightarrow t2$, $t3 \rightarrow t4$, $t5 \rightarrow t6$, $t7 \rightarrow \dots$, các trường hợp khác x đều bằng 1.

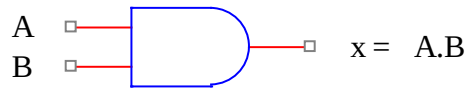
3.2.3- Phép toán AND và cổng AND

- Phép toán AND là hàm hai biến $x = f(A, B)$.
- Ký hiệu đại số: $x = A.B$ (đọc là x bằng A and B).
- Đây là phép toán cơ bản của đại số Bool, nó hoạt động theo bảng thực trị sau:

B	A	$x = A.B$
0	0	0
0	1	0
1	0	0
1	1	1

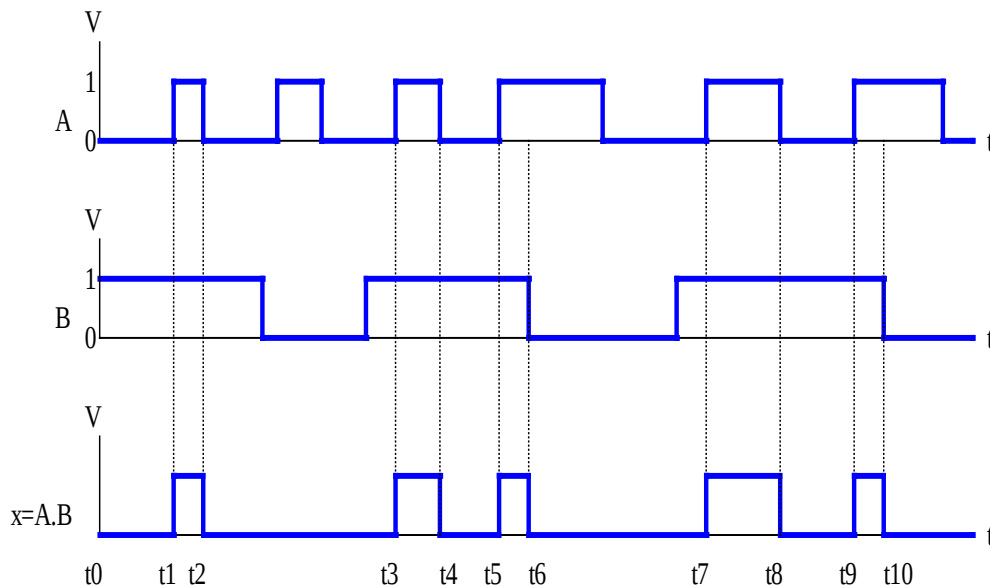
Bảng 23. Bảng thực trị của phép toán AND

- Cổng AND là mạch số thực hiện phép toán AND, có 2 ngõ nhập và 1 ngõ xuất và có ký hiệu cổng như sau:



Hình 16. Ký hiệu cổng AND

- Lưu ý: mặc dù là dùng ký hiệu ($\cdot$), nhưng đây không phải là phép nhân nhị phân thông thường, mà là phép AND logic. Nhưng lại có sự trùng hợp là phép AND logic hoàn toàn giống như phép nhân thông thường của 2 số nhị phân.
- Biểu đồ thời gian của hai biến A, B và hàm $x=A.B$ được biểu diễn như hình sau:



Hình 17. Biểu đồ thời gian của biến A, B và hàm $x=A.B$

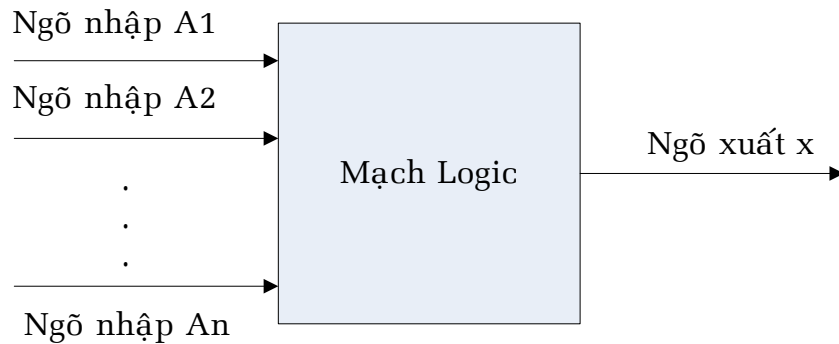
- Để vẽ được biểu đồ thời gian của $x = A \cdot B$, ta chỉ cần quan tâm đến những thời điểm A và B đồng thời bằng 1 $\Rightarrow x = 1$, trong trường hợp này là các thời điểm $t1 \rightarrow t2$, $t3 \rightarrow t4$, $t5 \rightarrow t6$, $t7 \rightarrow t8$, $t9 \rightarrow t10$, các trường hợp khác x đều bằng 0.

3.3- Mô Tả Mạch Logic Bằng Hàm Logic

- Thứ tự ưu tiên của các phép toán:
Phép toán NOT có độ ưu tiên cao nhất, kế đến là phép toán AND, cuối cùng là phép toán OR.

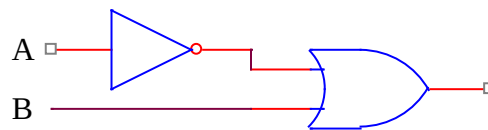
Nếu hai phép toán cùng độ ưu tiên , thì thực hiện theo thứ tự từ trái sang phải.

- Cho sơ đồ khối một mạch logic sau gồm một ngõ xuất x và n ngõ nhập đánh số từ $A_1, A_2, \dots, A_n$. Mạch này sẽ được biểu diễn bằng hàm logic n biến $x = f(A_1, A_2, \dots, A_n)$.



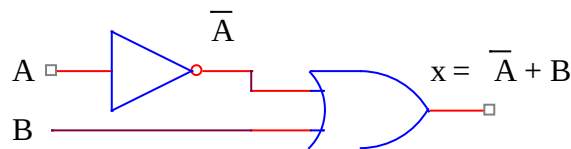
Hình 18. Sơ đồ khối một mạch logic

- Ta có thể xây dựng một hàm logic bất kỳ chỉ với 3 phép toán NOT, OR, AND đã đề cập. Hay nói cách khác ta có thể thiết kế một mạch số bất kỳ chỉ sử dụng 3 loại cổng NOT, OR, AND.
- Cho một số mạch logic cụ thể và mô tả chúng bằng hàm logic như sau:



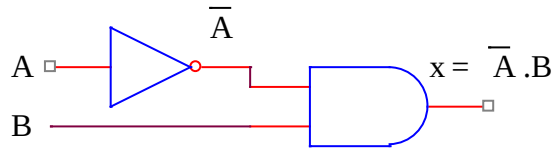
Hình 19. Sơ đồ một mạch logic dùng cổng NOT và cổng OR

Ta lần lượt ghi các hàm logic sau mỗi cổng như hình dưới, và ngõ xuất sau cùng chính là hàm logic của mạch logic đã cho.

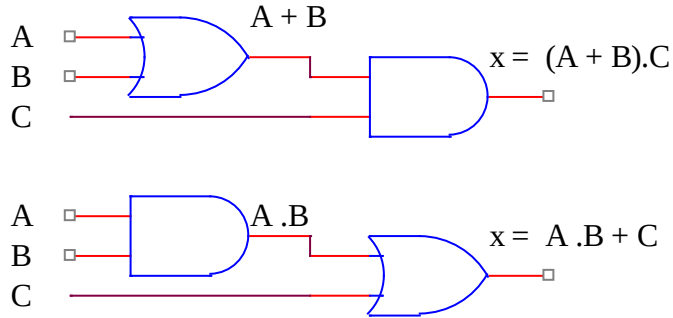


Hình 20. Hàm logic cho mạch dùng cổng NOT và cổng OR

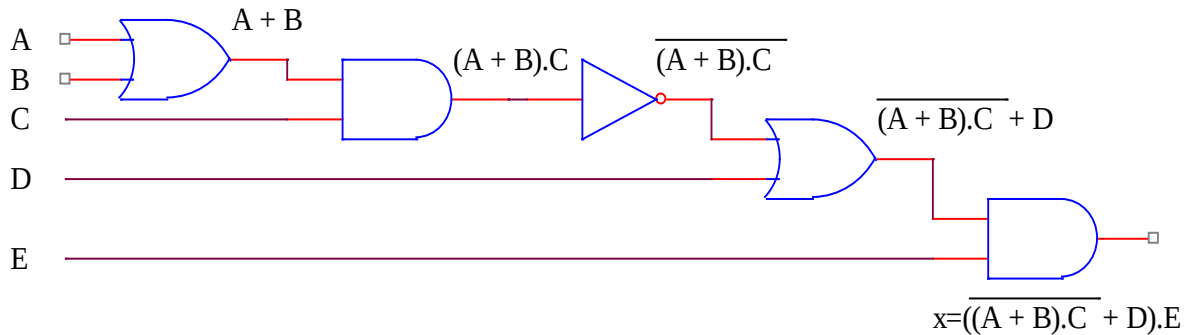
Tương tự cho những mạch logic đã cho ở dưới:



Hình 21. Hàm logic cho mạch dùng cổng NOT và cổng AND



Hình 22. Hàm Logic cho mạch dùng cổng OR và cổng AND



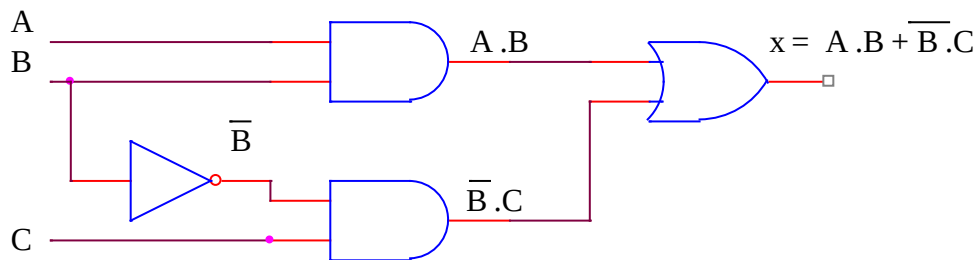
Hình 23. Mạch dùng cả 3 loại cổng OR, AND và NOT

3.4- Xây Dựng Mạch Từ Biểu Thức Logic Hay Hàm Logic

Từ một biểu thức logic hay hàm logic cho trước, ta xây dựng mạch logic bằng cách thay thế các phép toán logic bằng cổng logic tương ứng. như trong các ví dụ sau:

➤ Ví dụ: $x = A.B + \bar{B}.C$

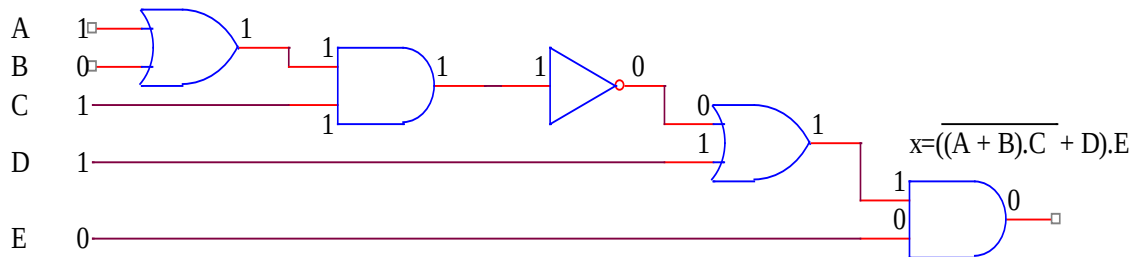
➤ Ta xây dựng sơ đồ mạch tương ứng:



Hình 24. Sơ đồ mạch tương ứng từ hàm logic đã cho

3.5- Xác Định Ngõ Xuất Của Mạch Logic Ứng Với Một Bộ Giá Trị Ngõ Nhập

Cho một mạch logic như hình dưới, ứng với một bộ giá trị của ngõ nhập là $(A,B,C,D,E) = (1,0,1,1,0)$ ta phải xác định giá trị ngõ xuất bằng cách lần lượt thay thế giá trị cụ thể của từng ngõ nhập. Lần lượt, tính ngõ xuất cho mỗi cổng, cho đến giá trị cuối cùng là $x = 0$ tương ứng với bộ giá trị đã cho.



Hình 25. Cách tính ngõ xuất của mạch logic ứng với một bộ giá trị ngõ nhập

3.6- Xác Định Giá Trị Của Hàm Logic Ứng Với Một Bộ Giá Trị Biến Logic

Cho một hàm logic $x = \overline{A.B} + \overline{B.C} + \overline{C+D}$ với bộ giá trị của biến logic là $(A,B,C,D) = (1,0,0,1)$ ta tính giá trị tương ứng của x bằng cách thay tên biến A,B,C,D bằng các giá trị logic cụ thể là 1,0,0,1:

$$x = \overline{1.0} + \overline{0.0} + \overline{0+1} = \overline{0.0} + \overline{1.0} + \overline{0+1} = \overline{0} + \overline{0} + \overline{1} = 1 + 1 + 0 = 1.$$

3.7- Các Phép Toán Logic Và Các Cổng Logic Được Xây Dựng Thêm

Trong nhu cầu thực tiễn, người ta xây dựng thêm 4 phép toán khác và chúng cũng được đưa vào nhóm phép toán cơ bản mở rộng. Đó là các phép toán NOR, NAND, EX-OR, EX-NOR. Với 4 phép toán mới thêm vào là 4 loại cổng mang tên tương ứng.

3.7.1- Phép toán NOR và cổng NOR

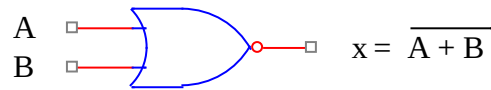
- Phép toán NOR là hàm hai biến $x = f(A, B)$.
- Ký hiệu đại số: $x = \overline{A+B}$ (đọc là x bằng A nor B).
- Đây là phép toán dựa trên hai phép toán là NOT và OR, nó hoạt động theo bảng thực trị sau:

B	A	$F = A + B$	$x = \overline{F} = \overline{A + B}$
0	0	0	1

0	1	1	0
1	0	1	0
1	1	1	0

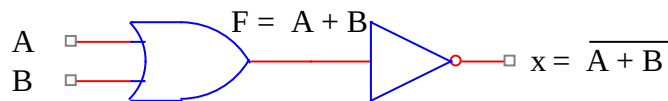
Bảng 24. Bảng thực trị của cổng NOR

- Cổng NOR là mạch số thực hiện phép toán NOR, có 2 ngõ nhập và 1 ngõ xuất và có ký hiệu cổng như sau:



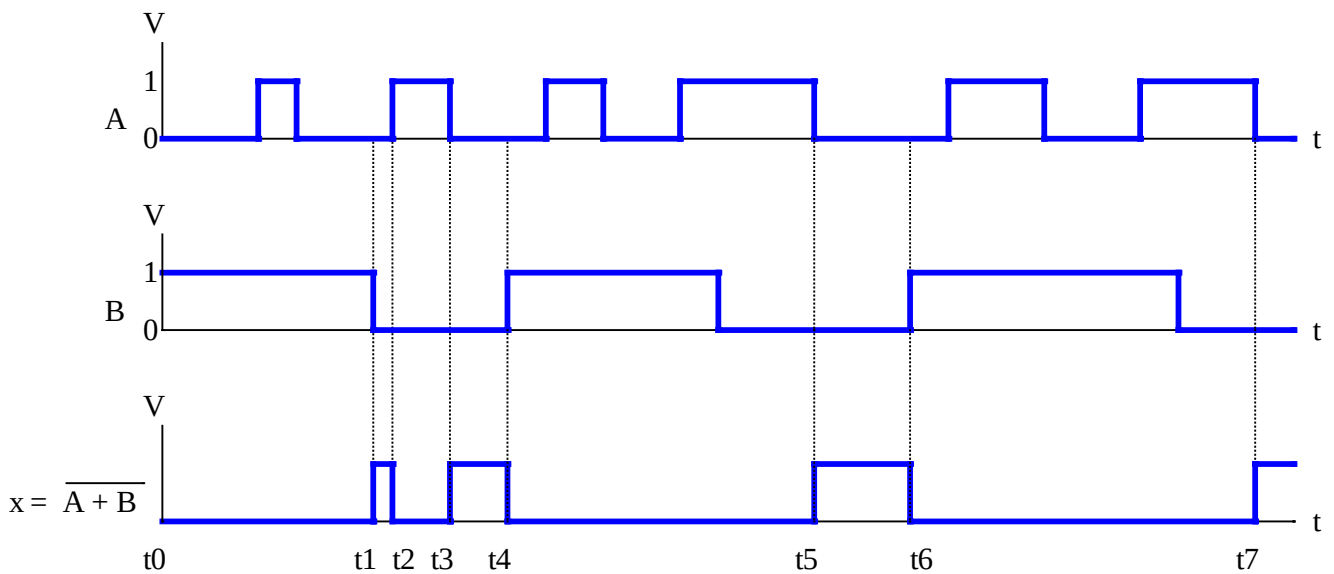
Hình 26. Ký hiệu cổng NOR

- Về nguyên tắc, thì cổng NOR có thể thay thế qua lại bằng hai cổng OR và NOT như hình sau:



Hình 27. Thay thế cổng NOR bằng hai cổng OR và NOT

- Giản đồ thời gian của hai biến A, B và hàm $x = \overline{A + B}$ được biểu diễn như hình sau:



Hình 28. Giản đồ thời gian của biến A, B và hàm $x = \overline{A + B}$

- Để vẽ được giản đồ thời gian của $x = \overline{A+B}$, ta chỉ cần quan tâm đến những thời điểm A và B đồng thời bằng 0 $\Rightarrow x = 1$, trong trường hợp này là các thời điểm $t1 \rightarrow t2$, $t3 \rightarrow t4$, $t5 \rightarrow t6$, $t7 \rightarrow \dots$, các trường hợp khác x đều bằng 0. Và chúng ta cũng thấy là giản đồ thời gian của hàm NOR/cổng NOR là nghịch đảo của hàm OR/cổng OR.
- Phép toán NOR và cổng NOR được ứng dụng rất nhiều, nên nó trở thành một phép toán và cổng cơ bản. Và người ta cũng đã chứng minh được là chỉ dùng duy nhất một phép toán NOR để xây dựng một hàm logic. Hay ở góc độ kỹ thuật, ta xây dựng được một mạch logic bất kỳ với một loại cổng duy nhất là cổng NOR.

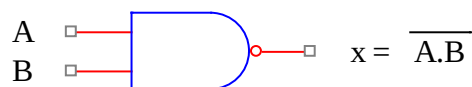
3.7.2- Phép toán NAND và cổng NAND

- Phép toán NAND là hàm hai biến $x = f(A, B)$.
- Ký hiệu đại số: $x = \overline{A.B}$ (đọc là x bằng A nand B).
- Đây là phép toán dựa trên hai phép toán là NOT và AND, nó hoạt động theo bảng thực trị sau:

B	A	$F = A.B$	$x = \overline{F} = \overline{A.B}$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

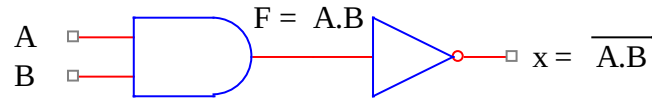
Bảng 25. Bảng thực trị của cổng NAND

- Cổng NAND là mạch số thực hiện phép toán NAND, có 2 ngõ nhập và 1 ngõ xuất và có ký hiệu cổng như sau:



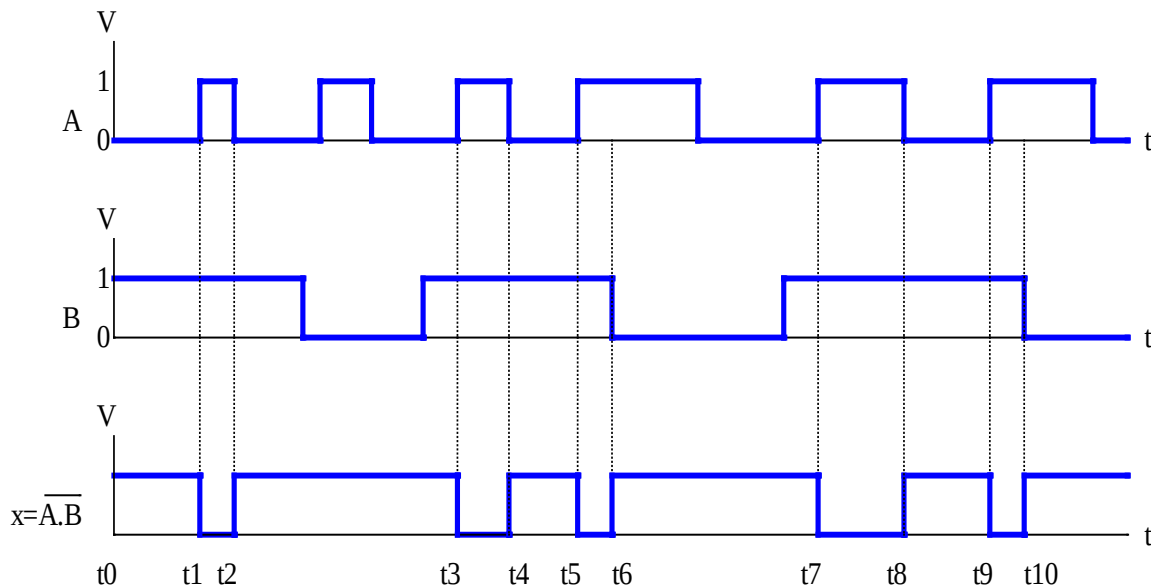
Hình 29. Ký hiệu cổng NAND

- Về nguyên tắc, thì cổng NAND có thể thay thế qua lại bằng hai cổng AND và NOT như hình sau:



Hình 30. Thay thế cổng NAND bằng hai cổng AND và NOT

- Biểu đồ thời gian của hai biến A, B và hàm $x = \overline{A.B}$ được biểu diễn như hình sau:



Hình 31. Biểu đồ thời gian của biến A, B và phép toán $x = \overline{A.B}$

- Để vẽ được biểu đồ thời gian của $x = \overline{A.B}$, ta chỉ cần quan tâm đến những thời điểm A và B đồng thời bằng 0 $\Rightarrow x = 1$, trong trường hợp này là các thời điểm $t_1 \rightarrow t_2$, $t_3 \rightarrow t_4$, $t_5 \rightarrow t_6$, $t_7 \rightarrow t_8$, $t_9 \rightarrow t_{10}$, các trường hợp khác x đều bằng 1. Và chúng ta cũng thấy là biểu đồ thời gian của hàm NAND/cổng NAND là nghịch đảo của hàm AND/cổng AND.
- Phép toán NAND và cổng NAND được ứng dụng rất nhiều, nên nó trở thành một phép toán và cổng cơ bản. Và người ta cũng đã chứng minh được là chỉ dùng duy nhất một phép toán NAND để xây dựng một hàm logic. Hay ở góc độ kỹ thuật, ta xây dựng được một mạch logic bất kỳ với một loại cổng duy nhất là cổng NAND.

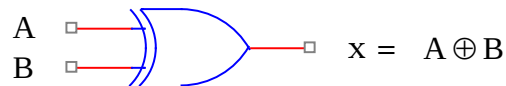
3.7.3- Phép toán EX-OR và cổng EX-OR

- Phép toán EX-OR là hàm hai biến $x = f(A, B)$.
- Ký hiệu đại số: $x = A \oplus B$ (đọc là x bằng A ex-or B).
- Đây là phép toán dựa trên ba phép toán là NOT, AND và OR, nó hoạt động theo bảng thực trị sau:

B	$\overline{B}$	A	$\overline{A}$	$F = A.\overline{B}$	$G = \overline{A}.B$	$x = F+G = A.\overline{B} + \overline{A}.B = A \oplus B$
0	1	0	1	0	0	0
0	1	1	0	1	0	1
1	0	0	1	0	1	1
1	0	1	0	0	0	0

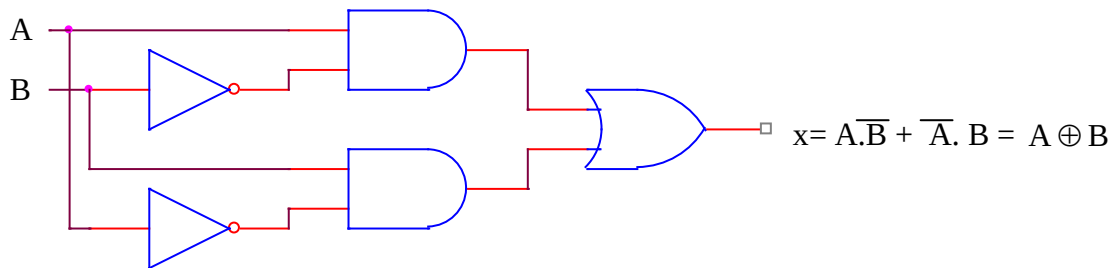
Bảng 26. Bảng thực trị của cổng EX-OR

- Cổng EX-OR là mạch số thực hiện phép toán EX-OR, có 2 ngõ nhập và 1 ngõ xuất và có ký hiệu cổng như sau:



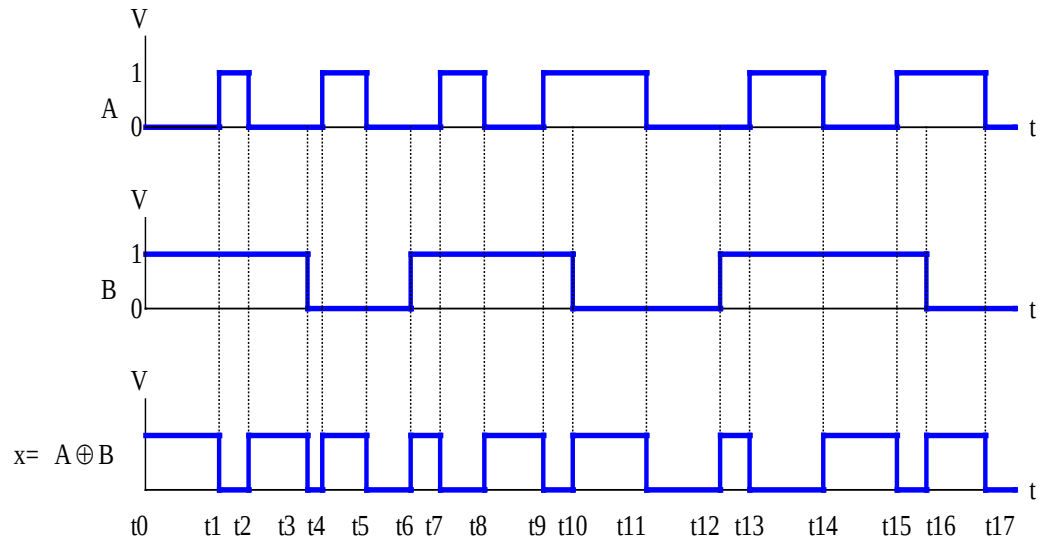
Hình 32. Ký hiệu cổng EX-OR

- Về nguyên tắc, thì cổng EX-OR có thể thay thế qua lại bằng ba loại cổng AND, OR và NOT như hình sau:



Hình 33. Thay thế cổng EX-OR bằng ba loại cổng AND, OR và NOT

- Giản đồ thời gian của hai biến A, B và phép toán $x = A \oplus B$ được biểu diễn như hình sau:



Hình 34. Giản đồ thời gian của biến A, B và hàm $x = A \oplus B$

- Để vẽ được giản đồ thời gian của $x = A \oplus B$, ta chỉ cần quan tâm đến những thời điểm A và B đồng thời bằng 0 hay đồng thời bằng 1 $\Rightarrow x = 0$, trong trường hợp này là các thời điểm $t1 \rightarrow t2$, $t3 \rightarrow t4$, $t5 \rightarrow t6$, $t7 \rightarrow t8$, $t9 \rightarrow t10$, $t11 \rightarrow t12$, $t13 \rightarrow t14$, $t15 \rightarrow t16$, $t17 \rightarrow \dots$, các trường hợp khác x đều bằng 1.

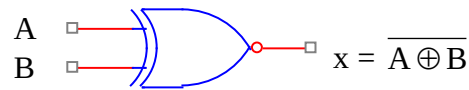
3.7.4- Phép toán EX-NOR và cổng EX-NOR

- Phép toán EX-NOR là hàm hai biến $x = f(A, B)$.
- Ký hiệu đại số: $x = \overline{A \oplus B}$ (đọc là x bằng A ex-nor B).
- Đây là phép toán dựa trên ba phép toán là NOT, AND và OR, nó hoạt động theo bảng thực trị sau:

B	$\overline{B}$	A	$\overline{A}$	$F = \overline{A} \cdot \overline{B}$	$G = A \cdot B$	$x = F + G = \overline{A} \cdot \overline{B} + A \cdot B = \overline{A \oplus B}$
0	1	0	1	1	0	1
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	0	1	1

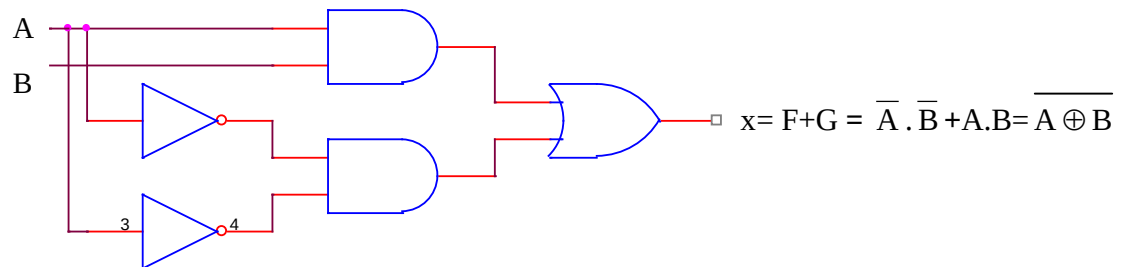
Bảng 27. Bảng thực trị của cổng EX-NOR

- Cổng EX-NOR là mạch số thực hiện phép toán EX-NOR, có 2 ngõ nhập và 1 ngõ xuất và có ký hiệu cổng như sau:



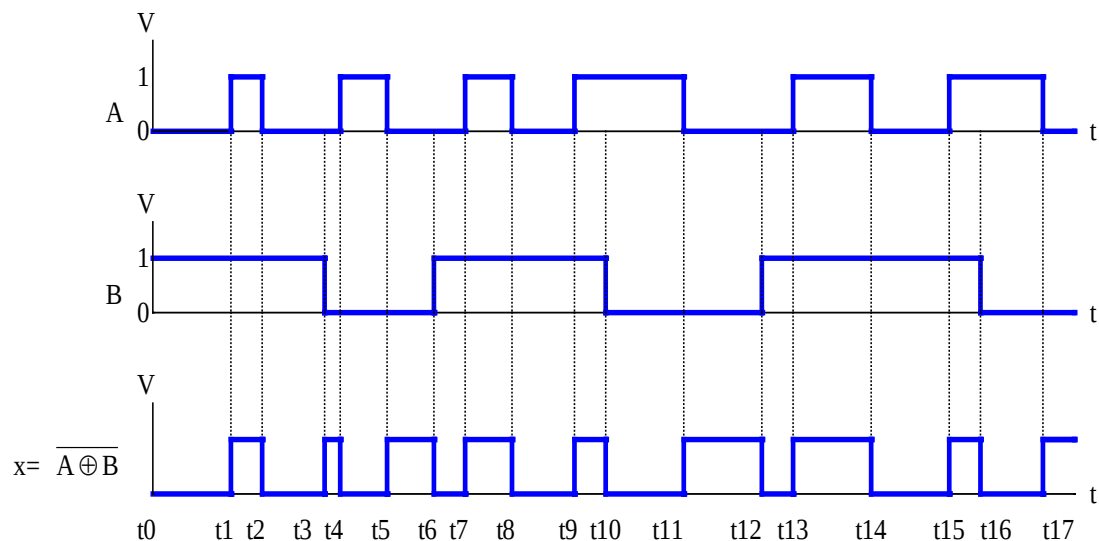
Hình 35. Ký hiệu cổng EX-NOR

- Về nguyên tắc, thì cổng EX-NOR có thể có thể thay thế qua lại bằng ba loại cổng AND, OR và NOT như hình sau:



Hình 36. Thay thế cổng EX-NOR bằng ba loại cổng AND, OR và NOT

- Giản đồ thời gian của hai biến A, B và phép toán $x = A \oplus B$ được biểu diễn như hình sau:



Hình 37. Giản đồ thời gian của biến A, B và hàm $x = A \oplus B$

- Để vẽ được giản đồ thời gian của $x = A \oplus B$, ta chỉ cần quan tâm đến những thời điểm A và B đồng thời bằng 0 hay đồng thời bằng 1 $\Rightarrow x = 1$, trong trường hợp này là các thời điểm

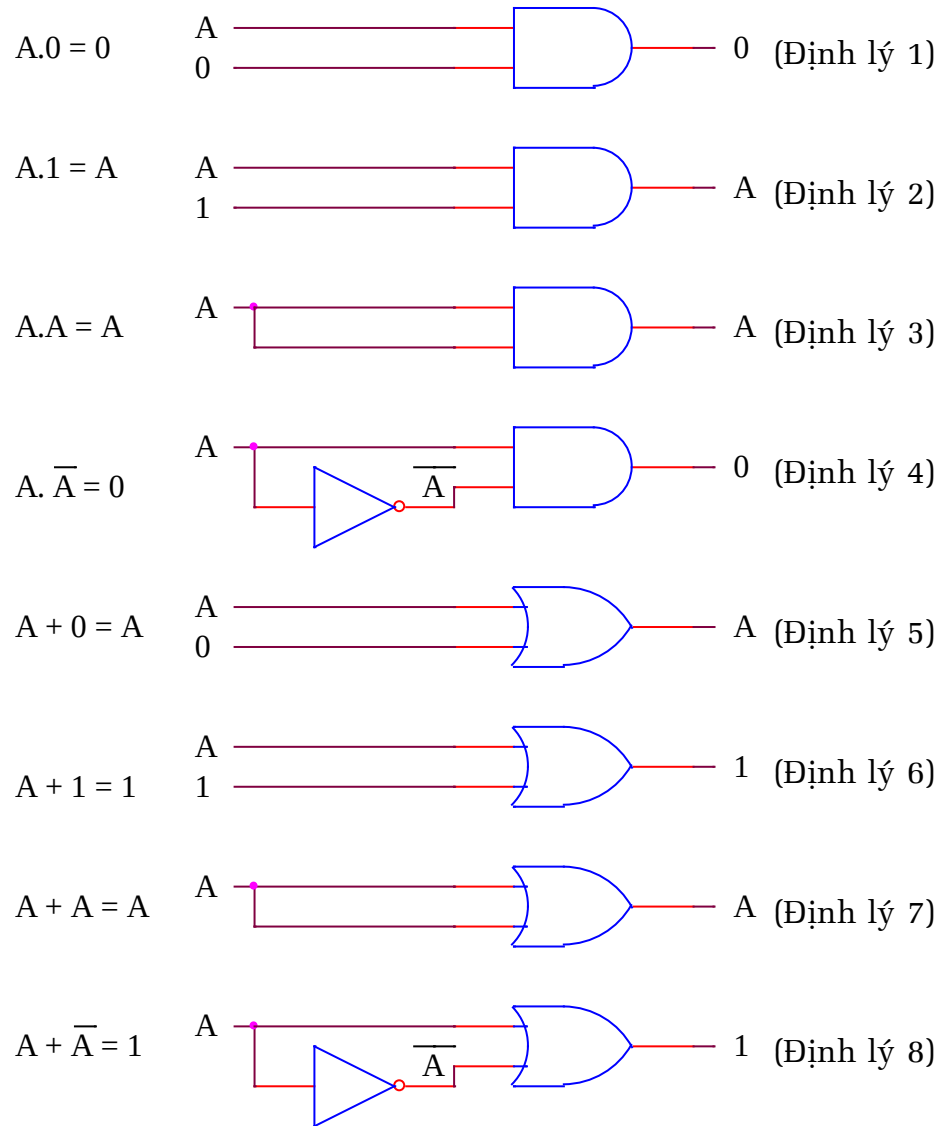
$t1 \rightarrow t2$, $t3 \rightarrow t4$, $t5 \rightarrow t6$, $t7 \rightarrow t8$, $t9 \rightarrow t10$, $t11 \rightarrow t12$, $t13 \rightarrow t14$, $t15 \rightarrow t16$, $t17 \rightarrow \dots$, các trường hợp khác x đều bằng 0. Và chúng ta cũng thấy là giản đồ thời gian của phép toán EX-NOR/cổng EX-NOR là nghịch đảo của phép toán EX-OR/cổng EX-OR .

3.8- Các Định Lý Của Đại Số Logic

Từ những ba phép toán cơ bản NOT, OR, AND, người ta xây dựng một số định lý nhằm đơn giản biểu thức logic hay mạch logic. Các định lý này sẽ được đi kèm với sơ đồ mạch logic minh họa.

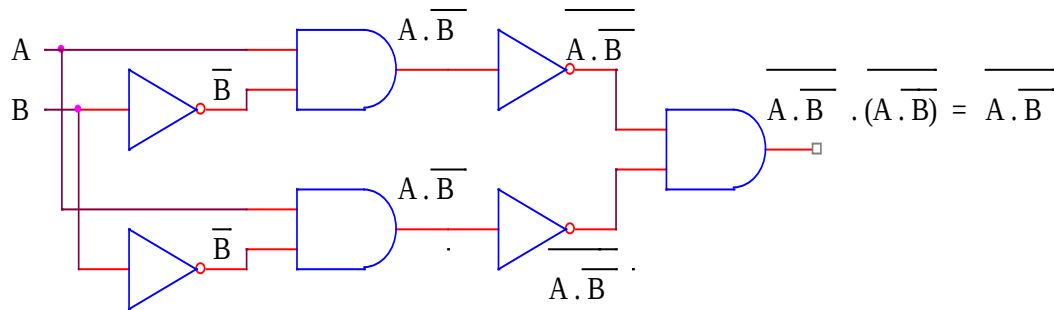
3.8.1- Các định lý cho biểu thức 1 biến hay 1 biến kết hợp với hằng

- Trước hết, ta xét các định lý cho 1 biến hay 1 biến kết hợp với hằng 0 hay hằng 1 như các định lý từ 1→8 ở dưới:

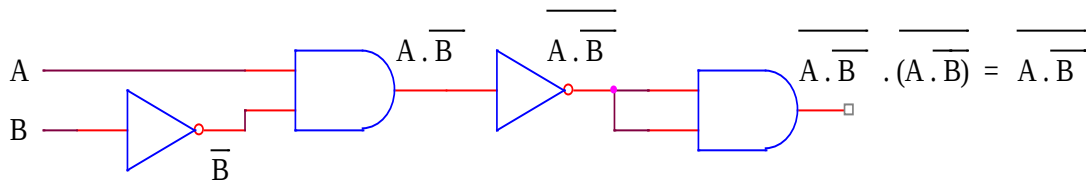


Hình 38. Các định lý cho 1 biến, hay 1 biến kết hợp với 1 hằng

- Tất cả định lý trên đều có thể chứng minh dựa trên bảng thực trị, nhưng riêng với phép toán AND, ta có thể xem nó như là phép nhân cho dễ nhớ. Nếu nhân với 0 thì sẽ bằng 0, còn nhân với 1 sẽ bằng chính nó.
- Ta cũng phải lưu ý đến việc mở rộng phạm vi áp dụng các định lý trên bằng cách dùng phương pháp thay thế một biểu thức logic thành một biến. Như trường hợp định lý 3, với biểu thức $\overline{A.B}.\overline{(A.B)}$, thì ta đặt biến phụ $F = \overline{A.B}$ thì biểu thức logic trên sẽ trở thành là $F.F = F = \overline{A.B} \Rightarrow \overline{A.B}.\overline{(A.B)} = \overline{A.B}$ như hình minh họa dưới.



Hay ta dùng mạch tương đương sau



Hình 39. Mở rộng phạm vi áp dụng cho một biểu thức

3.8.2- Các định lý về luật giao hoán và kết hợp

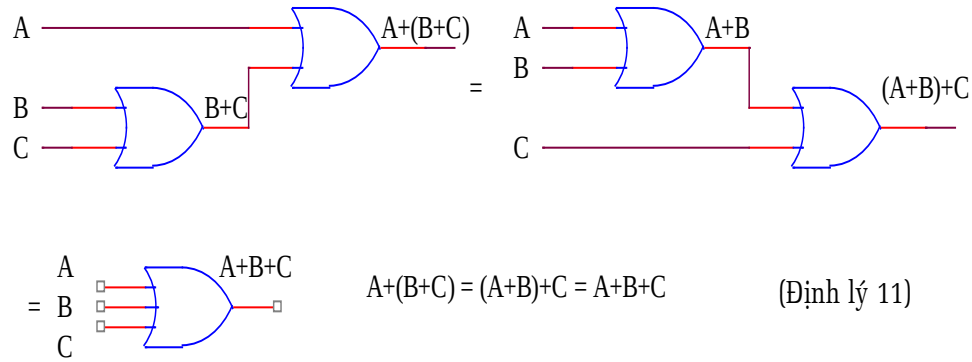
- Ta xét tiếp đến các định lý về luật giao hoán cho phép toán AND và OR. Với hai định lý này, ta thấy là thứ tự các toán hạng là không quan trọng, và ta dễ dàng chứng minh bằng bảng thực trị

$$A.B = B.A \quad \begin{array}{c} A \\ B \end{array} \begin{array}{c} \text{AND} \\ \text{gate} \end{array} \begin{array}{c} A.B \\ \square \end{array} = \begin{array}{c} B \\ A \end{array} \begin{array}{c} \text{AND} \\ \text{gate} \end{array} \begin{array}{c} B.A \\ \square \end{array} \quad (\text{Định lý 9})$$

$$A+B = B+A \quad \begin{array}{c} A \\ B \end{array} \begin{array}{c} \text{OR} \\ \text{gate} \end{array} \begin{array}{c} A+B \\ \square \end{array} = \begin{array}{c} B \\ A \end{array} \begin{array}{c} \text{OR} \\ \text{gate} \end{array} \begin{array}{c} B+A \\ \square \end{array} \quad (\text{Định lý 10})$$

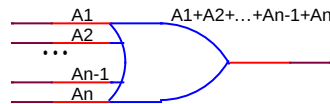
Hình 40. Luật giao hoán cho phép toán AND và OR

- Ta xét tiếp định lý về luật kết hợp cho phép toán OR như hình vẽ dưới.



Hình 41. Luật kết hợp cho phép toán OR

- Thứ tự thực hiện phép toán OR ở đây không quan trọng, Và như vậy ta có thể biểu diễn phép toán này theo nhiều cách như hình vẽ trên minh họa. Và ta cũng có nhận xét là có thể xây dựng cổng OR 3 ngõ nhập để tương ứng với cách biểu diễn $A + B + C$.
- Và ta có thể phát triển định lý 11 cho phép toán OR có N biến, ta biểu diễn nó như sau: $A_0 + A_1 + A_2 + A_3 + \dots + A_{N-2} + A_{N-1}$. Khi thể hiện nó bằng cổng logic, về mặt lý thuyết, ta sẽ thể hiện biểu thức này bằng cổng OR có N ngõ nhập.



Hình 42. Ký hiệu cổng OR có N ngõ nhập

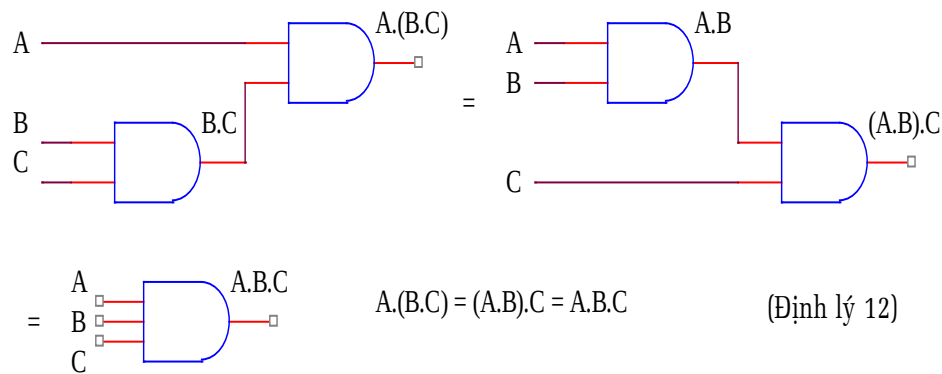
- Lập bảng thực trị cho phép toán OR có N ngõ nhập, ta thấy là giá trị của $x = A_{n-1} + A_{n-2} + \dots + A_2 + A_1 + A_0$ chỉ bằng 0 với một trường hợp duy nhất là $(A_{n-1}, A_{n-2}, \dots, A_2, A_1, A_0) = (0, 0, \dots, 0, 0, 0)$ còn mọi trường hợp khác thì $x = 1$.

A_{n-1}	A_{n-2}	...	A_2	A_1	A_0	$x = A_{n-1} + A_{n-2} + \dots + A_2 + A_1 + A_0$
0	0	...	0	0	0	0
0	0	...	0	0	1	1
.	.	.	.	.	.	.
.	.	.	.	.	.	.
.	.	.	.	.	.	.

1	1	...	1	1	0	1
1	1	...	1	1	1	1

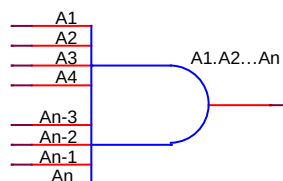
Bảng 28. Bảng thực trị cho phép toán OR có N ngõ nhập

- Về mặt kỹ thuật, các nhà sản xuất cũng đã sản xuất các cổng OR với số ngõ nhập từ 2 ngõ lên đến 8 ngõ. Nhưng do những loại cổng OR có từ 3 ngõ nhập trở lên thì khá hiếm trên thị trường, nên khi thi công ta vẫn phải dùng cổng OR hai ngõ nhập thay thế.
- Ta xét tiếp các định lý về luật kết hợp cho phép toán AND. Thứ tự thực hiện phép toán AND ở đây không quan trọng, và như vậy ta có thể biểu diễn phép toán này theo nhiều cách như hình vẽ dưới. Và ta cũng có nhận xét là có thể xây dựng cổng AND 3 ngõ nhập để tương ứng với cách biểu diễn $A.B.C$



Hình 43. Luật kết hợp cho phép toán AND.

- Và ta có thể phát triển định lý 12 cho phép toán AND có N biến, ta biểu diễn nó như sau: $A_0.A_1.A_2.A_3 \dots A_{N-2}.A_{N-1}$. Khi thể hiện nó bằng cổng logic, về mặt lý thuyết, ta sẽ thể hiện biểu thức này bằng cổng AND có N ngõ nhập.



Hình 44. Ký hiệu cổng AND có N ngõ nhập

- Lập bảng thực trị cho phép toán AND có N ngõ nhập, ta thấy là giá trị của $x = A_{n-1} \cdot A_{n-2} \dots A_2 \cdot A_1 \cdot A_0$ chỉ bằng 1 với một

trường hợp duy nhất là $(A_{n-1}, A_{n-2}, \dots, A_2, A_1, A_0) = (1, 1, \dots, 1, 1, 1)$ còn mọi trường hợp khác thì $x = 0$.

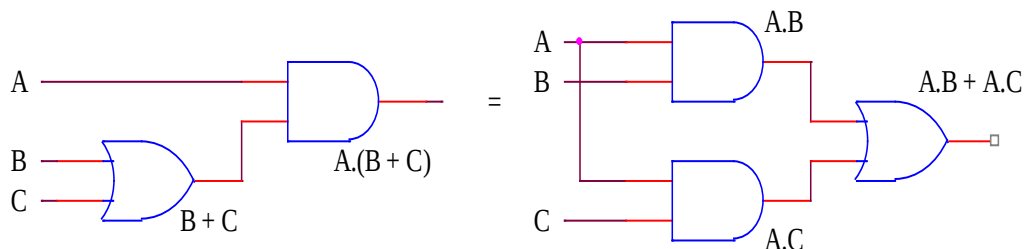
A_{n-1}	A_{n-2}	...	A_2	A_1	A_0	$x = A_{n-1} \cdot A_{n-2} \dots A_2 \cdot A_1 \cdot A_0$
0	0	...	0	0	0	0
0	0	...	0	0	1	0
.	.		.	.	.	.
.	.		.	.	.	.
.	.		.	.	.	.
1	1	...	1	1	0	0
1	1	...	1	1	1	1

Bảng 29. Bảng thực trị cho phép toán AND có N ngõ nhập

- Về mặt kỹ thuật, các nhà sản xuất cũng đã sản xuất các cổng AND với số ngõ nhập từ 2 ngõ lên đến 8 ngõ. Nhưng do những loại cổng AND có từ 3 ngõ nhập trở lên thì khá hiếm trên thị trường, nên khi thi công ta vẫn phải dùng cổng AND hai ngõ nhập thay thế.
- Để chứng minh các định lý về luật kết hợp của phép toán OR và AND này ta dùng bảng thực trị để chứng minh.

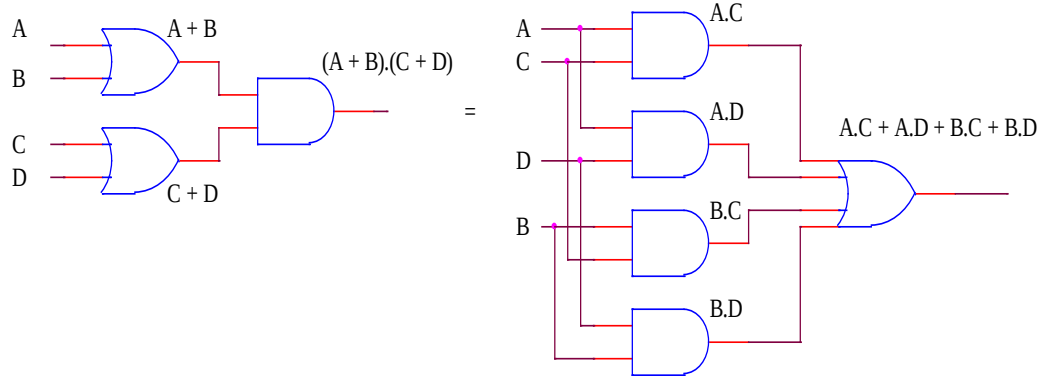
3.8.3- Các định lý về luật phân phối giữa phép toán OR và phép toán AND

(Định lý 13.a) $A.(B+C) = A.B + A.C$



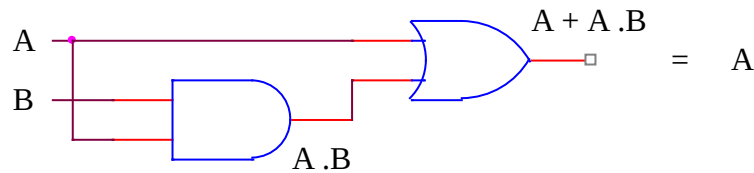
Hình 45. Minh họa định lý 13.a

(Định lý 13.b) $(A+B).(C+D) = A.C + A.D + B.C + B.D$



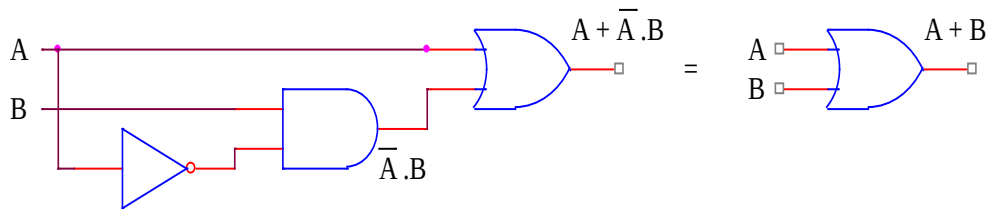
Hình 46. Minh họa định lý 13.b

(Định lý 13.c) $A + A.B = A$



Hình 47. Minh họa định lý 13.c

(Định lý 13.d) $A + \bar{A}.B = A + B$



Hình 48. Minh họa định lý 13.d

- Các định lý (13.a) và (13.b) có thể chứng minh bằng cách dùng bảng thực trị. Còn để dễ nhớ, ta xem luật này giống như cách ta khai triển biểu thức và đặt thừa số chung trong đại số thông thường.
- Các định lý (13.c) và (13,d) là đặc thù riêng của đại số logic, ta có thể dùng bảng thực trị để chứng minh hay áp dụng các định lý trước đó để chứng minh.
- Ta dùng các định lý trước để chứng minh định lý (13.c):

$$A + A.B = A.1 + A.B$$

áp dụng định lý (2) $A = A.1$

$$A.1 + A.B = A(1 + B)$$

áp dụng định lý (13.a) $A.B + A.C = A.(B+C)$

$$A(1 + B) = A . 1$$

áp dụng định lý (6) $1 + B = 1.$

$$A.1 = A$$

áp dụng định lý (2) $A.1 = A$

Vậy: $A + A.B = A$ định lý đã được chứng minh.

➤ Ta dùng các định lý trước để chứng minh định lý (13.d):

$$A + \bar{A}.B = A.1 + \bar{A}.B$$

áp dụng định lý (2) $A = A.1$

$$A.1 + \bar{A}.B = A.(B + \bar{B}) + \bar{A}.B$$

áp dụng định lý (8) $1 = B + \bar{B}$

$$A.(B + \bar{B}) + \bar{A}.B = A.B + A.\bar{B} + \bar{A}.B$$

áp dụng định lý (13.a) $A.(B + \bar{B}) = A.B + A.\bar{B}$

$$A.B + A.\bar{B} + \bar{A}.B = A.B + A.B + A.\bar{B} + \bar{A}.B$$

áp dụng định lý (3) $A.B = (A.B).(A.B)$

$$A.B + A.B + A.\bar{B} + \bar{A}.B = A.(B + \bar{B}) + B.(A + \bar{A})$$

áp dụng định lý (13.a) $A.B + A.\bar{B} = A.(B + \bar{B})$ và $A.B + \bar{A}.B = B.(A + \bar{A})$

$$A.(B + \bar{B}) + B.(A + \bar{A}) = A.1 + B.1$$

áp dụng định lý (8) $B + \bar{B} = 1$ và $A + \bar{A} = 1$

$$A.1 + B.1 = A + B$$

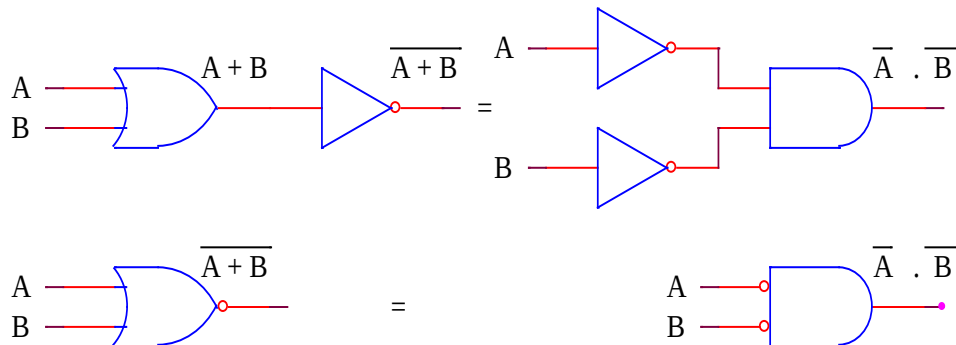
áp dụng định lý (2) $A.1 = A$ và $B.1 = B$

Vậy: $A + \bar{A}.B = A + B$ định lý đã được chứng minh.

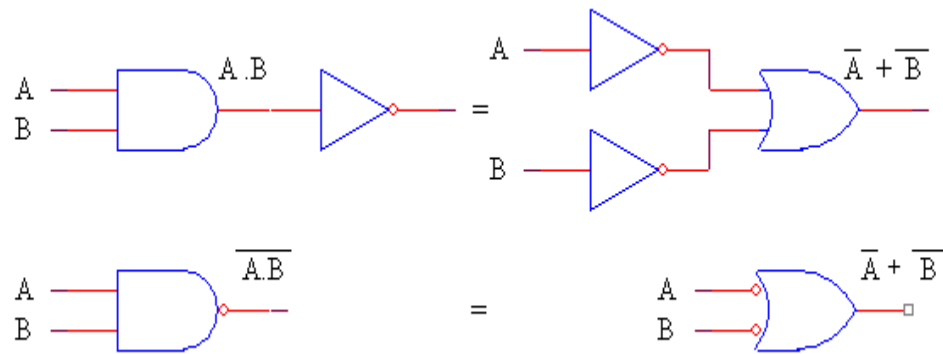
3.8.4- Các định lý DeMorgan

(Định lý 14) $\overline{A+B} = \bar{A} . \bar{B}$

(Định lý 15) $\overline{A.B} = \bar{A} + \bar{B}$

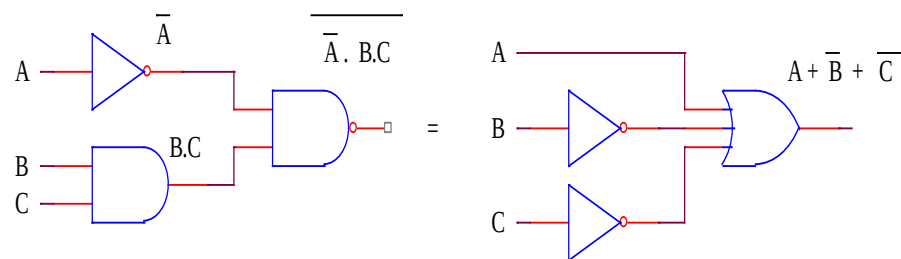


Hình 49. Minh họa cho định lý 14



Hình 50. Minh họa cho định lý 15

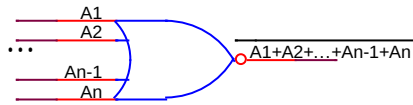
- Ta có thể chứng minh hai định lý trên bằng cách lập bảng thực trị. Đây là hai định lý rất quan trọng trong việc rút gọn các biểu thức có dạng phủ định của một tổng hay dạng phủ định của một tích.
- Nhận xét: định lý số 14 chính là phép toán NOR/cổng NOR, định lý 15 chính là phép toán NAND/cổng NAND. Với định lý DeMorgan, ta có một cách biểu diễn khác cho hai cổng trên như vẽ phải của các hình trên.
- Ta cũng phải lưu ý đến việc mở rộng phạm vi áp dụng các định lý trên bằng cách dùng phương pháp thay thế một biểu thức logic thành một biến. Như trường hợp định lý 14, với biểu thức $\overline{A \cdot B \cdot C}$, thì ta đặt biến phụ $F = \overline{A}$ và $G = B \cdot C$ thì biểu thức logic trên sẽ trở thành là $\overline{F \cdot G} = \overline{F} + \overline{G} = \overline{\overline{A}} + \overline{B \cdot C} = A + \overline{B} + \overline{C} \Rightarrow \overline{A \cdot B \cdot C} = A + \overline{B} + \overline{C}$ như hình minh họa dưới.



Hình 51. Mở rộng phạm vi áp dụng cho một biểu thức

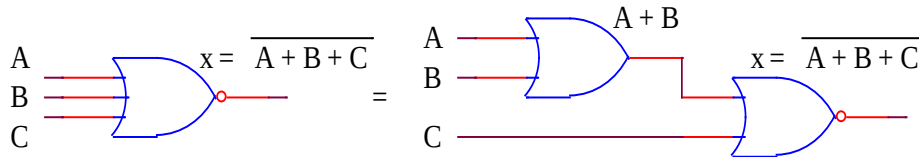
- Và ta có thể phát triển định lý 14 cho phép toán NOR có N biến, ta biểu diễn nó như sau: $A_1 + A_2 + A_3 + \dots + A_{N-1} + A_N$.

Khi thể hiện nó bằng cổng logic, về mặt lý thuyết, ta sẽ thể hiện biểu thức này bằng cổng NOR có N ngõ nhập.



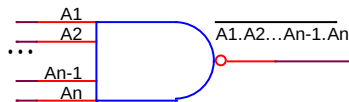
Hình 52. Ký hiệu cổng NOR có N ngõ nhập

- Về mặt kỹ thuật, các nhà sản xuất cũng đã sản xuất các cổng NOR với số ngõ nhập từ 2 ngõ lên đến 8 ngõ. Nhưng do những loại cổng OR có từ 3 ngõ nhập trở lên thì khá hiếm trên thị trường, nên khi thi công ta vẫn phải dùng cổng NOR hai ngõ nhập và cổng OR hai ngõ nhập để thay thế. Như trường hợp cổng NOR 3 ngõ nhập dưới, phải thay thế bằng 1 cổng OR hai ngõ nhập và 1 cổng NOR 2 ngõ nhập.



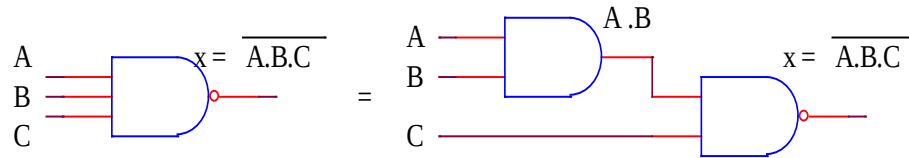
Hình 53. Thay thế cổng NOR 3 ngõ nhập

- Và ta có thể phát triển định lý 15 cho phép toán NAND có N biến, ta biểu diễn nó như sau: $A_1 + A_2 + A_3 + \dots + A_{N-1} + A_N$. Khi thể hiện nó bằng cổng logic, về mặt lý thuyết, ta sẽ thể hiện biểu thức này bằng cổng NAND có N ngõ nhập.



Hình 54. Ký hiệu cổng NAND có N ngõ nhập

- Về mặt kỹ thuật, các nhà sản xuất cũng đã sản xuất các cổng NAND với số ngõ nhập từ 2 ngõ lên đến 8 ngõ. Nhưng do những loại cổng NAND có từ 3 ngõ nhập trở lên thì khá hiếm trên thị trường, nên khi thi công ta vẫn phải dùng cổng NAND hai ngõ nhập và cổng AND hai ngõ nhập để thay thế. Như trường hợp cổng NAND 3 ngõ nhập dưới, phải thay thế bằng 1 cổng AND hai ngõ nhập và 1 cổng NAND 2 ngõ nhập.

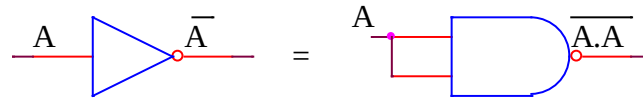


Hình 55. Thay thế cổng NAND 3 ngõ nhập

3.8.5- Tính đa dụng của phép toán NOR/ cổng NOR và phép toán NAND/ cổng NAND

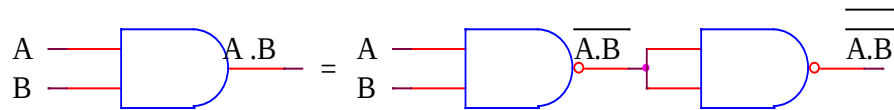
➤ Như trên đã nói, có thể dùng duy nhất một loại cổng NOR hay NAND để thiết kế mạch logic. Ta sẽ chứng minh bằng các định lý đã nêu ở phần trước.

➤ Thay thế cổng NOT bằng cổng NAND: $\overline{A} = \overline{A.A}$



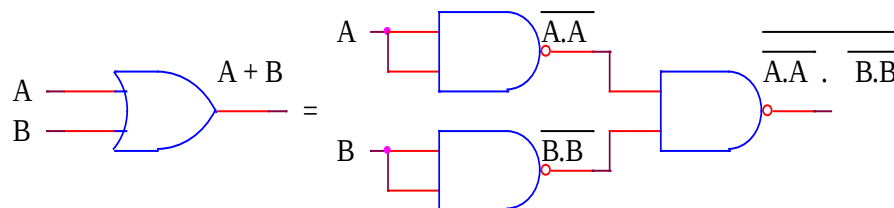
Hình 56. Thay thế một cổng NOT bằng một cổng NAND

➤ Thay thế cổng AND bằng cổng NAND: $A.B = \overline{\overline{A.B}}$



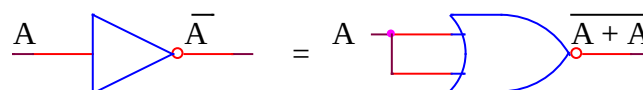
Hình 57. Thay thế một cổng AND bằng hai cổng NAND

➤ Thay thế cổng OR bằng cổng NAND: $A + B = \overline{\overline{A.A} . \overline{B.B}}$



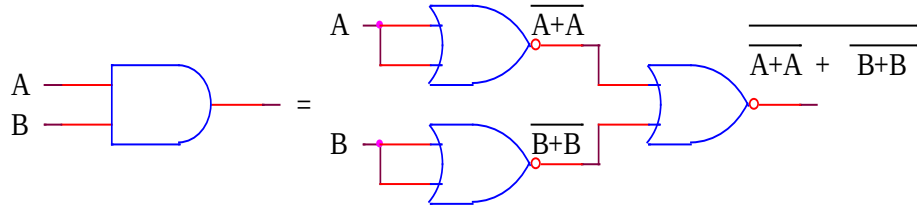
Hình 58. Thay thế một cổng OR bằng ba cổng NAND

➤ Thay thế cổng NOT bằng cổng NOR: $\overline{A} = \overline{A + A}$



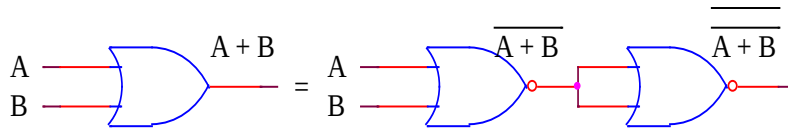
Hình 59. Thay thế một cổng NOT bằng một cổng NOR

- Thay thế cổng AND bằng cổng NOR: $A.B = \overline{\overline{A+A} + \overline{B+B}}$



Hình 60. Thay thế một cổng AND bằng ba cổng NOR

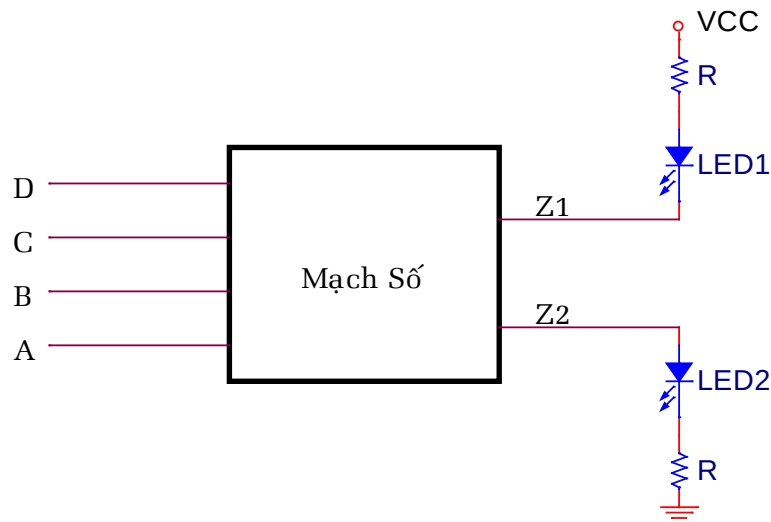
- Thay thế cổng OR bằng cổng NOR: $A + B = \overline{\overline{A+B}}$



Hình 61. Thay thế một cổng OR bằng hai cổng NOR

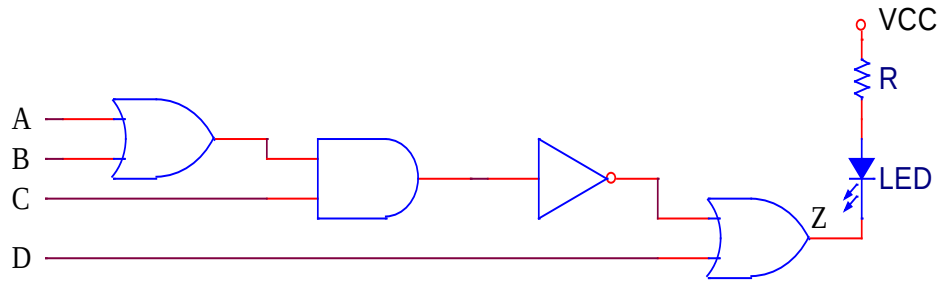
3.8.6- Mức logic tích cực

- Cho một mạch tổ hợp có sơ đồ khối như hình vẽ, đèn LED1 sẽ sáng khi $Z_1 = 0$ và đèn LED2 sẽ sáng khi $Z_2 = 1$. Trong trường hợp này ta nói, Z_1 là ngõ xuất tích cực mức thấp và Z_2 là ngõ xuất tích cực mức cao.



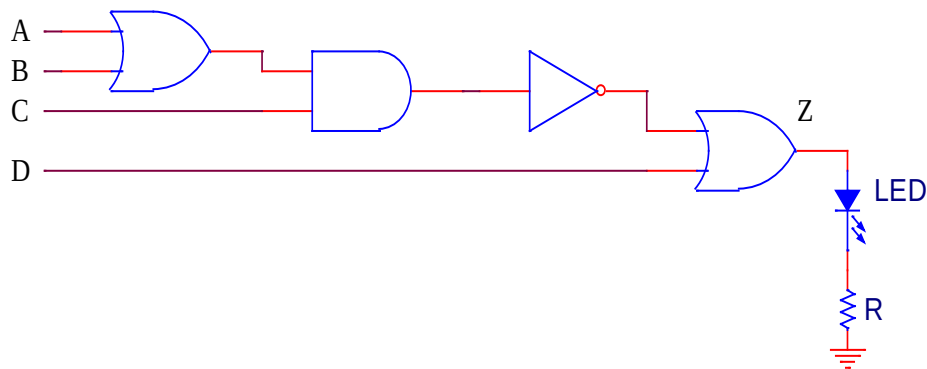
Hình 62. Sơ đồ khối cho khái niệm mức logic tích cực.

- Để minh họa rõ khái niệm mức logic tích cực, ta xét một trường hợp cụ thể sau:



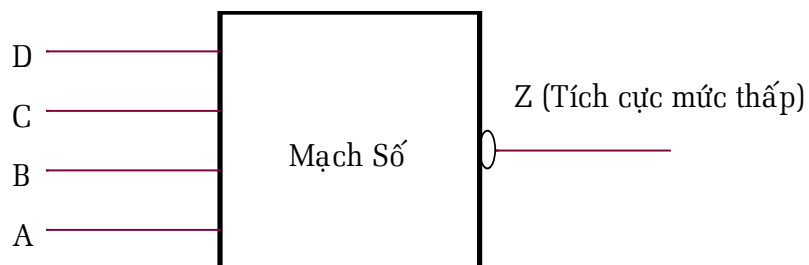
Hình 63. Z là ngõ xuất tích cực mức thấp.

- Cũng lấy mạch tổ hợp này, nhưng lần này nối với Led theo cách khác.

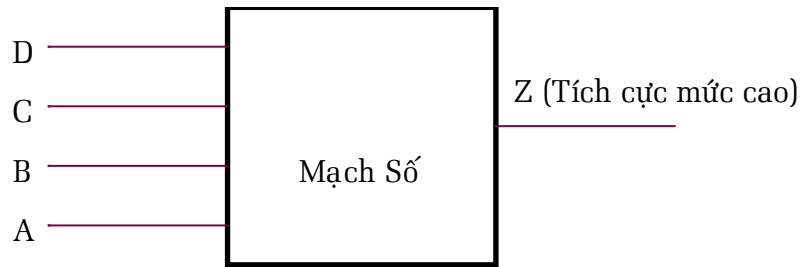


Hình 64. Z là ngõ xuất tích cực mức cao.

- Với mạch tổ hợp không thay đổi, chỉ nối với đèn LED theo hai cách khác nhau, ta có khái niệm ngõ xuất tích cực mức thấp và tích cực mức cao. Mạch trước, đèn sáng khi $Z = 0$, ta nói mạch này có ngõ xuất tích cực mức thấp. Mạch sau, đèn sáng khi $Z = 1$, ta nói mạch này có ngõ xuất tích cực mức cao. Còn bản chất của mạch tổ hợp không có gì thay đổi.
- Về mặt ký hiệu, thì sử dụng dấu tròn ($\circ$) để biểu thị tích cực mức thấp và bình thường để biểu thị tích cực mức cao.



Hình 65. Ký hiệu cho ngõ xuất tích cực mức thấp.

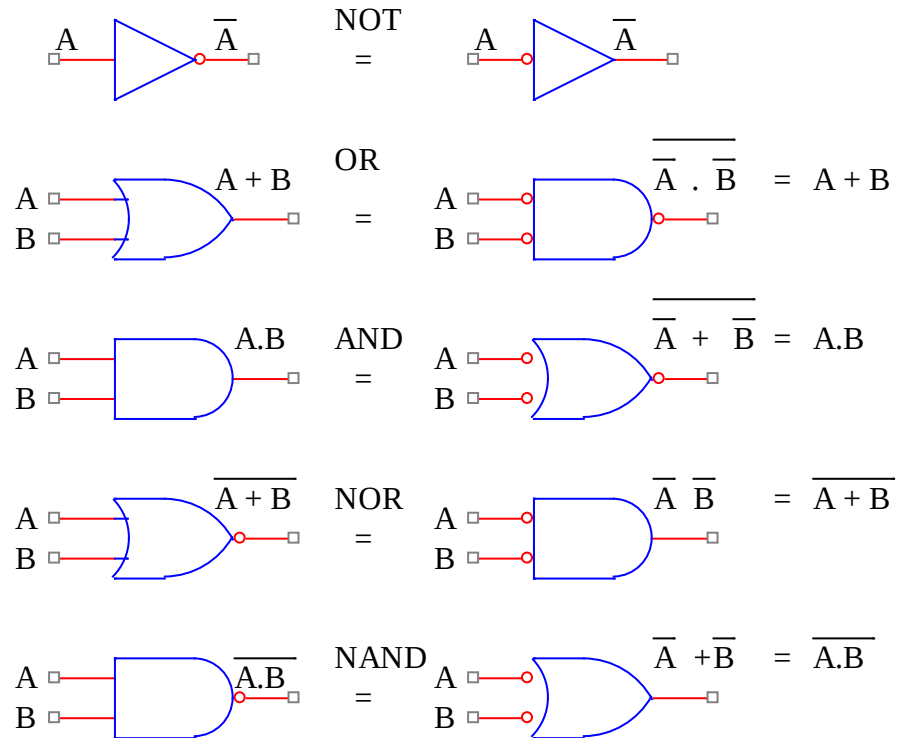


Hình 66. Ký hiệu cho ngõ xuất tích cực mức cao.

- Và khái niệm mức logic tích cực cũng áp dụng cho cả ngõ nhập, với ngõ nhập nào bằng 1 mà ngõ xuất tích cực thì ta nói ngõ nhập đó tích cực mức cao, ngõ nhập bằng 0 mà ngõ xuất tích cực thì ta nói ngõ nhập đó tích cực mức thấp, và ký hiệu mức logic tích cực cũng như ngõ xuất.
- Khái niệm mức logic tích cực được sử dụng với mục đích là phân tích mạch bằng lý luận (không dùng bảng thực trị) để hiểu được cách hoạt động của mạch.

3.8.7- Ký hiệu logic chuẩn và ký hiệu logic thay thế

- Với khái niệm mức logic tích cực đã đề cập trên, ta sẽ dùng chúng để thay đổi ký hiệu 5 loại cổng NOT, OR, AND, NOR, NAND. Những ký hiệu đó được gọi là ký hiệu thay thế. Và chúng được dùng khi phân tích mạch bằng lý luận.

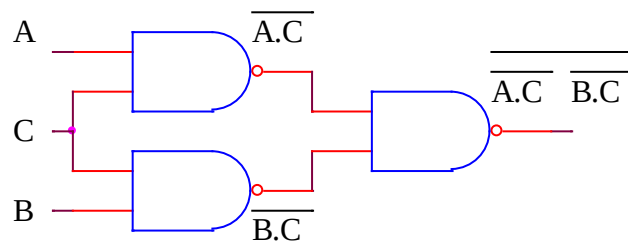


Hình 67. Ký hiệu chuẩn và các ký hiệu thay thế.

- Nhận xét 1: có thể mở rộng cho số ngõ nhập tới N.
- Nhận xét 2: toàn bộ ngõ nhập của ký hiệu chuẩn không có dấu tròn, còn toàn bộ ngõ nhập của ký hiệu thay thế đều có dấu tròn.

3.8.8- Ứng dụng của việc sử dụng ký hiệu logic thay thế

- Cho một sơ đồ mạch như sau:

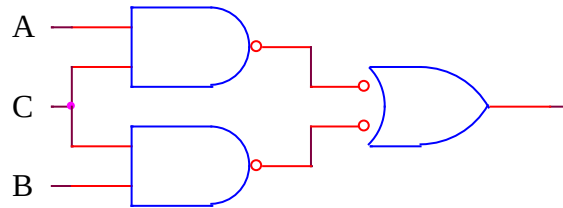


Hình 68. Mạch dùng toàn cổng NAND

- Rất khó nhận xét mạch hoạt động như thế nào, chỉ có thể nhận xét được sau khi lập xong bảng thực trị.

- Ta sẽ thay đổi ký hiệu logic chuẩn bằng ký hiệu logic thay thế để có thể dễ nhận xét mạch theo hai cách sau

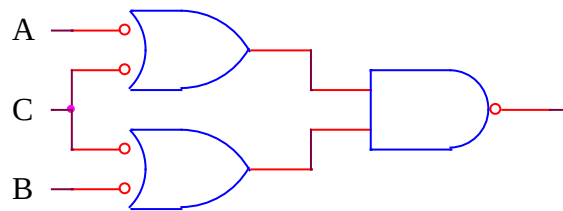
- Cách 1:



Hình 69. Thay cổng NAND cuối bằng ký hiệu thay thế.

- Nhìn vào hình vẽ ta nói, ngõ xuất của mạch bằng 1 khi ($A=1$ và $C=1$) hay ($B=1$ và $C=1$).

- Cách 2:

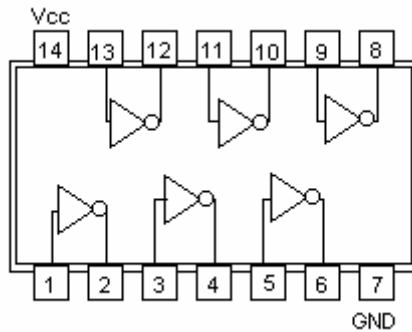


Hình 70. Thay thế hai cổng NAND đầu bằng ký hiệu thay thế.

- Nhìn vào hình vẽ ta nói, ngõ xuất của mạch bằng 0 khi ($A=0$ hoặc $B=0$) và ($C=0$).
- Lưu ý, khi dùng các ký hiệu thay thế, phải bảo đảm nguyên tắc khi nối giữa các cổng thì dấu tròn (°) phải nối với dấu tròn (°), không dấu nối với không dấu.
- Cách thay thế thứ nhất, dùng khi ngõ xuất là tích cực mức cao. Cách thay thế thứ hai dùng khi ngõ xuất là tích cực mức thấp.

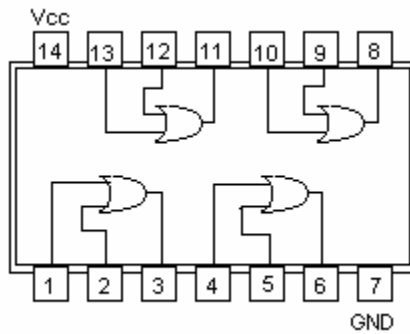
3.9- Các Vi Mạch Chứa Các Cổng Logic

3.9.1- Vi mạch chứa cổng NOT



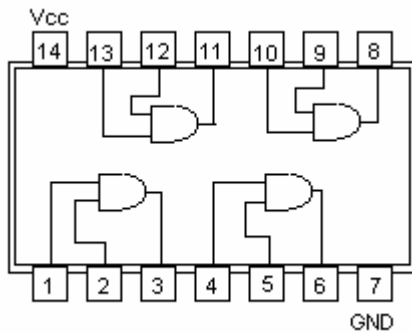
Hình 71. IC 7404 chứa sáu cổng NOT

3.9.2- Vi mạch chứa cổng OR



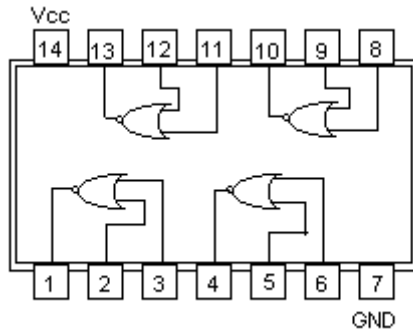
Hình 72. IC 7432 chứa 4 cổng OR

3.9.3- Vi mạch chứa cổng AND



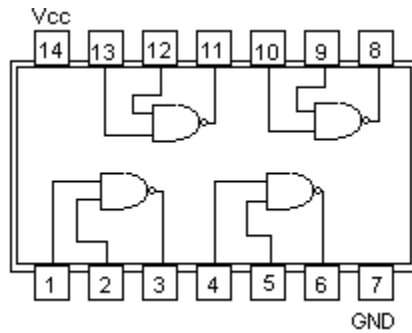
Hình 73. IC 7408 chứa 4 cổng NAND

3.9.4- Vi mạch chứa cổng NOR



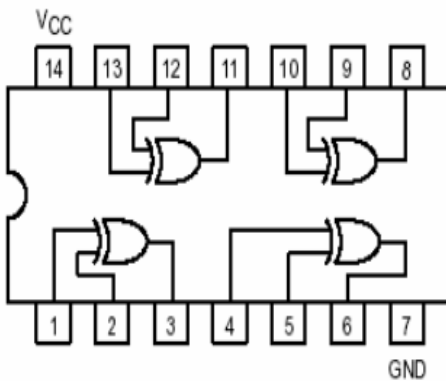
Hình 74. IC 7402 chứa 4 cổng NOR

3.9.5- Vi mạch chứa cổng NAND



Hình 75. IC 7400 chứa 4 cổng NAND

3.9.6- Vi mạch chứa cổng EX-OR



Hình 76. IC 7486 chứa 4 cổng EX-OR

3.10- Biểu Diễn Hàm Logic Theo Các Dạng Chuẩn

3.10.1- Dạng chuẩn tuyến và dạng chuẩn hội

- Để biểu diễn hàm logic bằng phương pháp đại số, có hai dạng chuẩn: **dạng chuẩn tuyến** (còn được gọi là dạng chuẩn giao hay tiếng Anh gọi là **minterm** hay **sum of products**) và **dạng chuẩn hội** (tiếng Anh gọi là **maxterm** hay **product of sums**).
- Các phép toán logic dùng ở đây là phép toán OR và phép toán AND, riêng phép toán NOT chỉ dùng cho 1 biến.
- Trong dạng chuẩn tuyến, mỗi biểu thức là một phép toán AND có N biến, các biến của phép toán AND có thể ở dạng bình thường hay qua phép toán NOT, tất cả các biểu thức đó sẽ là biến đầu vào của phép toán OR có N biến.
- Trong dạng chuẩn hội, mỗi biểu thức là một phép toán OR có N biến, các biến của phép toán OR có thể ở dạng bình thường hay qua phép toán NOT, tất cả các biểu thức đó sẽ là biến đầu vào của phép toán AND có N biến
- Ta sẽ đi lần lượt từ hàm logic có một biến đến N biến

3.10.2- Hàm logic 1 biến $x = f(A)$

- Lập bảng thực trị:

A	$x = f(A)$
0	x_0
1	x_1

Bảng 30. Bảng thực trị với $x_i \in \{0,1\}$

- Công thức đại số dạng chuẩn tuyến: $x = \bar{A}.x_0 + A.x_1$
- Công thức đại số dạng chuẩn hội: $x = (A + x_0).(\bar{A} + x_1)$
- Với hàm một biến, ta liệt kê tất cả trường hợp có thể xảy ra là $2^1 = 2^2 = 4$ trường hợp.
- Trường hợp 1: $x_0 = x_1 = 0 \Rightarrow x = 0$ (Đây là hằng 0)

A	$x = f(A)$
0	0

1	0
---	---

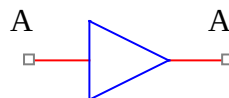
Bảng 31. Bảng thực trị với $x_0 = x_1 = 0$

- Trường hợp 1-dạng chuẩn tuyển: $x = \bar{A}.x_0 + A.x_1 \Rightarrow x = \bar{A}.0 + A.0 = 0$
- Trường hợp 1-dạng chuẩn hội: $x = (A+x_0).(\bar{A}+x_1) \Rightarrow x = (\bar{A}+0).(A+0) = 0$
- Trường hợp 2: $x_0 = 0$ & $x_1 = 1 \Rightarrow x = A$

A	$x = f(A)$
0	0
1	1

Bảng 32. Bảng thực trị với $x_0 = 0$ & $x_1 = 1$

- Trường hợp 2-dạng chuẩn tuyển: $x = \bar{A}.x_0 + A.x_1 \Rightarrow x = \bar{A}.0 + A.1 = A$
- Trường hợp 2-dạng chuẩn hội: $x = (A+x_0).(\bar{A}+x_1) \Rightarrow x = (A+0).(\bar{A}+1) = A$
- Trong trường hợp $x = A$, về mặt lý thuyết, thì nối thẳng biến A vào mạch, không thông qua cổng nào. Trong thực tế thì công mạch, nếu có sự suy giảm điện thế của các ngõ xuất, ta thường thiết kế cổng đệm với ngõ xuất bằng ngõ nhập, để bảo đảm mức logic của tín hiệu. Hay trong trường hợp làm trung gian các loại cổng có công nghệ chế tạo khác nhau như giữa TTL và CMOS...



Bảng 33. Ký hiệu cổng đệm

- Trường hợp 3: $x_0 = 1$ & $x_1 = 0 \Rightarrow x = \bar{A}$ (Đây là cổng NOT)

A	$x = f(A)$
0	1
1	0

Bảng 34. Bảng thực trị với $x_0 = 1$ & $x_1 = 0$

- Trường hợp 3-dạng chuẩn tuyến: $x = \bar{A}.x_0 + A.x_1 \Leftrightarrow x = \bar{A}.1 + A.0 = \bar{A}$
- Trường hợp 3-dạng chuẩn hội: $x = (A+x_0).(\bar{A}+x_1) \Leftrightarrow x = (A+1).(\bar{A}+0) = \bar{A}$
- Trường hợp 4: $x_0 = x_1 = 1 \Leftrightarrow x = 1$ (Đây là hằng 1)

A	$x = f(A)$
0	1
1	1

Bảng 35. Bảng thực trị với $x_0 = x_1 = 1$

- Trường hợp 4-dạng chuẩn tuyến: $x = \bar{A}.x_0 + A.x_1 \Leftrightarrow x = \bar{A}.1 + A.1 = 1$
- Trường hợp 4-dạng chuẩn hội: $x = (A+x_0).(\bar{A}+x_1) \Leftrightarrow x = (A+1).(\bar{A}+1) = 1$

3.10.3- Hàm 2 biến $x = f(B, A)$

- Lập bảng thực trị:

B	A	$x = f(B, A)$
0	0	x_0
0	1	x_1
1	0	x_2
1	1	x_3

Bảng 36. Bảng thực trị với $x_i \in \{0,1\}$

- Công thức đại số dạng chuẩn tuyến: $x = \bar{B}.\bar{A}.x_0 + \bar{B}.A.x_1 + B.\bar{A}.x_2 + B.A.x_3$
- Công thức đại số dạng chuẩn hội: $x = (B+A+x_0).(B+\bar{A}+x_1).(\bar{B}+A+x_2).(\bar{B}+\bar{A}+x_3)$
- Với hàm hai biến, ta liệt kê tất cả trường hợp có thể xảy ra là $2^2 = 2^4 = 16$ trường hợp. Ở đây, chỉ viết công thức dạng chuẩn tuyến.
- Trường hợp 1: $x_0 = x_1 = x_2 = x_3 = 0 \Leftrightarrow x = 0$ (Đây là hằng 0)

B	A	$x = f(B, A)$
0	0	0

0	1	0
1	0	0
1	1	0

Bảng 37. Bảng thực trị với $x_0 = x_1 = x_2 = x_3 = 0$

➤ Trường hợp 1- dạng chuẩn tuyến: $x = \bar{B} \cdot \bar{A} \cdot 0 + \bar{B} \cdot A \cdot 0 + B \cdot \bar{A} \cdot 0 + B \cdot A \cdot 0 = 0$

➤ Trường hợp 2: $x_0 = 1$ & $x_1 = x_2 = x_3 = 0 \Rightarrow x = \bar{B} \cdot \bar{A}$ (Đây là cổng NOR)

B	A	$x = f(B, A)$
0	0	1
0	1	0
1	0	0
1	1	0

Bảng 38. Bảng thực trị với $x_0 = 1$ & $x_1 = x_2 = x_3 = 0$

➤ Trường hợp 2- dạng chuẩn tuyến: $x = \bar{B} \cdot \bar{A} \cdot 1 + \bar{B} \cdot A \cdot 0 + B \cdot \bar{A} \cdot 0 + B \cdot A \cdot 0 = \bar{B} \cdot \bar{A}$

➤ Trường hợp 3: $x_1 = 1$ & $x_0 = x_2 = x_3 = 0$

B	A	$x = f(B, A)$
0	0	0
0	1	1
1	0	0
1	1	0

Bảng 39. Bảng thực trị với $x_1 = 1$ & $x_0 = x_2 = x_3 = 0$

➤ Trường hợp 3- dạng chuẩn tuyến: $x = \bar{B} \cdot \bar{A} \cdot 0 + \bar{B} \cdot A \cdot 1 + B \cdot \bar{A} \cdot 0 + B \cdot A \cdot 0 = \bar{B} \cdot A$

➤ Trường hợp 4: $x_0 = x_1 = 1$ & $x_2 = x_3 = 0 \Rightarrow x = \bar{B}$

B	A	$x = f(B, A)$
0	0	1
0	1	1
1	0	0
1	1	0

Bảng 40. Bảng thực trị với $x_0 = x_1 = 1$ & $x_2 = x_3 = 0$

- Trường hợp 4- dạng chuẩn tuyến: $x = \bar{B}. \bar{A}. 1 + \bar{B}. A. 1 + B. \bar{A}. 0 + B. A. 0 = \bar{B}. \bar{A} + \bar{B}. A = \bar{B}$

- Trường hợp 5: $x_0 = x_1 = x_3 = 0$ & $x_2 = 1 \Rightarrow x = B. \bar{A}$

B	A	x = f(B, A)
0	0	0
0	1	0
1	0	1
1	1	0

Bảng 41. Bảng thực trị với $x_0 = x_1 = x_3 = 0$ & $x_2 = 1$

- Trường hợp 5- dạng chuẩn tuyến: $\Rightarrow x = \bar{B}. \bar{A}. 0 + \bar{B}. A. 0 + B. \bar{A}. 1 + B. A. 0 = B. \bar{A}$

- Trường hợp 6: $x_0 = x_2 = 1$ & $x_1 = x_3 = 0 \Rightarrow x = \bar{A}$

B	A	x = f(B, A)
0	0	1
0	1	0
1	0	1
1	1	0

Bảng 42. Bảng thực trị với $x_0 = x_2 = 1$ & $x_1 = x_3 = 0$

- Trường hợp 6- dạng chuẩn tuyến: $x = \bar{B}. \bar{A}. 1 + \bar{B}. A. 0 + B. \bar{A}. 1 + B. A. 0 = \bar{B}. \bar{A} + B. \bar{A} = \bar{A}$

- Trường hợp 7: $x_1 = x_2 = 1$ & $x_0 = x_3 = 0 \Rightarrow x = B \oplus A$ (Đây là cổng EX-OR)

B	A	x = f(B, A)
0	0	0
0	1	1
1	0	1
1	1	0

Bảng 43. Bảng thực trị với $x_1 = x_2 = 1$ & $x_0 = x_3 = 0$

- Trường hợp 7- dạng chuẩn tuyến: $x = \bar{B}. \bar{A}. 0 + \bar{B}. A. 1 + B. \bar{A}. 1 + B. A. 0 = \bar{B}. A + B. \bar{A} = B \oplus A$

- Trường hợp 8: $x_0 = x_1 = x_2 = 1$ & $x_3 = 0 \Rightarrow x = \bar{B} + \bar{A}$ (Đây là cổng NAND)

B	A	x = f(B, A)
---	---	-------------

0	0	1
0	1	1
1	0	1
1	1	0

Bảng 44. Bảng thực trị với $x_0 = x_1 = x_2 = 1$ & $x_3 = 0$

- Trường hợp 8- dạng chuẩn tuyến: $x = \bar{B} \cdot \bar{A} \cdot 1 + \bar{B} \cdot A \cdot 1 + B \cdot \bar{A} \cdot 1 + B \cdot A \cdot 0 = \bar{B} \cdot \bar{A} + \bar{B} \cdot A + B \cdot \bar{A} = \bar{B} \cdot \bar{A} + \bar{B} \cdot A + B \cdot \bar{A} + B \cdot A = \bar{B} + \bar{A}$

- Trường hợp 9: $x_3 = 1$ & $x_0 = x_1 = x_2 = 0 \Rightarrow x = B \cdot A$ (Đây là cổng AND là trường hợp đảo của trường hợp 8)

B	A	$x = f(B, A)$
0	0	0
0	1	0
1	0	0
1	1	1

Bảng 45. Bảng thực trị với $x_3 = 1$ & $x_0 = x_1 = x_2 = 0$

- Trường hợp 9- dạng chuẩn tuyến: $x = \bar{B} \cdot \bar{A} \cdot 0 + \bar{B} \cdot A \cdot 0 + B \cdot \bar{A} \cdot 0 + B \cdot A \cdot 1 = B \cdot A$

- Trường hợp 10: $x_0 = x_3 = 1$ & $x_1 = x_2 = 0 \Rightarrow x = \overline{A \oplus B}$ (Đây là cổng EX-NOR)

B	A	$x = f(B, A)$
0	0	1
0	1	0
1	0	0
1	1	1

Bảng 46. Bảng thực trị với $x_0 = x_3 = 1$ & $x_1 = x_2 = 0$

- Trường hợp 10- dạng chuẩn tuyến: $x = \bar{B} \cdot \bar{A} \cdot 1 + \bar{B} \cdot A \cdot 0 + B \cdot \bar{A} \cdot 0 + B \cdot A \cdot 1 = \bar{B} \cdot \bar{A} + B \cdot A = \overline{A \oplus B}$

- Trường hợp 11: $x_1 = x_3 = 1$ & $x_0 = x_2 = 0 \Rightarrow x = A$

B	A	$X = f(B, A)$
0	0	0
0	1	1
1	0	0
1	1	1

Bảng 47. Bảng thực trị với $x_1 = x_3 = 1$ & $x_0 = x_2 = 0$

- Trường hợp 11- dạng chuẩn tuyến: $x = \bar{B} \cdot \bar{A} \cdot 0 + \bar{B} \cdot A \cdot 1 + B \cdot \bar{A} \cdot 0 + B \cdot A \cdot 1 = \bar{B} \cdot A + B \cdot A = A$

- Trường hợp 12: $x_0 = x_1 = x_3 = 1$ & $x_2 = 0 \Rightarrow x = \bar{B} + A$

B	A	X = f(B, A)
0	0	1
0	1	1
1	0	0
1	1	1

Bảng 48. Bảng thực trị với $x_0 = x_1 = x_3 = 1$ & $x_2 = 0$

- Trường hợp 12- dạng chuẩn tuyến: $x = \bar{B} \cdot \bar{A} \cdot 1 + \bar{B} \cdot A \cdot 1 + B \cdot \bar{A} \cdot 0 + B \cdot A \cdot 1 = \bar{B} \cdot \bar{A} + \bar{B} \cdot A + B \cdot A = \bar{B} \cdot \bar{A} + \bar{B} \cdot A + B \cdot A + \bar{B} \cdot A = \bar{B} + A$

- Trường hợp 13: $x_0 = x_1 = 0$ & $x_2 = x_3 = 1 \Rightarrow x = B$

B	A	x = f(B, A)
0	0	0
0	1	0
1	0	1
1	1	1

Bảng 49. Bảng thực trị với $x_0 = x_1 = 0$ & $x_2 = x_3 = 1$

- Trường hợp 13- dạng chuẩn tuyến: $x = \bar{B} \cdot \bar{A} \cdot 0 + \bar{B} \cdot A \cdot 0 + B \cdot \bar{A} \cdot 1 + B \cdot A \cdot 1 = B \cdot \bar{A} + B \cdot A = B$

- Trường hợp 14: $x_1 = 0$ & $x_0 = x_2 = x_3 = 1 \Rightarrow x = B + \bar{A}$

B	A	x = f(B, A)
0	0	1
0	1	0
1	0	1
1	1	1

Bảng 50. Bảng thực trị với $x_1 = 0$ & $x_0 = x_2 = x_3 = 1$

- Trường hợp 14- dạng chuẩn tuyến: $x = \bar{B} \cdot \bar{A} \cdot 1 + \bar{B} \cdot A \cdot 0 + B \cdot \bar{A} \cdot 1 + B \cdot A \cdot 1 = \bar{B} \cdot \bar{A} + B \cdot \bar{A} + B \cdot A = \bar{B} \cdot \bar{A} + B \cdot \bar{A} + B \cdot A + B \cdot \bar{A} = B + \bar{A}$

- Trường hợp 15: $x_1 = x_2 = x_3 = 1$ & $x_0 = 0 \Rightarrow x = A+B$ (Đây chính là cổng OR – là đảo của trường hợp 2, cổng NOR)

B	A	$x = f(B, A)$
0	0	0
0	1	1
1	0	1
1	1	1

Bảng 51. Bảng thực trị với $x_1 = x_2 = x_3 = 1$ & $x_0 = 0$

- Trường hợp 15- dạng chuẩn tuyến: $x = \bar{B} \cdot \bar{A} \cdot 0 + \bar{B} \cdot A \cdot 1 + B \cdot \bar{A} \cdot 1 + B \cdot A \cdot 1 = \bar{B} \cdot A + B \cdot \bar{A} + B \cdot A = \bar{B} \cdot A + B \cdot A + B \cdot \bar{A} + B \cdot A = A+B$

- Trường hợp 16: $x_0 = x_1 = x_2 = x_3 = 1 \Rightarrow x = 1$ (Đây là hằng 1)

B	A	$x = f(B, A)$
0	0	1
0	1	1
1	0	1
1	1	1

Bảng 52. Bảng thực trị với $x_0 = x_1 = x_2 = x_3 = 1$

- Trường hợp 16- dạng chuẩn tuyến: $x = \bar{B} \cdot \bar{A} \cdot 1 + \bar{B} \cdot A \cdot 1 + B \cdot \bar{A} \cdot 1 + B \cdot A \cdot 1 = \bar{B} \cdot \bar{A} + \bar{B} \cdot A + B \cdot \bar{A} + B \cdot A = \bar{B} + B = 1$

3.10.4- Hàm 3 biến $x = f(C, B, A)$

- Lập bảng thực trị:

C	B	A	$X = f(C, B, A)$
0	0	0	x_0
0	0	1	x_1
0	1	0	x_2
0	1	1	x_3
1	0	0	x_4
1	0	1	x_5
1	1	0	x_6
1	1	1	x_7

Bảng 53. Bảng thực trị với $x_i \in \{0,1\}$

- Công thức đại số dạng chuẩn tuyến: $x = \bar{C} \cdot \bar{B} \cdot \bar{A} \cdot x_0 + \bar{C} \cdot \bar{B} \cdot A \cdot x_1 + \bar{C} \cdot B \cdot \bar{A} \cdot x_2 + \bar{C} \cdot B \cdot A \cdot x_3 + C \cdot \bar{B} \cdot \bar{A} \cdot x_4 + C \cdot \bar{B} \cdot A \cdot x_5 + C \cdot B \cdot \bar{A} \cdot x_6 + C \cdot B \cdot A \cdot x_7$
- Công thức đại số dạng chuẩn hội: $x = (C+B+A+x_0) \cdot (C+B+\bar{A}+x_1) \cdot (C+\bar{B}+A+x_2) \cdot (C+\bar{B}+\bar{A}+x_3) \cdot (\bar{C}+B+A+x_4) \cdot (\bar{C}+B+\bar{A}+x_5) \cdot (\bar{C}+\bar{B}+A+x_6) \cdot (\bar{C}+\bar{B}+\bar{A}+x_7)$
- Với hàm ba biến, tất cả trường hợp có thể xảy ra là $2^3 = 2^8 = 256$ trường hợp.

3.10.5- Hàm 4 biến $x = f(D, C, B, A)$

- Lập bảng thực trị:

D	C	B	A	$x=f(D, C, B, A)$
0	0	0	0	x_0
0	0	0	1	x_1
0	0	1	0	x_2
0	0	1	1	x_3
0	1	0	0	x_4
0	1	0	1	x_5
0	1	1	0	x_6
0	1	1	1	x_7
1	0	0	0	x_8
1	0	0	1	x_9
1	0	1	0	x_{10}
1	0	1	1	x_{11}
1	1	0	0	x_{12}
1	1	0	1	x_{13}
1	1	1	0	x_{14}
1	1	1	1	x_{15}

Bảng 54. Bảng thực trị với $x_i \in \{0,1\}$

- Công thức đại số dạng chuẩn tuyến: $x = \bar{D} \cdot \bar{C} \cdot \bar{B} \cdot \bar{A} \cdot x_0 + \bar{D} \cdot \bar{C} \cdot \bar{B} \cdot A \cdot x_1 + \bar{D} \cdot \bar{C} \cdot B \cdot \bar{A} \cdot x_2 + \bar{D} \cdot \bar{C} \cdot B \cdot A \cdot x_3 + \bar{D} \cdot C \cdot \bar{B} \cdot \bar{A} \cdot x_4 + \bar{D} \cdot C \cdot \bar{B} \cdot A \cdot x_5 + \bar{D} \cdot C \cdot B \cdot \bar{A} \cdot x_6 + \bar{D} \cdot C \cdot B \cdot A \cdot x_7 + D \cdot \bar{C} \cdot \bar{B} \cdot \bar{A} \cdot x_8 + D \cdot \bar{C} \cdot \bar{B} \cdot A \cdot x_9 + D \cdot \bar{C} \cdot B \cdot \bar{A} \cdot x_{10} + D \cdot \bar{C} \cdot B \cdot A \cdot x_{11} + D \cdot C \cdot \bar{B} \cdot \bar{A} \cdot x_{12} + D \cdot C \cdot \bar{B} \cdot A \cdot x_{13} + D \cdot C \cdot B \cdot \bar{A} \cdot x_{14} + D \cdot C \cdot B \cdot A \cdot x_{15}$

- Công thức đại số dạng chuẩn hội: $x = (D+C+B+A+x_0). (D+C+B+\bar{A}+x_1). (D+C+\bar{B}+A+x_2). (D+C+\bar{B}+\bar{A}+x_3). (D+\bar{C}+B+A+x_4) . (D+\bar{C}+B+\bar{A}+x_5). (D+\bar{C}+\bar{B}+A+x_6). (D+\bar{C}+\bar{B}+\bar{A}+x_7). (\bar{D}+C+B+A+x_8). (\bar{D}+C+B+\bar{A}+x_9). (\bar{D}+C+\bar{B}+A+x_{10}). (\bar{D}+C+\bar{B}+\bar{A}+x_{11}). (\bar{D}+\bar{C}+B+A+x_{12}) . (\bar{D}+\bar{C}+B+\bar{A}+x_{13}). (\bar{D}+\bar{C}+\bar{B}+A+x_{14}). (\bar{D}+\bar{C}+\bar{B}+\bar{A}+x_{15})$
- Với hàm bốn biến, tất cả trường hợp có thể xảy ra là $2^{2^4} = 2^{16} = 65536$ trường hợp.

3.10.6- Hàm N biến $x = f(A_{n-1}, A_{n-2}, \dots, A_1, A_0)$

- Lập bảng thực trị:

A_{n-1}	A_{n-2}	...	A_0	$x=f(A_{n-1}, A_{n-2}, \dots, A_0)$
0	0	...	0	X_0
0	0	...	1	x_1
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
1	1	...	0	x_{n-2}
1	1	...	1	x_{n-1}

Bảng 55. Bảng thực trị với $x_i \in \{0,1\}$

- Công thức đại số dạng chuẩn tuyến: $x = \bar{A}_{n-1}. \bar{A}_{n-2} \dots \bar{A}_1. \bar{A}_0. x_0 + \bar{A}_{n-1}. \bar{A}_{n-2} \dots \bar{A}_1. \bar{A}_0. x_1 + \bar{A}_{n-1}. \bar{A}_{n-2} \dots A_1. \bar{A}_0. x_2 + \dots + A_{n-1}. A_{n-2} \dots A_1. A_0. x_{n-1}$.
- Công thức đại số dạng chuẩn hội: $x = (A_{n-1}+A_{n-2}+\dots+A_1+A_0+x_0) . (A_{n-1}+A_{n-2}+\dots+A_1+\bar{A}_0+x_1). (A_{n-1}+A_{n-2}+\dots+\bar{A}_1+A_0+x_2) \dots (\bar{A}_{n-1}+\bar{A}_{n-2}+\dots+\bar{A}_1+\bar{A}_0+x_{n-1})$
- Với hàm N biến, tất cả trường hợp có thể xảy ra là 2^{2^N} trường hợp.
- Nhận xét 1: từ dạng chuẩn tuyến, ta thu được dạng chuẩn hội bằng cách lấy phủ định hai lần của dạng chuẩn tuyến kết hợp với định lý DeMorgan.
- Nhận xét 2: Trong dạng chuẩn tuyến, do phép toán AND với 0 thì sẽ bằng 0, và AND với 1 thì bằng chính nó, nên ta chỉ viết những biểu thức logic tương ứng với những giá trị $x_i=1$, còn

những biểu thức logic ứng với $x_i=0$ thì không viết vào công thức.

- Nhận xét 3: Trong dạng chuẩn hội, do phép toán OR với 0 thì sẽ bằng chính nó, và OR với 1 thì bằng 1, nên ta chỉ viết những biểu thức logic tương ứng với những giá trị $x_i=0$, còn những biểu thức logic ứng với $x_i=1$ thì không viết vào công thức.

- Ví dụ minh họa:

D	C	B	A	x	Chuẩn tuyến	Chuẩn hội
0	0	0	0	1	$\overline{D}.\overline{C}.\overline{B}.\overline{A}$	
0	0	0	1	1	$\overline{D}.\overline{C}.\overline{B}.A$	
0	0	1	0	1	$\overline{D}.\overline{C}.B.\overline{A}$	
0	0	1	1	0		$D+C+\overline{B}+\overline{A}$
0	1	0	0	1	$\overline{D}.C.\overline{B}.\overline{A}$	
0	1	0	1	1	$\overline{D}.C.\overline{B}.A$	
0	1	1	0	1	$\overline{D}.C.B.\overline{A}$	
0	1	1	1	0		$D+\overline{C}+\overline{B}+\overline{A}$
1	0	0	0	1	$D.\overline{C}.\overline{B}.\overline{A}$	
1	0	0	1	1	$D.\overline{C}.\overline{B}.A$	
1	0	1	0	1	$D.\overline{C}.B.\overline{A}$	
1	0	1	1	0		$\overline{D}+C+\overline{B}+\overline{A}$
1	1	0	0	0		$\overline{D}+\overline{C}+B+A$
1	1	0	1	0		$\overline{D}+\overline{C}+B+\overline{A}$
1	1	1	0	0		$\overline{D}+\overline{C}+\overline{B}+A$
1	1	1	1	0		$\overline{D}+\overline{C}+\overline{B}+\overline{A}$

Bảng 56. Cách viết công thức đại số dạng chuẩn tuyến và chuẩn hội

- Ta viết x dưới dạng chuẩn tuyến như sau:

$$x = \overline{D}.\overline{C}.\overline{B}.\overline{A} + \overline{D}.\overline{C}.\overline{B}.A + \overline{D}.\overline{C}.B.\overline{A} + \overline{D}.\overline{C}.B.A + \overline{D}.C.\overline{B}.\overline{A} + \overline{D}.C.\overline{B}.A + \overline{D}.C.B.\overline{A} + \overline{D}.C.B.A + D.\overline{C}.\overline{B}.\overline{A} + D.\overline{C}.\overline{B}.A + D.\overline{C}.B.\overline{A} + D.\overline{C}.B.A + D.C.\overline{B}.\overline{A} + D.C.\overline{B}.A + D.C.B.\overline{A} + D.C.B.A$$

- Ta viết x dưới dạng chuẩn hội như sau:
 $x = (D+C+\bar{B}+\bar{A}). (D+\bar{C}+\bar{B}+\bar{A}).$
 $(\bar{D}+C+\bar{B}+\bar{A}). (\bar{D}+\bar{C}+B+A).$
 $(\bar{D}+\bar{C}+\bar{B}+A). (\bar{D}+\bar{C}+\bar{B}+\bar{A})$
- Trong những trường hợp biểu thức đại số không ở dạng chuẩn, thì ta phải dùng các định lý Bool và định lý DeMorgan để đưa về dạng chuẩn. Những trường hợp không dùng dạng chuẩn như chỉ dùng một loại cổng NAND hay một loại cổng NOR, hay dùng hai loại cổng EX-OR và EX-NOR chỉ sử dụng với những yêu cầu cụ thể trong những trường hợp đặc biệt.
- Những công thức viết dưới hai dạng chuẩn trên là những công thức viết đầy đủ, và người thiết kế mạch số phải có nhiệm vụ rút gọn mạch để giảm số cổng hay số vi mạch sử dụng cho mạch đó ở mức tối thiểu.

3.11- Rút Gọn Một Biểu Thức Bằng Phương Pháp Đại Số

- Thường phương pháp này được áp dụng cho dạng chuẩn tuyển.
- 3.11.1.1- Rút gọn từ dạng đầy đủ, hay từ bảng thực trị viết ra
- Lấy một ví dụ từ bảng thực trị trên, ta có:
 $x = \bar{D}.\bar{C}.\bar{B}.\bar{A} + \bar{D}.\bar{C}.\bar{B}.A + \bar{D}.\bar{C}.B.\bar{A} + \bar{D}.C.\bar{B}.\bar{A} + \bar{D}.C.\bar{B}.A +$
 $\bar{D}.C.B.\bar{A} + D.\bar{C}.\bar{B}.\bar{A} + D.\bar{C}.\bar{B}.A + D.\bar{C}.B.\bar{A}$
 Nhận xét: lấy $\bar{D}.\bar{C}.\bar{B}.\bar{A}$ ra xét, xem có bao nhiêu biểu thức khác nó một bit, ta sẽ thấy đó là các biểu thức $\bar{D}.\bar{C}.\bar{B}.A$, $\bar{D}.C.\bar{B}.\bar{A}$, $\bar{D}.C.\bar{B}.A$
 ta có thể rút gọn 4 biểu thức trên như sau
 $* \bar{D}.\bar{C}.\bar{B}.\bar{A} + \bar{D}.\bar{C}.\bar{B}.A + \bar{D}.C.\bar{B}.\bar{A} + \bar{D}.C.\bar{B}.A = \bar{D}.\bar{C}.\bar{B}.(\bar{A} + A) + \bar{D}.C.\bar{B}.(\bar{A} + A) = \bar{D}.\bar{C}.\bar{B} + \bar{D}.C.\bar{B} = \bar{D}.\bar{B}.(\bar{C} + C) = \bar{D}.\bar{B}$
 hay viết một cách ngắn gọn hơn là
 $* \bar{D}.\bar{C}.\bar{B}.\bar{A} + \bar{D}.\bar{C}.\bar{B}.A + \bar{D}.C.\bar{B}.\bar{A} + \bar{D}.C.\bar{B}.A = \bar{D}.\bar{B}.(\bar{C}.\bar{A} + \bar{C}.A + C.\bar{A} + C.A) = \bar{D}.\bar{B}$
 tương tự ta sẽ có các cặp
 $* \bar{D}.\bar{C}.\bar{B}.\bar{A} + \bar{D}.\bar{C}.B.\bar{A} + \bar{D}.C.\bar{B}.\bar{A} + \bar{D}.C.B.\bar{A} = \bar{D}.\bar{A}.(\bar{C}.\bar{B} + \bar{C}.B + C.\bar{B} + C.B) = \bar{D}.\bar{A}$

$$* \bar{D}.\bar{C}.\bar{B}.\bar{A} + \bar{D}.\bar{C}.\bar{B}.A + D.\bar{C}.\bar{B}.\bar{A} + D.\bar{C}.\bar{B}.A = \bar{C}.\bar{B}.(\bar{D}.\bar{A} + \bar{D}.A + D.\bar{A} + D.A) = \bar{C}.\bar{B}$$

$$* \bar{D}.\bar{C}.\bar{B}.\bar{A} + \bar{D}.\bar{C}.B.\bar{A} + D.\bar{C}.\bar{B}.\bar{A} + D.\bar{C}.B.\bar{A} = \bar{C}.\bar{A}.(\bar{D}.\bar{B} + \bar{C}.B + C.\bar{B} + C.B) = \bar{C}.\bar{A}$$

như vậy, ta đã dùng $\bar{D}.\bar{C}.\bar{B}.\bar{A}$ 4 lượt, $\bar{D}.\bar{C}.\bar{B}.A$ hai lượt, $\bar{D}.\bar{C}.B.\bar{A}$ hai lượt, $\bar{D}.C.\bar{B}.\bar{A}$ hai lượt, $D.\bar{C}.\bar{B}.\bar{A}$ ba lượt $\Rightarrow$ rút gọn x như sau

$$\begin{aligned} x &= \bar{D}.\bar{C}.\bar{B}.\bar{A} + \bar{D}.\bar{C}.\bar{B}.A + \bar{D}.\bar{C}.B.\bar{A} + \bar{D}.C.\bar{B}.\bar{A} + \bar{D}.C.\bar{B}.A + \bar{D}.C.B.\bar{A} + D.\bar{C}.\bar{B}.\bar{A} + D.\bar{C}.\bar{B}.A + D.\bar{C}.B.\bar{A} + D.C.\bar{B}.\bar{A} \\ &= (\bar{D}.\bar{C}.\bar{B}.\bar{A} + \bar{D}.\bar{C}.\bar{B}.\bar{A} + \bar{D}.\bar{C}.\bar{B}.\bar{A} + \bar{D}.\bar{C}.\bar{B}.\bar{A}) + (\bar{D}.\bar{C}.\bar{B}.A + \bar{D}.\bar{C}.\bar{B}.A) + (\bar{D}.\bar{C}.B.\bar{A} + \bar{D}.\bar{C}.B.\bar{A}) + (\bar{D}.C.\bar{B}.\bar{A} + \bar{D}.C.\bar{B}.\bar{A}) + \bar{D}.C.\bar{B}.A + \bar{D}.C.B.\bar{A} \\ &+ (D.\bar{C}.\bar{B}.\bar{A} + D.\bar{C}.\bar{B}.\bar{A} + D.\bar{C}.\bar{B}.\bar{A}) + D.\bar{C}.\bar{B}.A + D.C.\bar{B}.\bar{A} = \bar{D}.\bar{B} + \bar{D}.\bar{A} + \bar{C}.\bar{B} + \bar{C}.\bar{A} \\ x &= \bar{D}.\bar{B} + \bar{D}.\bar{A} + \bar{C}.\bar{B} + \bar{C}.\bar{A} \end{aligned}$$

➤ Lấy một ví dụ khác

$$x = D.C.\bar{B}.\bar{A} + D.C.\bar{B}.A + \bar{D}.C.\bar{B}.\bar{A} + D.\bar{C}.\bar{B}.A + \bar{D}.\bar{C}.B.A + \bar{D}.\bar{C}.B.\bar{A}$$

Nhận xét: lấy $D.C.\bar{B}.\bar{A}$ ra xét, xem có bao nhiêu biểu thức khác nó một bit, ta thấy chỉ có $D.C.\bar{B}.A \Rightarrow$ ta có thể rút gọn biểu thức trên thành

$$* D.C.\bar{B}.\bar{A} + D.C.\bar{B}.A = D.C.\bar{B}(\bar{A} + A) = D.C.\bar{B}$$

tương tự ta có các cặp:

$$* \bar{D}.C.\bar{B}.A + D.C.\bar{B}.A + D.\bar{C}.\bar{B}.A = \bar{D}.C.\bar{B}.A + (D.C.\bar{B}.A + D.C.\bar{B}.A) + D.\bar{C}.\bar{B}.A = C.\bar{B}.A.(\bar{D} + D) + D.\bar{B}.A.(C + \bar{C}) = C.\bar{B}.A + D.\bar{B}.A$$

$$* \bar{D}.\bar{C}.B.A + \bar{D}.\bar{C}.B.\bar{A} = \bar{D}.\bar{C}.B.(A + \bar{A}) = \bar{D}.\bar{C}.B$$

Như vậy, ta đã dùng $D.C.\bar{B}.A$ ba lượt, $\Rightarrow$ ta rút gọn x như sau:

$$\begin{aligned} x &= D.C.\bar{B}.\bar{A} + D.C.\bar{B}.A + \bar{D}.C.\bar{B}.\bar{A} + D.\bar{C}.\bar{B}.A + \bar{D}.\bar{C}.B.A + \bar{D}.\bar{C}.B.\bar{A} \\ &= D.C.\bar{B}.\bar{A} + (D.C.\bar{B}.A + D.C.\bar{B}.A + D.C.\bar{B}.A) + \bar{D}.\bar{C}.\bar{B}.\bar{A} + \bar{D}.\bar{C}.B.A + \bar{D}.\bar{C}.B.\bar{A} \\ &= D.C.\bar{B} + C.\bar{B}.A + D.\bar{B}.A + \bar{D}.\bar{C}.B \end{aligned}$$

3.11.1.2- Rút gọn từ dạng không đầy đủ

➤ Lấy một ví dụ:

$$x = C.\bar{B} + \bar{D}.C.\bar{B} + \bar{D}.C.A + \bar{D}.C.B.A + C.\bar{B}.A + D.\bar{B}.A + D.\bar{C}.\bar{B}.A$$

Nhận xét: lấy biểu thức đã rút gọn nhất là $C.\bar{B}$ và tìm trong đó có biểu thức nào có chứa $C.\bar{B}$, ta có $\bar{D}.C.\bar{B}$ và $C.\bar{B}.A$, $\Rightarrow C.\bar{B} + \bar{D}.C.\bar{B} + C.\bar{B}.A = C.\bar{B}.(1 + \bar{D} + A) = C.\bar{B}$

lấy biểu thức rút gọn còn lại, mà chưa được liệt kê ở phần trên là $\bar{D}.\bar{C}.A$ và tiếp tục tìm xem có biểu thức nào chứa $\bar{D}.\bar{C}.A$, ta có $\bar{D}.C.B.A \Rightarrow \bar{D}.C.A + \bar{D}.C.B.A = \bar{D}.C.A.(1 + B) = \bar{D}.C.A$ tương tự ta có cặp $D.\bar{B}.A$ và $D.\bar{C}.\bar{B}.A \Rightarrow D.\bar{B}.A + D.\bar{C}.\bar{B}.A = D.\bar{B}.A.(1 + \bar{C}) = D.\bar{B}.A$

Vậy $x = C.\bar{B} + \bar{D}.C.\bar{B} + \bar{D}.C.A + \bar{D}.C.B.A + C.\bar{B}.A + D.\bar{B}.A + D.\bar{C}.\bar{B}.A = C.\bar{B}.(1 + \bar{D} + A) + \bar{D}.C.A.(1 + B) + D.\bar{B}.A.(1 + \bar{C}) = C.\bar{B} + \bar{D}.C.A + D.\bar{B}.A$

3.11.1.3- Rút gọn từ dạng chuẩn hội

- Ta nhân các biểu thức lại với nhau. xong áp dụng các phương pháp nêu trên
- Lấy một ví dụ: $x = (A + C).(B + C + \bar{D}).D = A.B.D + A.C.D + A.\bar{D}.D + B.C.D + C.C.D + C.\bar{D}.D = A.B.D + A.C.D + B.C.D + C.D$
Áp dụng phương pháp trên ta có $x = C.D.(1 + A + B) + A.B.D = C.D + A.B.D$

3.11.1.4- Rút gọn từ dạng chưa được chuẩn hoá

- Áp dụng định lý DeMorgan và các định lý đã học để đưa về dạng chuẩn rồi rút gọn
- Lấy một ví dụ: $x = \bar{A}.C.(\overline{\bar{A}.B.D}) + \bar{A}.B.\bar{C}.\bar{D} + A.\bar{B}.C = \bar{A}.C.(A + \bar{B} + \bar{D}) + \bar{A}.B.\bar{C}.\bar{D} + A.\bar{B}.C = \bar{A}.C.A + \bar{A}.C.\bar{B} + \bar{A}.C.\bar{D} + \bar{A}.B.\bar{C}.\bar{D} + A.\bar{B}.C = \bar{A}.C.\bar{B} + \bar{A}.C.\bar{D} + \bar{A}.B.\bar{C}.\bar{D} + A.\bar{B}.C = A.\bar{B}.C.(1 + \bar{D}) + \bar{B}.C.(\bar{A} + A) + \bar{A}.C.\bar{D} = \bar{B}.C + A.\bar{B}.C + \bar{A}.C.\bar{D}$

3.12- Rút Gọn Một Biểu Thức Bằng Phương Pháp Biểu Đồ Karnaugh – Dạng Chuẩn Tuyến

- Trên cơ sở của các định lý Bool, nhà toán Học Karnaugh đã đưa ra một phương pháp trực quan để rút gọn một biểu thức Bool, phương pháp có tên là biểu đồ Karnaugh.
- Phương pháp này chỉ áp dụng được cho các hàm 6 biến trở xuống, và dễ áp dụng nhất là 4 biến trở xuống. Trong tài liệu

này, ta sẽ xét từ trường hợp 1 biến đến 4 biến, trước khi đi vào phương pháp rút gọn tổng quát.

- Công thức đại số trong các phần dưới được viết dưới dạng chuẩn tuyến, còn dạng chuẩn hội sẽ đề cập ở phần sau.

3.12.1- Biểu đồ Karnaugh cho hàm 1 biến $x = f(A)$

Hàm 1 biến chỉ có 4 trường hợp và quá đơn giản, nên không dùng biểu đồ Karnaugh để đơn giản.

3.12.2- Biểu đồ Karnaugh cho hàm 2 biến $x = f(B, A)$

- Biểu đồ Karnaugh một hình vuông có 4 ô như sau:

$\overline{B}.\overline{A}.x_0$	$\overline{B}.A.x_1$
$B.\overline{A}.x_2$	$B.A.x_3$

Bảng 57. Biểu đồ Karnaugh cho hàm hai biến

- Viết như vậy thì rườm rà và khó rút gọn, nên đưa các giá trị $\overline{B}, B, \overline{A}, A$ ra ngoài như hình dưới:

	$\overline{A}$	A
$\overline{B}$	x_0	x_1
B	x_2	x_3

Bảng 58. Cách biểu diễn khác của biểu đồ Karnaugh

- Cũng có một cách biểu diễn khác, thay cho $\overline{A}, \overline{B}$ là bit 0 và thay cho A, B là bit 1.

		A	
		0	1
B	0	x_0	x_1
	1	x_2	x_3

Bảng 59. Cách biểu diễn khác của biểu đồ Karnaugh

- Hai dạng biểu diễn sau thường được sử dụng hơn.
- Khi áp dụng vào một trường hợp cụ thể, thì ta thay các giá trị $x_i = 0$ hay $x_i = 1$ vào các ô trong hình vuông.

Một số nguyên tắc khi xây dựng biểu đồ Karnaugh hai biến:

- Biểu đồ Karnaugh cho hàm hai biến là 1 hình vuông gồm 4 ô.
- Mỗi ô ứng với 1 dòng trong bảng thực trị (tích của 2 biến B, A và x_i).
- Mỗi ô nằm kế cận 2 ô.
- Hai ô cạnh nhau chỉ khác nhau duy nhất 1 bit.
- Ô nằm riêng lẻ một mình có giá trị $x_i = 1$, khoanh chúng lại thành một **vòng** gồm 1 ô, được gọi là LOOP1, không rút gọn được biến nào.
- Hai ô nằm cạnh nhau có giá trị là $x_i = 1$, khoanh chúng lại thành một **vòng** gồm 2 ô, được gọi là LOOP2, thì rút gọn được một biến.
- Bốn ô nằm cạnh nhau có giá trị là $x_i = 1$, khoanh chúng lại thành một **vòng** gồm 4 ô, được gọi là LOOP4, thì rút gọn được hai biến.

Xét một số trường hợp của hàm hai biến sau:

	$\bar{A}$	A
$\bar{B}$	1	1
B	1	1

Bảng 60. Một biểu đồ Karnaugh với LOOP4

- Hàm 2 biến có một LOOP4 bị rút gọn hết 2 biến $\Rightarrow x = 1$.

	$\bar{A}$	A
$\bar{B}$	0	1
B	1	1

Bảng 61. Một biểu đồ Karnaugh với hai LOOP2

- Như trường hợp này có hai LOOP2, LOOP2 thứ nhất gồm có $\bar{B} + A.B$, thì bỏ biến B chỉ còn lại biến A. LOOP2 thứ hai gồm có $\bar{A}.B + A.B$, thì bỏ biến A chỉ còn lại biến B, như vậy công thức rút gọn được là $x = A + B$. Hay dễ nhìn hơn là LOOP2 thứ nhất là cột A và LOOP2 thứ 2 là cột B $\Rightarrow x = A + B$.
- Từ biểu đồ Karnaugh, ta có thể áp dụng vào việc rút gọn bằng phương pháp đại số. Lấy ví dụ trên, biểu thức chưa rút gọn là $x = B.A + B.\bar{A} + \bar{B}.A$, căn cứ vào biểu đồ Karnaugh, thì ô B.A được dùng hai lần, ta áp dụng như sau: $x = B.A + B.\bar{A} + \bar{B}.A = B.A + B.A + B.\bar{A} + \bar{B}.A = B.(A + \bar{A}) + A.(B + \bar{B}) = B + A$.

	$\bar{A}$	A
$\bar{B}$	0	1
B	1	0

Bảng 62. Một biểu đồ Karnaugh với hai LOOP1

- Như trường hợp này có hai LOOP1, LOOP1 thứ nhất là $A.\bar{B}$ và LOOP1 thứ 2 là $\bar{A}.B$, trường hợp này không thể rút gọn $\Rightarrow x = A.\bar{B} + \bar{A}.B$

3.12.3- Biểu đồ Karnaugh cho hàm 3 biến $x = f(C, B, A)$

- Biểu đồ Karnaugh một hình chữ nhật có 8 ô như sau:

$\bar{C}.\bar{B}.\bar{A}.x_0$	$\bar{C}.\bar{B}.A.x_1$	$\bar{C}.B.A.x_3$	$\bar{C}.B.\bar{A}.x_2$
$C.\bar{B}.\bar{A}.x_4$	$C.\bar{B}.A.x_5$	$C.B.A.x_7$	$C.B.\bar{A}.x_6$

Hình 77. Biểu đồ Karnaugh cho hàm ba biến

- Viết như vậy thì rườm rà và khó rút gọn, nên đưa các giá trị $\bar{B}, B, \bar{A}, A$ ra ngoài như hình dưới:

	$\bar{B}.\bar{A}$	$\bar{B}.A$	$B.A$	$B.\bar{A}$
$\bar{C}$	x_0	x_1	x_3	x_2
C	x_4	x_5	x_7	x_6

Bảng 63. Cách biểu diễn khác của biểu đồ Karnaugh

- Cũng có một cách biểu diễn khác, thay cho $\bar{A}, \bar{B}$ là bit 0 và thay cho A, B là bit 1.

	B.A	00	01	11	10
C					
0		x_0	x_1	x_3	x_2
1		x_4	x_5	x_7	x_6

Bảng 64. Cách biểu diễn khác của biểu đồ Karnaugh

- Hai dạng biểu diễn sau thường được sử dụng hơn.
- Khi áp dụng vào một trường hợp cụ thể, thì ta thay các giá trị $x_i = 0$ hay $x_i = 1$ vào các ô trong hình chữ nhật.

Một số nguyên tắc khi xây dựng biểu đồ Karnaugh ba biến:

- Biểu đồ Karnaugh cho hàm hai biến là 1 hình chữ nhật gồm 8 ô.
- Mỗi ô ứng với 1 dòng trong bảng thực trị (tích của 3 biến C, B, A và x_i).
- Mỗi ô nằm kế cận 3 ô.
- Hai ô cạnh nhau chỉ khác nhau duy nhất 1 bit.
- Ô nằm riêng lẻ một mình có giá trị $x_i = 1$, khoanh chúng lại thành một **vòng** gồm 1 ô, được gọi là LOOP1, không rút gọn được biến nào.
- Hai ô nằm cạnh nhau có giá trị là $x_i = 1$, khoanh chúng lại thành một **vòng** gồm 2 ô, được gọi là LOOP2, thì rút gọn được một biến.
- Bốn ô nằm cạnh nhau có giá trị là $x_i = 1$, khoanh chúng lại thành một **vòng** gồm 4 ô, được gọi là LOOP4, thì rút gọn được hai biến.
- Tám ô nằm cạnh nhau có giá trị là $x_i = 1$, khoanh chúng lại thành một **vòng** gồm 8 ô, được gọi là LOOP8, thì rút gọn được ba biến.

Xét một trường hợp của hàm ba biến sau:

- Cho một hàm ba biến với bảng thực trị sau:

C	B	A	$x = f(C, B, A)$
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Bảng 65. Bảng thực trị của 1 hàm ba biến

- Từ bảng thực trị, ta viết công thức đại số: $x = \bar{C} \cdot \bar{B} \cdot \bar{A} \cdot 0 + \bar{C} \cdot \bar{B} \cdot A \cdot 1 + \bar{C} \cdot B \cdot \bar{A} \cdot 1 + \bar{C} \cdot B \cdot A \cdot 1 + C \cdot \bar{B} \cdot \bar{A} \cdot 0 + C \cdot \bar{B} \cdot A \cdot 1 + C \cdot B \cdot \bar{A} \cdot 0 + C \cdot B \cdot A \cdot 1 \Rightarrow x = \bar{C} \cdot \bar{B} \cdot A + \bar{C} \cdot B \cdot \bar{A} + \bar{C} \cdot B \cdot A + C \cdot \bar{B} \cdot A + C \cdot B \cdot A$

- Ta có biểu đồ Karnaugh như hình dưới

	$\bar{B} \cdot \bar{A}$	$\bar{B} \cdot A$	$B \cdot A$	$B \cdot \bar{A}$
$\bar{C}$	0	1	1	1
C	0	1	1	0

Bảng 66. Một biểu đồ Karnaugh gồm 1 LOOP4 và 1 LOOP2

- Hay biểu diễn dưới hình thức khác

BA \ C	00	01	11	10
0	0	1	1	1
1	0	1	1	0

Bảng 67. Cách biểu diễn khác của biểu đồ Karnaugh

- Với biểu đồ Karnaugh gồm 1 LOOP4 và 1 LOOP2 $\Rightarrow x = A + \bar{C} \cdot B$.

3.12.4- Biểu đồ Karnaugh cho hàm 4 biến $x = f(D, C, B, A)$

- Để tổng quát cách xây dựng biểu đồ Karnaugh, ta đưa ra một số nguyên tắc sau:

- Biểu đồ Karnaugh cho hàm 4 biến là 1 hình vuông có 16 ô.
- Mỗi ô ứng với 1 dòng trong bảng thực trị (tích của 4 biến D, C, B, A và x_i).
- Mỗi ô nằm kế cận 4 ô, như ô x_5 kế 4 ô x_1, x_4, x_7, x_{13} , ô x_6 kế với 4 ô x_2, x_4, x_7, x_{14} , ô x_0 kế 4 ô $x_1, x_4, x_8, x_{10}, \dots$

- Hai ô cạnh nhau chỉ khác nhau duy nhất 1 bit.
- LOOP là tập hợp các ô có giá trị $x_i = 1$ nằm cạnh nhau theo nguyên tắc 2^n , với $n = 0, 1, 2, 3, 4 \Rightarrow$ LOOP1, LOOP2, LOOP4, LOOP8, LOOP16.
- Ô nằm riêng lẻ một mình có giá trị $x_i = 1$, khoanh chúng lại thành một **vòng** gồm 1 ô, được gọi là LOOP1, không rút gọn được biến nào.
- Hai ô nằm cạnh nhau có giá trị là $x_i = 1$, khoanh chúng lại thành một **vòng** gồm 2 ô, được gọi là LOOP2, thì rút gọn được một biến.
- Bốn ô nằm cạnh nhau có giá trị là $x_i = 1$, khoanh chúng lại thành một **vòng** gồm 4 ô, được gọi là LOOP4, thì rút gọn được hai biến. Trong biểu đồ Karnaugh 4 biến, LOOP4 là 4 ô tạo thành hình vuông, hay tạo thành một cột, một hàng hay 4 ô ở 4 góc.
- Tám ô nằm cạnh nhau có giá trị là $x_i = 1$, khoanh chúng lại thành một **vòng** gồm 8 ô, được gọi là LOOP8, thì rút gọn được ba biến. Trong biểu đồ Karnaugh 4 biến, LOOP8 là 8 ô tạo thành hai hàng hay hai cột.
- Mười sáu ô nằm cạnh nhau có giá trị là $x_i = 1$, khoanh chúng lại thành một **vòng** gồm 16 ô, được gọi là LOOP16, thì rút gọn được bốn biến.

Từ bảng thực trị, hay từ công thức đại số, ta xây dựng biểu đồ Karnaugh theo các nguyên tắc trên, tùy thuộc vào thứ tự D, C, B, A chúng ta sẽ có các biểu đồ Karnaugh khác nhau về mặt hình thức, nhưng kết quả vẫn như nhau. Như một số biểu đồ Karnaugh dưới đây.

	BA	00	10	11	01
DC					
00		x_0	x_2	x_3	x_1
10		x_8	x_{10}	x_{11}	x_9
11		x_{12}	x_{14}	x_{15}	x_{13}
01		x_4	x_6	x_7	x_5

hay

BA \ DC	11	10	00	01
01	x ₇	x ₆	x ₄	x ₅
11	x ₁₅	x ₁₄	x ₁₂	x ₁₃
10	x ₁₁	x ₁₀	x ₈	x ₉
00	x ₃	x ₂	x ₀	x ₁

hay ...

Bảng 68. Một số cách biểu diễn khác nhau của khác nhau của biểu đồ Karnaugh

- Trong tài liệu này, để thống nhất về mặt ký hiệu, biểu đồ Karnaugh được xây dựng một trong hai dạng với thứ tự sau:

BA \ DC	00	01	11	10
00	x ₀	x ₁	x ₃	x ₂
01	x ₄	x ₅	x ₇	x ₆
11	x ₁₂	x ₁₃	x ₁₅	x ₁₄
10	x ₈	x ₉	x ₁₁	x ₁₀

Hay

	$\overline{B}.\overline{A}$	$\overline{B}.A$	$B.A$	$B.\overline{A}$
$\overline{D}.\overline{C}$	x ₀	x ₁	x ₃	x ₂
$\overline{D}.C$	x ₄	x ₅	x ₇	x ₆
$D.C$	x ₁₂	x ₁₃	x ₁₅	x ₁₄
$D.\overline{C}$	x ₈	x ₉	x ₁₁	x ₁₀

Bảng 69. Cách biểu diễn biểu đồ Karnaugh dùng trong tài liệu

- Và để cho gọn, những ô có giá trị $x_i = 0$ sẽ để trống, chỉ đưa vào những ô có giá trị $x_i = 1$.
- Hàng số 1 là $\overline{D}.\overline{C}$, hàng số 2 là $\overline{D}.C$, hàng số 3 là $D.C$, hàng số 4 là $D.\overline{C}$.

- Cột số 1 là $\overline{B}.\overline{A}$, cột số 2 là $\overline{B}.A$, cột số 3 là $B.A$, cột số 4 là $B.\overline{A}$.
- Giải thuật: (bài toán tô màu, hay bài toán tìm phủ tối thiểu) - bao gồm 4 bước)
 - Xét bài toán tô màu được phát biểu như sau: cho 1 hình gồm nhiều ô, những ô cùng nhóm được tô 1 màu, một ô có thể thuộc nhiều nhóm, tô làm sao kín hình ban đầu với số màu ít nhất và lượng màu dùng cho mỗi màu là ít nhất.
 - Giải thuật rút gọn trong biểu đồ Karnaugh thực chất là bài toán tô màu (hay là bài toán tìm phủ tối thiểu), với mỗi LOOP là 1 màu, số ô trong mỗi LOOP là khối lượng của mỗi màu. Rút gọn làm sao số LOOP ít nhất, và mỗi LOOP thì có ít biến nhất.
- Bước 1: liệt kê tất cả các LOOP8, LOOP4, LOOP2, LOOP1, lưu ý là không được có trường hợp $LOOP_i \subseteq LOOP_j$, và viết công thức rút gọn cho mỗi LOOP.
- Bước 2: tìm ô nào chỉ thuộc duy nhất 1 LOOP, ta chọn LOOP đó, lặp lại bước này đến khi nào khi nào không còn ô nào có tính chất trên. Nếu thực hiện xong bước này mà tô kín biểu đồ Karnaugh ban đầu, thì ta có duy nhất 1 kết quả rút gọn, và kết quả này là tối giản.
- Bước 3:
 - Nếu sau bước 2 mà biểu đồ Karnaugh vẫn chưa được tô kín, chọn tùy ý 1 LOOP, lặp lại bước này cho đến khi tô kín biểu đồ Karnaugh ban đầu. Ta có một kết quả rút gọn, nhưng do những ô ở bước này thuộc ít nhất là 2 LOOP trở lên, nên việc lựa chọn tùy ý 1 LOOP chứa những ô thuộc loại này để tô, dẫn đến kết quả là chưa bảo đảm tối giản.
 - Do đó, ta phải xét đến phản đề, là để tô ô được chọn tùy ý mà không dùng LOOP ở bước (3.i), thì phải dùng đến LOOP kia và ta sẽ có một kết quả khác. Và việc xét phản đề sẽ đi theo thứ tự ngược với bước (3.i).
- Bước 4:
 - Sau bước 3, ta sẽ có nhiều kết quả rút gọn khác nhau, ở bước này, ta xét các kết quả theo tiêu chuẩn phát biểu ban đầu là số LOOP ít nhất, nếu số LOOP bằng nhau, thì xét kết quả nào chứa LOOP có ít biến hơn thì tối giản hơn. Và có khả năng là

chỉ có một công thức tối giản duy nhất hay nhiều công thức tương đương.

- Xét một ví dụ cụ thể cho giải thuật nêu trên:

- Cho $x = f(D, C, B, A) = \sum m(0, 1, 2, 3, 4, 5, 6, 8, 10, 11, 13, 15)$ với D là MSB, A là LSB

BA \ DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_{13}	x_{15}	
10	x_8		x_{11}	x_{10}

Hình 78. Biểu đồ Karnaugh cho hàm 4 biến trên

- Bước 1: Liệt kê tất cả các LOOP của biểu đồ
- LOOP16: không có.
- LOOP8: không có.
- LOOP4: có 5 LOOP đặt tên là L4-1, L4-2, L4-3, L4-4, L4-5
- LOOP2: có 3 LOOP đặt tên là L2-1, L2-2, L2-3
- LOOP1: không có.
- Ta liệt kê cụ thể từng LOOP như sau
L4-1 là tập hợp 4 ô $x_0, x_1, x_3, x_2 \Rightarrow L4-1 = \{x_0, x_1, x_3, x_2\} = \overline{D} \cdot \overline{C}$

BA DC	00	01	11	10
00	x ₀	x ₁	x ₃	x ₂
01	x ₄	x ₅		x ₆
11		x ₁₃	x ₁₅	
10	x ₈		x ₁₁	x ₁₀

Bảng 70. L4-1 = $\overline{D} \cdot \overline{C}$ chứa 4 ô trên một hàng

L4-2 là tập hợp 4 ô x₀, x₁, x₄, x₅ $\Rightarrow$ L4-2 = {x₀, x₁, x₄, x₅} = $\overline{D} \cdot \overline{B}$

BA DC	00	01	11	10
00	x ₀	x ₁	x ₃	x ₂
01	x ₄	x ₅		x ₆
11		x ₁₃	x ₁₅	
10	x ₈		x ₁₁	x ₁₀

Bảng 71. L4-2 = $\overline{D} \cdot \overline{B}$ chứa 4 ô tạo thành hình vuông

L4-3 là tập hợp 4 ô x₀, x₂, x₄, x₆ $\Rightarrow$ L4-3 = {x₀, x₂, x₄, x₆} = $\overline{D} \cdot \overline{A}$

BA DC	00	01	11	10
00	x ₀	x ₁	x ₃	x ₂
01	x ₄	x ₅		x ₆
11		x ₁₃	x ₁₅	
10	x ₈		x ₁₁	x ₁₀

Bảng 72. L4-3 = $\overline{D} \cdot \overline{A}$ chứa 4 ô ở hai cạnh

L4-4 là tập hợp 4 ô x₃, x₂, x₁₁, x₁₀ $\Rightarrow$ L4-4 = {x₃, x₂, x₁₁, x₁₀} = $\overline{C} \cdot B$

BA \ DC	00	01	11	10
00	x ₀	x ₁	x ₃	x ₂
01	x ₄	x ₅		x ₆
11		x ₁₃	x ₁₅	
10	x ₈		x ₁₁	x ₁₀

Bảng 73. L4-4 = $\overline{C}.B$ chứa 4 ô ở hai cạnh

L4-5 là tập hợp 4 ô x₀, x₂, x₈, x₁₀ $\Rightarrow$ L4-5 = {x₀, x₂, x₈, x₁₀} = $\overline{C}.\overline{A}$

BA \ DC	00	01	11	10
00	x ₀	x ₁	x ₃	x ₂
01	x ₄	x ₅		x ₆
11		x ₁₃	x ₁₅	
10	x ₈		x ₁₁	x ₁₀

Bảng 74. L4-5 = $\overline{C}.\overline{A}$ chứa 4 ô ở 4 góc

L2-1 là tập hợp 2 ô x₅, x₁₃ $\Rightarrow$ L2-1 = {x₅, x₁₃} = $\overline{C}.\overline{B}.A$

BA \ DC	00	01	11	10
00	x ₀	x ₁	x ₃	x ₂
01	x ₄	x ₅		x ₆
11		x ₁₃	x ₁₅	
10	x ₈		x ₁₁	x ₁₀

Bảng 75. L2-1 = $\overline{C}.\overline{B}.A$ chứa 2 ô kề nhau

L2-2 là tập hợp 2 ô x₁₃, x₁₅ $\Rightarrow$ L2-2 = {x₁₃, x₁₅} = D.C.A

BA DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_{13}	x_{15}	
10	x_8		x_{11}	x_{10}

Bảng 76. L2-2 = D.C.A chứa 2 ô kề nhau

L2-3 là tập hợp 2 ô $x_{15}, x_{11} \Rightarrow L2-3 = \{x_{15}, x_{11}\} = D.B.A$

BA DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_{13}	x_{15}	
10	x_8		x_{11}	x_{10}

Bảng 77. L2-3 = D.B.A chứa 2 ô kề nhau

- Bước 2: ô x_6 thuộc duy nhất $L4-3 = \{x_0, x_2, x_4, x_6\} = \overline{D}.\overline{A}$ và ô x_8 thuộc duy nhất $L4-5 = \{x_0, x_2, x_8, x_{10}\} = \overline{C}.\overline{A} \Rightarrow x = \overline{D}.\overline{A} + \overline{C}.\overline{A} + \dots$ ở bước 2 ta đã tô được 6 ô, còn lại 6 ô $x_1, x_3, x_5, x_{13}, x_{15}, x_{11}$ chưa tô

BA DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_{13}	x_{15}	
10	x_8		x_{11}	x_{10}

Bảng 78. Kết quả tô biểu đồ Karnaugh sau bước 2

- Bước 3:

Thiết Kế Hệ Thống Số

Bước 3.1:

Để tô những ô còn lại, ta chọn tùy ý một LOOP, trong trường hợp này ta chọn $L4-1 = \{x_0, x_1, x_2, x_3\} = \overline{D}.\overline{C} \Rightarrow$ tô thêm 2 ô $x_1, x_3 \Rightarrow x = \overline{D}.\overline{A} + \overline{C}.\overline{A} + \overline{D}.\overline{C} + \dots$ còn 4 ô $x_5, x_{13}, x_{11}, x_{15}$ chưa tô.

BA \ DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_{13}	x_{15}	
10	x_8		x_{11}	x_9

Bảng 79. Kết quả tô biểu đồ Karnaugh sau bước 3.1

Bước 3.1.1:

Để tô những ô còn lại, ta chọn tùy ý một LOOP, trong trường hợp này ta chọn $L2-3 = \{x_{11}, x_{15}\} = C.B.A \Rightarrow$ tô thêm 2 ô $x_{11}, x_{15} \Rightarrow x = \overline{D}.\overline{A} + \overline{C}.\overline{A} + \overline{D}.\overline{C} + C.B.A + \dots$ còn lại 2 ô x_5, x_{13} chưa tô

BA \ DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_{13}	x_{15}	
10	x_8		x_{11}	x_9

Bảng 80. Kết quả tô biểu đồ Karnaugh sau bước 3.1.1

Bước 3.1.1.1:

Để tô hai ô x_5 và x_{13} còn lại, ta chọn tùy ý $L2-1 = \{x_5, x_{13}\} = D.\overline{B}.A \Rightarrow x = \overline{D}.\overline{A} + \overline{C}.\overline{A} + \overline{D}.\overline{C} + C.B.A + D.\overline{B}.A$ (**Công thức 1**) và biểu đồ Karnaugh đã được tô xong hay ta đã có một công thức rút gọn.

BA \ DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_7	x_{15}	
10	x_8		x_{11}	x_{10}

Bảng 81. Kết quả tô biểu đồ Karnaugh sau bước 3.1.1.1

Bước 3.1.1.2: (phản đề của 3.1.1.1)

Trong bước 3.1.1.1 việc lựa chọn L2-1 để tô hai ô x_5 và x_{13} là tùy ý, nên để tô 2 ô x_5, x_{13} còn lại mà không dùng L2-1, bắt buộc phải dùng $L4-2 = \{x_0, x_1, x_4, x_5\} = \overline{D}.\overline{B}$ và $L2-2 = \{x_{13}, x_{15}\} = D.C.A \Rightarrow x = \overline{D}.\overline{A} + \overline{C}.\overline{A} + \overline{D}.\overline{C} + C.B.A + \overline{D}.\overline{B} + D.C.A$ (**Công thức 2**)

BA \ DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_{13}	x_{15}	
10	x_8		x_{11}	x_{10}

Bảng 82. Kết quả tô biểu đồ Karnaugh sau bước 3.1.1.2

Bước 3.1.2: (phản đề của 3.1.1)

Để tô thêm hai ô x_{11}, x_{15} mà không dùng L2-3 thì bắt buộc phải dùng $L2-2 = \{x_{13}, x_{15}\} = D.C.A$ và $L4-4 = \{x_2, x_3, x_{10}, x_{11}\} = \overline{C}.B \Rightarrow x = \overline{D}.\overline{A} + \overline{C}.\overline{A} + \overline{D}.\overline{C} + D.C.A + \overline{C}.B + \dots$ còn lại 1 ô x_5 chưa tô

BA \ DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_{13}	x_{15}	
10	x_8		x_{14}	x_9

Bảng 83. Kết quả tô biểu đồ Karnaugh sau bước 3.1.2

Bước 3.1.2.1:

Còn 1 ô x_5 chưa tô, ta chọn L4-2 = $\{x_0, x_1, x_4, x_5\} = \bar{D}.\bar{B}$ để tô xong $\Rightarrow x = \bar{D}.\bar{A} + \bar{C}.\bar{A} + \bar{D}.\bar{C} + D.C.A + \bar{C}.B + \bar{D}.\bar{B}$ (**Công thức 3**)

BA \ DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_{13}	x_{15}	
10	x_8		x_{14}	x_9

Bảng 84. Kết quả tô biểu đồ Karnaugh sau bước 3.1.2.1

Bước 3.1.2.2: (phản đề của 3.1.2.1)

Để tô ô x_5 mà không chọn L4-2 thì bắt buộc phải dùng L2-1 = $\{x_5, x_{13}\} = C.\bar{B}.A$ để tô xong $\Rightarrow x = \bar{D}.\bar{A} + \bar{C}.\bar{A} + \bar{D}.\bar{C} + D.C.A + \bar{C}.B + C.\bar{B}.A$ (**Công thức 4**)

BA \ DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_{13}	x_{15}	
10	x_8		x_{14}	x_9

Bảng 85. Kết quả tô biểu đồ Karnaugh sau bước 3.1.2.2

Bước 3.2: (phản đề của 3.1)

Để tô được hai ô x_1, x_3 mà không chọn L4-1 thì bắt buộc phải chọn L4-2 = $\{x_0, x_1, x_4, x_5\} = \overline{D} \cdot \overline{B}$ và L4-4 = $\{x_2, x_3, x_{10}, x_{11}\} = \overline{C} \cdot B \Rightarrow x = \overline{D} \cdot \overline{A} + \overline{C} \cdot \overline{A} + \overline{D} \cdot \overline{B} + \overline{C} \cdot B + \dots$ còn lại hai ô x_{13}, x_{15} chưa tô.

BA \ DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_{13}	x_{15}	
10	x_8		x_{11}	x_{14}

Bảng 86. Kết quả tô biểu đồ Karnaugh sau bước 3.2

Bước 3.2.1:

Để tô hai ô x_{13}, x_{15} còn lại, ta chọn L2-2 = $\{x_{13}, x_{15}\} = D.C.A$ để tô xong $\Rightarrow x = \overline{D} \cdot \overline{A} + \overline{C} \cdot \overline{A} + \overline{D} \cdot \overline{B} + \overline{C} \cdot B + D.C.A$ (**Công thức 5**)

BA \ DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5		x_6
11		x_{13}	x_{15}	
10	x_8		x_{11}	x_{14}

Bảng 87. Kết quả tô biểu đồ Karnaugh sau bước 3.2.1

Bước 3.2.2 (phản đề của 3.2.1)

Để tô hai ô x_{13}, x_{15} còn lại, mà không chọn L2-2 = $\{x_{13}, x_{15}\} = D.C.A$ thì phải chọn L2-1 = $\{x_5, x_{13}\} = C \cdot \overline{B} \cdot A$ và L2-3 = $\{x_{11}, x_{15}\} = D.B.A$ để tô xong $\Rightarrow x = \overline{D} \cdot \overline{A} + \overline{C} \cdot \overline{A} + \overline{D} \cdot \overline{B} + \overline{C} \cdot B + C \cdot \overline{B} \cdot A + D.B.A$ (**Công thức 6**)

BA \ DC	00	01	11	10
00	X_0	X_1	X_3	X_2
01	X_4	X_5		X_6
11		X_7	X_8	
10	X_9		X_{11}	X_{10}

Bảng 88. Kết quả tô biểu đồ Karnaugh sau bước 3.2.2

Bước 4:

Sau bước 3 ta có sáu công thức:

$$x = \overline{D}.\overline{A} + \overline{C}.\overline{A} + \overline{D}.\overline{C} + C.B.A + D.\overline{B}.A \quad (\text{Công thức 1})$$

$$x = \overline{D}.\overline{A} + \overline{C}.\overline{A} + \overline{D}.\overline{C} + C.B.A + \overline{D}.\overline{B} + D.C.A \quad (\text{Công thức 2})$$

$$x = \overline{D}.\overline{A} + \overline{C}.\overline{A} + \overline{D}.\overline{C} + D.C.A + \overline{C}.B + \overline{D}.\overline{B} \quad (\text{Công thức 3})$$

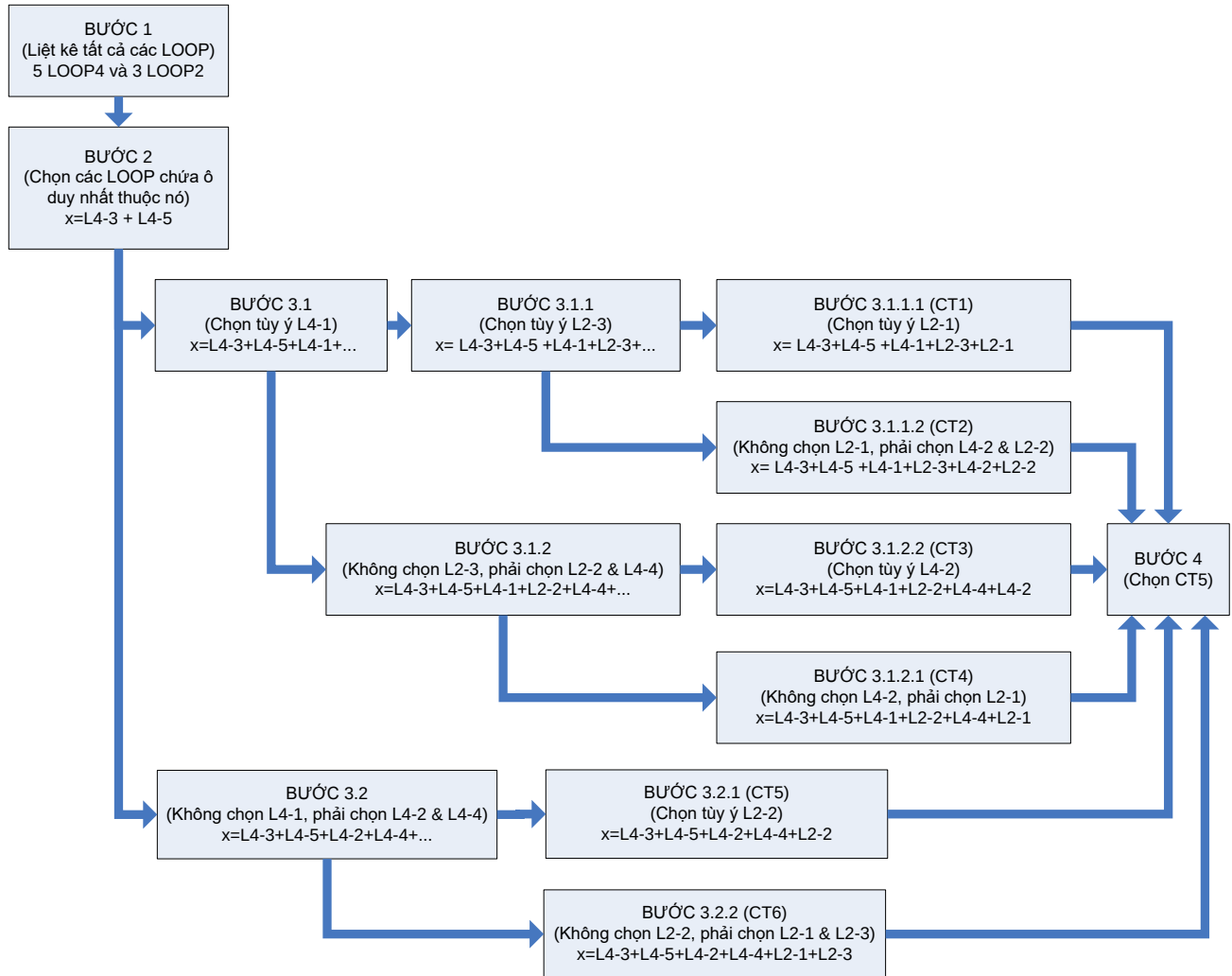
$$x = \overline{D}.\overline{A} + \overline{C}.\overline{A} + \overline{D}.\overline{C} + D.C.A + \overline{C}.B + C.\overline{B}.A \quad (\text{Công thức 4})$$

$$x = \overline{D}.\overline{A} + \overline{C}.\overline{A} + \overline{D}.\overline{B} + \overline{C}.B + D.C.A \quad (\text{Công thức 5})$$

$$x = \overline{D}.\overline{A} + \overline{C}.\overline{A} + \overline{D}.\overline{B} + \overline{C}.B + C.\overline{B}.A + D.B.A \quad (\text{Công thức 6})$$

Ta chọn công thức 5 vì đó công thức rút gọn nhất.

- Giải thuật dùng biểu đồ Karnaugh cho hàm 4 biến nói trên được minh họa bằng lưu đồ sau:

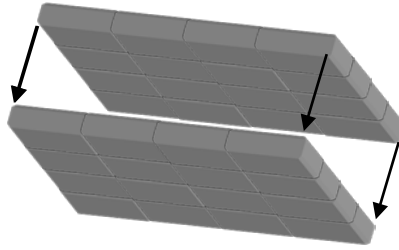


Hình 79. Lưu đồ cho giải thuật rút gọn hàm 4 biến đã cho bằng biểu đồ Karnaugh.

3.12.5- Biểu đồ Karnaugh cho hàm 5 biến $x = f(E, D, C, B, A)$ và hàm 6 biến $x = f(F, E, D, C, B, A)$

3.12.5.1- Biểu đồ Karnaugh cho hàm 5 biến

- Ta có thể mở rộng phạm vi áp dụng biểu đồ Karnaugh lên đến hàm 5 biến và hàm 6 biến. Biểu đồ Karnaugh cho hàm 5 biến là hai biểu đồ Karnaugh cho hàm 4 biến chồng lên nhau, gồm 32 ô. Như vậy, mỗi ô sẽ kề với 5 ô như hình vẽ dưới:



Bảng 89. Biểu đồ Karnaugh cho hàm 5 biến

Trong mặt phẳng, ta lật biểu đồ Karnaugh của hàm 5 biến thành hai biểu đồ Karnaugh cho hàm 4 biến nằm cạnh nhau như hình vẽ:

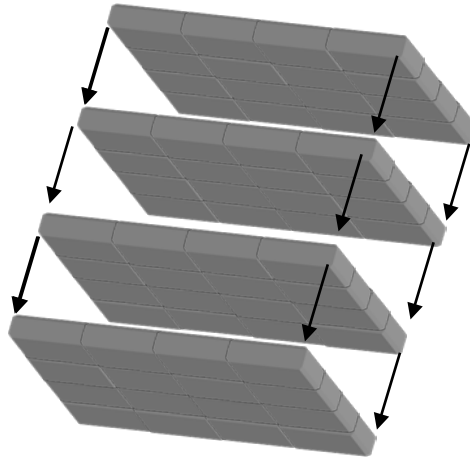
E				$\bar{E}$				
$B.\bar{A}$	$B.A$	$\bar{B}.A$	$\bar{B}.\bar{A}$	$\bar{B}.\bar{A}$	$\bar{B}.A$	$B.A$	$B.\bar{A}$	
$\bar{D}.\bar{C}$	x_{18}	x_{19}	x_{17}	x_{16}	x_0	x_1	x_3	x_2 $\bar{D}.\bar{C}$
$\bar{D}.C$	x_{22}	x_{23}	x_{21}	x_{20}	x_4	x_5	x_7	x_6 $\bar{D}.C$
$D.C$	x_{30}	x_{31}	x_{29}	x_{28}	x_{12}	x_{13}	x_{15}	x_{14} $D.C$
$D.\bar{C}$	x_{26}	x_{27}	x_{25}	x_{24}	x_8	x_9	x_{11}	x_{10} $D.\bar{C}$

Bảng 90. Biểu đồ Karnaugh cho hàm 5 biến trên mặt phẳng

- Khi chiếu lên mặt phẳng, thì chúng ta có hai biểu đồ Karnaugh đối xứng gương qua trục giữa, những ô kề nhau thuộc bên E hay $\bar{E}$ thì cách tính không có gì thay đổi. Riêng những ô kề nhau thuộc bảng bên kia thì lưu ý là đối xứng gương như ô x_0 sẽ kề ô x_{16} , ô x_5 kề với ô x_{21} , x_{15} kề với ô x_{31} , x_{10} kề với x_{26}
- Vậy 8 ô có giá trị $x_i = 1$ trên cùng một hàng thì rút gọn được 3 biến, 16 ô có giá trị $x_i = 1$ trên hai hàng kề nhau thì rút gọn được 4 biến, 8 ô thuộc hai cột đối xứng gương qua trục giữa có giá trị $x_i = 1$ sẽ rút gọn được 3 biến, 16 ô thuộc 4 cột kề nhau sẽ rút gọn được 4 biến

3.12.5.2- Biểu đồ Karnaugh cho hàm 6 biến là:

- Biểu đồ Karnaugh cho hàm 6 biến là một hình lập phương gồm 4 biểu đồ Karnaugh cho hàm 4 biến chồng lên nhau, gồm 64 ô. Mỗi ô sẽ kề với 6 ô như hình vẽ:



Bảng 91. Biểu đồ Karnaugh cho hàm 6 biến

Trong mặt phẳng, ta lật biểu đồ Karnaugh của hàm 6 biến thành bốn biểu đồ Karnaugh cho hàm 4 biến nằm cạnh nhau như hình vẽ:

$\overline{F}.E$				$\overline{F}.\overline{E}$					
$B.\overline{A}$	$B.A$	$\overline{B}.A$	$\overline{B}.\overline{A}$	$\overline{B}.\overline{A}$	$\overline{B}.A$	$B.A$	$B.\overline{A}$		
$\overline{D}.\overline{C}$	x_{18}	x_{19}	x_{17}	x_{16}	x_0	x_1	x_3	x_2	$\overline{D}.\overline{C}$
$\overline{D}.C$	x_{22}	x_{23}	x_{21}	x_{20}	x_4	x_5	x_7	x_6	$\overline{D}.C$
$D.C$	x_{30}	x_{31}	x_{29}	x_{28}	x_{12}	x_{13}	x_{15}	x_{14}	$D.C$
$D.\overline{C}$	x_{26}	x_{27}	x_{25}	x_{24}	x_8	x_9	x_{11}	x_{10}	$D.\overline{C}$
$D.\overline{C}$	x_{58}	x_{59}	x_{57}	x_{56}	x_{40}	x_{41}	x_{43}	x_{42}	$D.\overline{C}$
$D.C$	x_{62}	x_{63}	x_{61}	x_{60}	x_{44}	x_{45}	x_{47}	x_{46}	$D.C$
$\overline{D}.C$	x_{54}	x_{55}	x_{53}	x_{52}	x_{36}	x_{37}	x_{39}	x_{38}	$\overline{D}.C$
$\overline{D}.\overline{C}$	x_{50}	x_{51}	x_{49}	x_{48}	x_{32}	x_{33}	x_{35}	x_{34}	$\overline{D}.\overline{C}$
$B.\overline{A}$	$B.A$	$\overline{B}.A$	$\overline{B}.\overline{A}$	$\overline{B}.\overline{A}$	$\overline{B}.A$	$B.A$	$B.\overline{A}$		
$F.E$				$F.\overline{E}$					

Bảng 92. Biểu đồ Karnaugh cho hàm 6 biến trên mặt phẳng

- Cách tính các ô kề nhau cũng giống như biểu đồ Karnaugh hàm 5 biến, lưu ý đến việc đối xứng gương qua các trục và đối xứng qua tâm tọa độ
- Giải thuật rút gọn cho biểu đồ Karnaugh của hàm 5 và 6 biến cũng tương tự như hàm 4 biến nêu trên.

3.13- Rút Gọn Một Biểu Thức Bằng Phương Pháp Biểu Đồ Karnaugh – Dạng Chuẩn Hội

- Về nguyên tắc, thì rút gọn một biểu thức logic bằng phương pháp biểu đồ Karnaugh dạng chuẩn hội cũng giống như dạng chuẩn tuyển, điều khác biệt duy nhất là tính những giá trị $x_i = 0$ thay vì chọn $x_i = 1$
- Để tổng quát cách xây dựng biểu đồ Karnaugh, ta đưa ra một số nguyên tắc sau cho biểu đồ Karnaugh 4 biến dạng chuẩn hội:
 - Biểu đồ Karnaugh cho hàm 4 biến là 1 hình vuông có 16 ô.
 - Mỗi ô ứng với 1 dòng trong bảng thực trị (tích của 4 biến D, C, B, A và x_i).
 - Mỗi ô nằm kế cận 4 ô, như ô x_5 kế 4 ô x_1, x_4, x_7, x_{13} , ô x_6 kế với 4 ô x_2, x_4, x_7, x_{14} , ô x_0 kế 4 ô $x_1, x_4, x_8, x_{10}, \dots$
 - Hai ô cạnh nhau chỉ khác nhau duy nhất 1 bit.
 - LOOP là tập hợp các ô có giá trị $x_i = 0$ nằm cạnh nhau theo nguyên tắc 2^n , với $n = 0, 1, 2, 3, 4 \Rightarrow \text{LOOP1, LOOP2, LOOP4, LOOP8, LOOP16}$.
 - Ô nằm riêng lẻ một mình có giá trị $x_i = 0$, khoanh chúng lại thành một **vòng** gồm 1 ô, được gọi là LOOP1, không rút gọn được biến nào.
 - Hai ô nằm cạnh nhau có giá trị là $x_i = 0$, khoanh chúng lại thành một **vòng** gồm 2 ô, được gọi là LOOP2, thì rút gọn được một biến.
 - Bốn ô nằm cạnh nhau có giá trị là $x_i = 0$, khoanh chúng lại thành một **vòng** gồm 4 ô, được gọi là LOOP4, thì rút gọn được hai biến. Trong biểu đồ Karnaugh 4 biến, LOOP4 là 4 ô tạo

thành hình vuông, hay tạo thành một cột, một hàng hay 4 ô ở 4 góc.

- Tám ô nằm cạnh nhau có giá trị là $x_i = 0$, khoanh chúng lại thành một **vòng** gồm 8 ô, được gọi là LOOP8, thì rút gọn được ba biến. Trong biểu đồ Karnaugh 4 biến, LOOP8 là 8 ô tạo thành hai hàng hay hai cột.
- Mười sáu ô nằm cạnh nhau có giá trị là $x_i = 0$, khoanh chúng lại thành một **vòng** gồm 16 ô, được gọi là LOOP16, thì rút gọn được bốn biến.
- Cách biểu diễn biểu đồ Karnaugh cho hàm 4 biến dùng trong dạng chuẩn hội

BA \ DC	00	01	11	10
00	x_0	x_1	x_3	x_2
01	x_4	x_5	x_7	x_6
11	x_{12}	x_{13}	x_{15}	x_{14}
10	x_8	x_9	x_{11}	x_{10}

Bảng 93. Cách biểu diễn biểu đồ Karnaugh dùng trong tài liệu

- Và để cho gọn, những ô có giá trị $x_i = 1$ sẽ để trống, chỉ đưa vào những ô có giá trị $x_i = 0$.
- Hàng số 1 là $D + C$, hàng số 2 là $D + \bar{C}$, hàng số 3 là $\bar{D} + \bar{C}$, hàng số 4 là $\bar{D} + C$.
- Cột số 1 là $B + A$, cột số 2 là $B + \bar{A}$, cột số 3 là $\bar{B} + \bar{A}$, cột số 4 là $\bar{B} + A$.
- Phương pháp rút gọn giống như trong trường hợp dạng chuẩn tuyển

3.14- Rút Gọn Một Biểu Thức Bằng Phương Pháp Biểu Đồ Karnaugh – Có Những Giá Trị Của Hàm Là Tùy Chọn

- Trong trường hợp biểu đồ Karnaugh có 1 giá trị x_i là tùy chọn, có nghĩa là chọn $x_i = 0$ hay chọn $x_i = 1$ đều được, thì ta phải phân tích thành hai biểu đồ Karnaugh, một biểu đồ Karnaugh ứng với $x_i = 0$ và một biểu đồ Karnaugh ứng với $x_i = 1$. Nếu có hai giá trị x_i, x_j là tùy chọn thì ta lập thành 4 biểu đồ Karnaugh ứng với các cặp $x_i, x_j = 0,0$ hay $x_i, x_j = 0,1$ hay $x_i, x_j = 1,0$ hay $x_i, x_j = 1,1$. Nếu có N giá trị x_i là tùy chọn thì ta sẽ có 2^N biểu đồ Karnaugh để rút gọn.
- Sau khi tách ra các trường hợp, thì ta rút gọn từng trường hợp một để tìm ra công thức rút gọn nhất.

